

HelixTerminator — 10_submodule_integration

HelixTerminator — Submodule Integration Specification

Document: 10_submodule_integration.md

Module path: helixterm.io

Go version: 1.25

Last updated: 2026-06-28

Status: Authoritative

Table of Contents

1. [Overview](#)
 2. [digital.vasic.security](#) — AES-256 Encryption & Keychain
 3. [digital.vasic.auth](#) — OAuth2 & Token Management
 4. [digital.vasic.cache](#) — Redis L1/L2 Caching
 5. [digital.vasic.database](#) — Database Abstraction
 6. [digital.vasic.messaging](#) — Kafka + RabbitMQ
 7. [digital.vasic.middleware](#) — Gin Middleware
 8. [digital.vasic.observability](#) — Prometheus + OpenTelemetry
 9. [digital.vasic.ratelimiter](#) — Rate Limiting
 10. [digital.vasic.recovery](#) — Circuit Breakers & Fault Tolerance
 11. [digital.vasic.concurrency](#) — Concurrency Utilities
 12. [digital.vasic.containers](#) — ContainerRuntime Abstraction
 13. [digital.vasic.docs_chain](#) — Salsa-style DAG Document Engine
 14. [digital.vasic.challenges](#) — Challenges Submodule
 15. [helixqa](#) — AI-driven QA Orchestration
 16. [helixtrack.ru/core](#) — Project Management Integration
 17. [HelixConstitution](#) — AGENTS.MD, CLAUDE.MD, Constitution.md
 18. [Appendix A: go.work](#) Workspace File
 19. [Appendix B: helix-deps.yaml](#)
 20. [Appendix C: Makefile Submodule Targets](#)
 21. [Appendix D: GitHub Actions — Submodule Compliance](#)
 22. [Appendix E: Dependency Graph \(Mermaid\)](#)
-

1. Overview

HelixTerminator (`helixterm.io`) is a 25-microservice platform built on Go 1.25, Gin Gonic, Kafka, RabbitMQ, PostgreSQL, and Redis, deployed on Kubernetes. The platform is composed of first-party services under `helixterm.io/services/<name>` and a Flutter/Dart client at package `io.helixterm.client` .

This specification is the authoritative reference for integrating every external submodule into HelixTerminator. Each section covers:

- **Purpose** of the submodule within HelixTerminator
- **Integration points** across services
- **Complete, compilable Go code** (or Dart code where applicable)
- **Configuration** artifacts (YAML, env variables)
- **Error handling** and operational concerns

1.1 Submodule Inventory

#	Submodule	Origin	Scope
1	<code>digital.vasic.security</code>	vasic-digital	All services, Flutter client
2	<code>digital.vasic.auth</code>	vasic-digital	Auth, Gateway, all 25 services
3	<code>digital.vasic.cache</code>	vasic-digital	All services
4	<code>digital.vasic.database</code>	vasic-digital	All services
5	<code>digital.vasic.messaging</code>	vasic-digital	Event-driven services
6	<code>digital.vasic.middleware</code>	vasic-digital	Gateway, all Gin routers
7	<code>digital.vasic.observability</code>	vasic-digital	All 25 services
8	<code>digital.vasic.ratelimiter</code>	vasic-digital	Gateway, SSH Proxy, Auth
9	<code>digital.vasic.recovery</code>	vasic-digital	All inter-service calls
10	<code>digital.vasic.concurrency</code>	vasic-digital	SSH Proxy, Vault, Terminal
11	<code>digital.vasic.containers</code>	vasic-digital	

#	Submodule	Origin	Scope
			Container Bridge, SSH Proxy
12	<code>digital.vasic.docs_chain</code>	vasic-digital	Documentation CI
13	<code>digital.vasic.challenges</code>	vasic-digital	AI Service, User Service
14	<code>helixqa</code>	HelixDevelopment	QA, CI
15	<code>helixtrack.ru/core</code>	Helix-Track	HelixTrack Bridge Service
16	HelixConstitution	HelixDevelopment	Entire codebase governance

1.2 Service Registry

All 25 microservices under `helixterm.io/services/` :

```

helixterm.io/services/api-gateway
helixterm.io/services/auth
helixterm.io/services/vault
helixterm.io/services/ssh-proxy
helixterm.io/services/terminal
helixterm.io/services/sftp
helixterm.io/services/host-manager
helixterm.io/services/user
helixterm.io/services/workspace
helixterm.io/services/notification
helixterm.io/services/audit
helixterm.io/services/analytics
helixterm.io/services/ai
helixterm.io/services/container-bridge
helixterm.io/services/helixtrack-bridge
helixterm.io/services/billing
helixterm.io/services/scheduler
helixterm.io/services/file-manager
helixterm.io/services/config
helixterm.io/services/identity
helixterm.io/services/team
helixterm.io/services/secret
helixterm.io/services/webhook

```

```
helixterm.io/services/search
helixterm.io/services/onboarding
```

1.3 Go Module Layout

```
helixterm.io/
├── go.work
├── helix-deps.yaml
├── AGENTS.MD
├── CLAUDE.MD
├── Makefile
├── services/
│   ├── api-gateway/
│   │   └── go.mod          (module helixterm.io/services/api-
│   │   gateway)
│   ├── ...
│   ├── auth/
│   │   └── go.mod          (module helixterm.io/services/
│   │   auth)
│   ├── ...
│   └── ... (22 more services)
├── pkg/
│   └── shared/             (helixterm.io/pkg/shared)
└── docs/
```

2. digital.vasic.security

Import path: `digital.vasic/security`

Go module: `digital.vasic/security v1.x.x`

2.1 Purpose

`digital.vasic.security` provides AES-256-GCM encryption, PBKDF2/Argon2id key derivation, and a unified platform keychain/keystore abstraction. Within HelixTerminator it is the single encryption boundary for:

- Vault items stored in PostgreSQL (at-rest encryption)
- SSH private keys stored in PostgreSQL
- Session tokens cached in Redis
- Platform keychain integration on iOS, Android, macOS, Windows, and Linux

2.2 Go: Module Initialization

Every service that performs encryption imports and initialises the security module once at startup via a singleton:

```

// File: helixterm.io/services/vault/internal/crypto/
//      crypto.go
package crypto

import (
    "context"
    "fmt"
    "os"

    "digital.vasic/security"
    "digital.vasic/security/keyderivation"
    "digital.vasic/security/keystore"
    "go.uber.org/zap"
)

// Manager wraps digital.vasic.security for the Vault
//      Service.
type Manager struct {
    enc    security.Encryptor
    kd     keyderivation.Deriver
    ks     keystore.Store
    logger *zap.Logger
}

// Config holds security module configuration sourced from
//      environment.
type Config struct {
    MasterKeyID string // identifier for the current master
    //      key
    KDFAlgorithm string // "argon2id" | "pbkdf2-sha512"
    Argon2Memory uint32 // memory parameter for Argon2id (KB)
    Argon2Time    uint32 // time parameter for Argon2id
    Argon2Lanes   uint8  // parallelism parameter for Argon2id
}

// ConfigFromEnv reads security config from environment
//      variables.
func ConfigFromEnv() Config {
    return Config{
        MasterKeyID: mustEnv("HELIX_MASTER_KEY_ID"),
        KDFAlgorithm: envOrDefault("HELIX_KDF_ALGO",
            "argon2id"),
        Argon2Memory: 65536, // 64 MiB
        Argon2Time:    3,
        Argon2Lanes:   4,
    }
}

func mustEnv(key string) string {
    v := os.Getenv(key)

```

```

    if v == "" {
        panic(fmt.Sprintf("required environment variable %q
            is not set", key))
    }
    return v
}

func envOrDefault(key, def string) string {
    if v := os.Getenv(key); v != "" {
        return v
    }
    return def
}

// NewManager initialises the security manager. Must be
// called once during
// service startup before any encryption operations.
func NewManager(ctx context.Context, cfg Config, logger
    *zap.Logger) (*Manager, error) {
    // Initialise key derivation
    var kd keyderivation.Deriver
    var err error
    switch cfg.KDFAlgorithm {
    case "argon2id":
        kd, err =
            keyderivation.NewArgon2id(keyderivation.Argon2idParams{
                Memory:      cfg.Argon2Memory,
                Iterations:  cfg.Argon2Time,
                Parallelism:  cfg.Argon2Lanes,
                KeyLength:    32, // 256-bit key
            })
    case "pbkdf2-sha512":
        kd, err =
            keyderivation.NewPBKDF2(keyderivation.PBKDF2Params{
                Iterations: 600_000,
                KeyLength:   32,
                Hash:        "sha512",
            })
    default:
        return nil, fmt.Errorf("crypto: unsupported KDF
            algorithm %q", cfg.KDFAlgorithm)
    }
    if err != nil {
        return nil, fmt.Errorf("crypto: initialising KDF:
            %w", err)
    }

    // Initialise AES-256-GCM encryptor with the current
    // master key.
    enc, err := security.NewAES256GCM(ctx,
        security.AES256GCMConfig{
            MasterKeyID: cfg.MasterKeyID,

```

```

    })
    if err != nil {
        return nil, fmt.Errorf("crypto: initialising AES-256-
            GCM encryptor: %w", err)
    }

    // Initialise keystore (platform-specific on client; on
        server uses HashiCorp Vault).
    ks, err := keystore.NewServerKeystore(ctx,
        keystore.ServerConfig{
            Backend: keystore.BackendVaultAgent,
            Path:    os.Getenv("VAULT_AGENT_PATH"),
        })
    if err != nil {
        return nil, fmt.Errorf("crypto: initialising
            keystore: %w", err)
    }

    logger.Info("security manager initialised",
        zap.String("kdf", cfg.KDFAlgorithm),
        zap.String("master_key_id", cfg.MasterKeyID),
    )

    return &Manager{
        enc:    enc,
        kd:     kd,
        ks:     ks,
        logger: logger,
    }, nil
}

```

2.3 Go: Encrypting and Decrypting Vault Payloads

```

// File: helixterm.io/services/vault/internal/crypto/
    vault_payload.go
package crypto

import (
    "context"
    "encoding/json"
    "fmt"

    "digital.vasic/security"
)

// VaultPayload represents a vault item before persistence.
type VaultPayload struct {
    ItemID    string    `json:"item_id"`
    Type      string    `json:"type"`
}

```

```

    Fields    json.RawMessage `json:"fields"`
    Metadata  map[string]string `json:"metadata,omitempty"`
}

// EncryptedBlob is the ciphertext envelope persisted in
// PostgreSQL.
type EncryptedBlob struct {
    KeyID      string `json:"key_id"`
    Algorithm   string `json:"algorithm"`
    Nonce       []byte `json:"nonce"`
    Ciphertext []byte `json:"ciphertext"`
    AAD         []byte `json:"aad,omitempty"`
}

// EncryptVaultPayload serialises and encrypts a
// VaultPayload.
// The item_id is used as Additional Authenticated Data (AAD)
// to bind
// the ciphertext to the item's identity, preventing
// ciphertext transplanting.
func (m *Manager) EncryptVaultPayload(ctx context.Context,
    payload VaultPayload) (*EncryptedBlob, error) {
    plaintext, err := json.Marshal(payload)
    if err != nil {
        return nil, fmt.Errorf("crypto: marshalling vault
            payload: %w", err)
    }

    aad := []byte(payload.ItemID)

    result, err := m.enc.Encrypt(ctx,
        security.EncryptRequest{
            Plaintext: plaintext,
            AAD:       aad,
        })
    if err != nil {
        return nil, fmt.Errorf("crypto: encrypting vault
            payload for item %q: %w", payload.ItemID, err)
    }

    return &EncryptedBlob{
        KeyID:      result.KeyID,
        Algorithm:   "AES-256-GCM",
        Nonce:       result.Nonce,
        Ciphertext:  result.Ciphertext,
        AAD:         aad,
    }, nil
}

// DecryptVaultPayload decrypts an EncryptedBlob back to a
// VaultPayload.

```

```

func (m *Manager) DecryptVaultPayload(ctx context.Context,
    blob *EncryptedBlob) (*VaultPayload, error) {
    result, err := m.enc.Decrypt(ctx,
        security.DecryptRequest{
            KeyID:      blob.KeyID,
            Nonce:      blob.Nonce,
            Ciphertext: blob.Ciphertext,
            AAD:        blob.AAD,
        })
    if err != nil {
        return nil, fmt.Errorf("crypto: decrypting vault
            payload: %w", err)
    }

    var payload VaultPayload
    if err := json.Unmarshal(result.Plaintext, &payload);
        err != nil {
        return nil, fmt.Errorf("crypto: unmarshalling
            decrypted vault payload: %w", err)
    }
    return &payload, nil
}

// DeriveItemKey derives a per-item encryption key using
// Argon2id.
// This implements envelope encryption: each vault item has a
// unique DEK
// (Data Encryption Key) wrapped by the master KEK.
func (m *Manager) DeriveItemKey(ctx context.Context, itemID,
    userSecret string) ([]byte, error) {
    salt := []byte("helixterm:vault:item:" + itemID)
    key, err := m.kd.Derive(ctx, []byte(userSecret), salt)
    if err != nil {
        return nil,
            fmt.Errorf("crypto: deriving item key for %q: %w",
                itemID, err)
    }
    return key, nil
}

```

2.4 Go: Key Rotation Orchestration

HelixTerminator's key rotation orchestrator lives in the `vault` service and coordinates re-encryption of all vault items when a new master key is introduced.

```

// File: helixterm.io/services/vault/internal/rotation/
// rotation.go
package rotation

import (

```

```

    "context"
    "fmt"
    "time"

    "digital.vasic/security"
    "go.uber.org/zap"

    "helixterm.io/services/vault/internal/crypto"
    "helixterm.io/services/vault/internal/repository"
)

// Rotator orchestrates AES-256 master key rotation for all
// vault items.
type Rotator struct {
    crypto      *crypto.Manager
    repo        repository.VaultRepository
    newKeyID    string
    batchSize   int
    logger      *zap.Logger
}

// NewRotator constructs a Rotator.
func NewRotator(
    cm *crypto.Manager,
    repo repository.VaultRepository,
    newKeyID string,
    logger *zap.Logger,
) *Rotator {
    return &Rotator{
        crypto:    cm,
        repo:      repo,
        newKeyID:  newKeyID,
        batchSize: 100,
        logger:    logger,
    }
}

// RotateAll iterates all encrypted vault items and re-
// encrypts them with
// the new master key. Uses batched reads to avoid OOM on
// large vaults.
func (r *Rotator) RotateAll(ctx context.Context) error {
    r.logger.Info("starting key rotation",
        zap.String("new_key_id", r.newKeyID))
    start := time.Now()

    var cursor string
    total := 0

    for {
        items, nextCursor, err :=
            r.repo.ListEncryptedItems(ctx, cursor, r.batchSize)
    }

```

```

        if err != nil {
            return fmt.Errorf("rotation: listing items at
cursor %q: %w", cursor, err)
        }

        for _, item := range items {
            if err := r.rotateItem(ctx, item); err != nil {
                // Log and continue – partial rotation is
                recoverable.
                r.logger.Error("rotation: failed to rotate
item",
                    zap.String("item_id", item.ID),
                    zap.Error(err),
                )
                continue
            }
            total++
        }

        if nextCursor == "" {
            break
        }
        cursor = nextCursor
    }

    r.logger.Info("key rotation complete",
        zap.Int("total_rotated", total),
        zap.Duration("elapsed", time.Since(start)),
    )
    return nil
}

// rotateItem decrypts with the old key and re-encrypts with
// the new key.
func (r *Rotator) rotateItem(ctx context.Context, item
repository.EncryptedItem) error {
    // Decrypt using the key ID recorded in the blob (old
    key).
    payload, err := r.crypto.DecryptVaultPayload(ctx,
        &crypto.EncryptedBlob{
            KeyID:      item.KeyID,
            Nonce:      item.Nonce,
            Ciphertext: item.Ciphertext,
            AAD:        item.AAD,
        })
    if err != nil {
        return fmt.Errorf("rotation: decrypting item %q:
%w", item.ID, err)
    }

    // Re-encrypt with the new master key by temporarily
    overriding the encryptor.

```

```

    newBlob, err := r.crypto.EncryptWithKey(ctx, r.newKeyID,
        *payload)
    if err != nil {
        return fmt.Errorf("rotation: re-encrypting item %q
            with new key: %w", item.ID, err)
    }

    // Persist the new blob atomically.
    if err := r.repo.UpdateEncryptedBlob(ctx, item.ID,
        newBlob); err != nil {
        return fmt.Errorf("rotation: persisting rotated blob
            for item %q: %w", item.ID, err)
    }
    return nil
}

```

2.5 Go: SSH Key Encryption in the SSH Key Service

```

// File: helixterm.io/services/vault/internal/handler/
//       ssh_key_handler.go
package handler

import (
    "net/http"

    "github.com/gin-gonic/gin"
    "go.uber.org/zap"

    "digital.vasic/security"
    "helixterm.io/services/vault/internal/crypto"
    "helixterm.io/services/vault/internal/repository"
    "helixterm.io/services/vault/internal/model"
)

// SSHKeyHandler handles storage and retrieval of encrypted
// SSH private keys.
type SSHKeyHandler struct {
    crypto *crypto.Manager
    repo   repository.SSHKeyRepository
    logger *zap.Logger
}

// StoreSSHKey encrypts an SSH private key and stores it in
// PostgreSQL.
// POST /v1/vault/ssh-keys
func (h *SSHKeyHandler) StoreSSHKey(c *gin.Context) {
    var req model.StoreSSHKeyRequest
    if err := c.ShouldBindJSON(&req); err != nil {

```

```

        c.JSON(http.StatusBadRequest, gin.H{"error":
            "invalid request: " + err.Error()})
        return
    }

    ctx := c.Request.Context()

    // Build the payload that wraps the private key material.
    payload := crypto.VaultPayload{
        ItemID: req.KeyID,
        Type:    "ssh_private_key",
        Fields: marshalSSHKeyFields(req.PrivateKeyPEM,
            req.Passphrase),
        Metadata: map[string]string{
            "algorithm": req.Algorithm,
            "fingerprint": req.Fingerprint,
            "owner_id": req.OwnerID,
        },
    }

    blob, err := h.crypto.EncryptVaultPayload(ctx, payload)
    if err != nil {
        h.logger.Error("failed to encrypt SSH key",
            zap.String("key_id", req.KeyID), zap.Error(err))
        c.JSON(http.StatusInternalServerError,
            gin.H{"error": "encryption failed"})
        return
    }

    if err := h.repo.Store(ctx, repository.SSHKeyRecord{
        ID: req.KeyID,
        OwnerID: req.OwnerID,
        KeyID: blob.KeyID,
        Nonce: blob.Nonce,
        Ciphertext: blob.Ciphertext,
        AAD: blob.AAD,
    }); err != nil {
        h.logger.Error("failed to store SSH key",
            zap.String("key_id", req.KeyID), zap.Error(err))
        c.JSON(http.StatusInternalServerError,
            gin.H{"error": "storage failed"})
        return
    }

    c.JSON(http.StatusCreated, gin.H{"key_id": req.KeyID,
        "fingerprint": req.Fingerprint})
}

// RetrieveSSHKey decrypts and returns an SSH private key.
// GET /v1/vault/ssh-keys/:key_id
func (h *SSHKeyHandler) RetrieveSSHKey(c *gin.Context) {
    keyID := c.Param("key_id")

```

```

    ctx := c.Request.Context()

    record, err := h.repo.Get(ctx, keyID)
    if err != nil {
        h.logger.Error("SSH key not found",
            zap.String("key_id", keyID), zap.Error(err))
        c.JSON(http.StatusNotFound, gin.H{"error": "key not
            found"})
        return
    }

    payload, err := h.crypto.DecryptVaultPayload(ctx,
        &crypto.EncryptedBlob{
            KeyID:      record.KeyID,
            Nonce:      record.Nonce,
            Ciphertext: record.Ciphertext,
            AAD:        record.AAD,
        })
    if err != nil {
        h.logger.Error("failed to decrypt SSH key",
            zap.String("key_id", keyID), zap.Error(err))
        c.JSON(http.StatusInternalServerError,
            gin.H{"error": "decryption failed"})
        return
    }

    c.JSON(http.StatusOK, payload.Fields)
}

func marshalSSHKeyFields(pem, passphrase string) []byte {
    import_json_raw, _ := json.Marshal(map[string]string{
        "private_key_pem": pem,
        "passphrase":      passphrase,
    })
    return import_json_raw
}

```

2.6 Dart/Flutter: Platform Keychain Integration

The Flutter client uses `digital.vasic.security` via a platform channel bridge. The Dart-side wrapper is in `io.helixterm.client.security`.

```

// File: lib/src/security/security_manager.dart
// Package: io.helixterm.client

import 'dart:typed_data';
import 'package:flutter/services.dart';
import 'package:vasic_security/vasic_security.dart';

```

```

/// [SecurityManager] wraps digital.vasic.security for
/// Flutter.
/// On iOS/macOS it delegates to the Keychain Services API.
/// On Android it delegates to the Android Keystore System.
/// On Windows/Linux it delegates to DPAPI / libsecret
/// respectively.
class SecurityManager {
  static const MethodChannel _channel =
    MethodChannel('io.helixterm.client/security');

  final VasicSecurityPlugin _plugin;

  SecurityManager({VasicSecurityPlugin? plugin})
    : _plugin = plugin ?? VasicSecurityPlugin.instance;

  /// Stores [value] in the platform keychain under [key].
  /// Uses the service label "io.helixterm.client" so all
  /// items
  /// are grouped under HelixTerminator in the OS keychain
  /// UI.
  Future<void> storeSecure({
    required String key,
    required String value,
  }) async {
    try {
      await _plugin.write(
        service: 'io.helixterm.client',
        account: key,
        data: value,
        accessibility:
          KeychainAccessibility.afterFirstUnlock,
      );
    } on PlatformException catch (e) {
      throw SecurityException(
        'Failed to store key "$key": ${e.message}',
        code: e.code,
      );
    }
  }

  /// Retrieves the value for [key] from the platform
  /// keychain.
  /// Returns null if the key does not exist.
  Future<String?> retrieveSecure({required String key})
    async {
    try {
      return await _plugin.read(
        service: 'io.helixterm.client',
        account: key,
      );
    } on PlatformException catch (e) {
      if (e.code == 'KEY_NOT_FOUND') return null;
    }
  }
}

```

```

        throw SecurityException(
            'Failed to retrieve key "$key": ${e.message}',
            code: e.code,
        );
    }
}

/// Deletes [key] from the platform keychain.
Future<void> deleteSecure({required String key}) async {
    try {
        await _plugin.delete(
            service: 'io.helixterm.client',
            account: key,
        );
    } on PlatformException catch (e) {
        throw SecurityException(
            'Failed to delete key "$key": ${e.message}',
            code: e.code,
        );
    }
}

/// Encrypts [plaintext] using the AES-256-GCM key stored
    in the keychain.
/// The key is retrieved once, used for encryption, then
    zeroed in memory.
Future<Uint8List> encryptClientSide({
    required String keyID,
    required Uint8List plaintext,
    Uint8List? aad,
}) async {
    final keyMaterial = await retrieveSecure(key: keyID);
    if (keyMaterial == null) {
        throw SecurityException('Key "$keyID" not found in
            keychain');
    }
    try {
        return await _plugin.encryptAES256GCM(
            keyBase64: keyMaterial,
            plaintext: plaintext,
            aad: aad,
        );
    } finally {
        // The plugin implementation zeros the key in its
            native buffer.
        await _plugin.zeroKey(keyID: keyID);
    }
}

/// Decrypts [ciphertext] using the keychain-stored
    AES-256-GCM key.
Future<Uint8List> decryptClientSide({

```

```

        required String keyID,
        required Uint8List ciphertext,
        Uint8List? aad,
    }) async {
        final keyMaterial = await retrieveSecure(key: keyID);
        if (keyMaterial == null) {
            throw SecurityException('Key "$keyID" not found in
                keychain');
        }
        try {
            return await _plugin.decryptAES256GCM(
                keyBase64: keyMaterial,
                ciphertext: ciphertext,
                aad: aad,
            );
        } finally {
            await _plugin.zeroKey(keyID: keyID);
        }
    }
}

/// Thrown when a security operation fails.
class SecurityException implements Exception {
    final String message;
    final String? code;

    const SecurityException(this.message, {this.code});

    @override
    String toString() => 'SecurityException($code): $message';
}

```

```

// File: lib/src/security/vault_client.dart
// Encrypts vault items before sending to the HelixTerminator
// API.

import 'dart:convert';
import 'dart:typed_data';
import 'package:vasic_security/vasic_security.dart';

import 'security_manager.dart';

class VaultClient {
    final SecurityManager _security;

    VaultClient(this._security);

    /// Encrypts a vault item locally before upload.
    /// This provides end-to-end encryption independent of TLS.

```

```

Future<Map<String, dynamic>> encryptVaultItem({
    required String itemID,
    required Map<String, dynamic> fields,
}) async {
    final plaintext = utf8.encode(jsonEncode(fields)) as
        Uint8List;
    final aad = utf8.encode(itemID) as Uint8List;

    final ciphertext = await _security.encryptClientSide(
        keyID: 'vault_master_key',
        plaintext: plaintext,
        aad: aad,
    );

    return {
        'item_id': itemID,
        'ciphertext': base64Encode(ciphertext),
        'aad': base64Encode(aad),
        'algorithm': 'AES-256-GCM',
    };
}

```

3. digital.vasic.auth

Import path: digital.vasic/auth

Go module: digital.vasic/auth v2.x.x

3.1 Purpose

digital.vasic.auth provides OAuth2/OIDC integration, JWT issuance and validation, refresh token rotation, SCIM 2.0 provisioning, and a unified identity provider adapter layer. Within HelixTerminator it underpins the Auth Service, the API Gateway's token validation middleware, and all 25 services' inter-service token propagation.

3.2 Go: Auth Service – JWT Issuance

```

// File: helixterm.io/services/auth/internal/handler/
//       token_handler.go
package handler

import (
    "net/http"
    "time"

    "github.com/gin-gonic/gin"

```

```

    "go.uber.org/zap"

    "digital.vasic/auth"
    "digital.vasic/auth/jwt"
    "digital.vasic/auth/oauth2"
    "helixterm.io/services/auth/internal/model"
    "helixterm.io/services/auth/internal/repository"
)

// TokenHandler manages JWT issuance and refresh token
rotation.
type TokenHandler struct {
    issuer      jwt.Issuer
    validator   jwt.Validator
    tokenRepo   repository.TokenRepository
    userRepo    repository.UserRepository
    logger      *zap.Logger
}

// TokenConfig holds JWT signing configuration.
type TokenConfig struct {
    Issuer      string
    AccessTTL   time.Duration
    RefreshTTL  time.Duration
    SigningAlgorithm string // "RS256" | "ES256" | "EdDSA"
    PrivateKeyPath string
}

// NewTokenHandler constructs a TokenHandler.
func NewTokenHandler(cfg TokenConfig, tokenRepo
    repository.TokenRepository, userRepo
    repository.UserRepository, logger *zap.Logger)
    (*TokenHandler, error) {
    issuer, err := jwt.NewIssuer(jwt.IssuerConfig{
        Issuer:      cfg.Issuer,
        Algorithm:   cfg.SigningAlgorithm,
        KeyPath:     cfg.PrivateKeyPath,
        AccessTTL:   cfg.AccessTTL,
    })
    if err != nil {
        return nil, fmt.Errorf("token_handler: creating JWT
            issuer: %w", err)
    }

    validator, err := jwt.NewValidator(jwt.ValidatorConfig{
        Issuer:      cfg.Issuer,
        Algorithm:   cfg.SigningAlgorithm,
        KeyPath:     cfg.PrivateKeyPath,
    })
    if err != nil {
        return nil, fmt.Errorf("token_handler: creating JWT
            validator: %w", err)
    }
}

```

```

    }

    return &TokenHandler{
        issuer:    issuer,
        validator: validator,
        tokenRepo: tokenRepo,
        userRepo:  userRepo,
        logger:    logger,
    }, nil
}

// IssueTokens creates an access token + refresh token pair.
// POST /v1/auth/token
func (h *TokenHandler) IssueTokens(c *gin.Context) {
    var req model.TokenRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error":
            err.Error()})
        return
    }

    ctx := c.Request.Context()

    user, err := h.userRepo.FindByCredentials(ctx,
        req.Username, req.Password)
    if err != nil {
        h.logger.Warn("invalid credentials",
            zap.String("username", req.Username))
        c.JSON(http.StatusUnauthorized, gin.H{"error":
            "invalid credentials"})
        return
    }

    claims := jwt.Claims{
        Subject:  user.ID,
        Audience: []string{"helixterm.io"},
        Issuer:   "auth.helixterm.io",
        ExpiresAt: time.Now().Add(15 * time.Minute),
        Extra: map[string]interface{}{
            "org_id":    user.OrgID,
            "roles":     user.Roles,
            "workspaces": user.WorkspaceIDs,
        },
    }

    accessToken, err := h.issuer.Sign(ctx, claims)
    if err != nil {
        h.logger.Error("failed to sign access token",
            zap.Error(err))
        c.JSON(http.StatusInternalServerError,
            gin.H{"error": "token issuance failed"})
        return
    }

```

```

    }

    refreshToken, err := auth.GenerateOpaqueToken(32)
    if err != nil {
        h.logger.Error("failed to generate refresh token",
            zap.Error(err))
        c.JSON(http.StatusInternalServerError,
            gin.H{"error": "token issuance failed"})
        return
    }

    if err := h.tokenRepo.StoreRefreshToken(ctx,
        repository.RefreshToken{
            Token:      refreshToken,
            UserID:     user.ID,
            ExpiresAt: time.Now().Add(30 * 24 * time.Hour),
            IPAddress: c.ClientIP(),
            UserAgent: c.GetHeader("User-Agent"),
        }); err != nil {
        h.logger.Error("failed to store refresh token",
            zap.Error(err))
        c.JSON(http.StatusInternalServerError,
            gin.H{"error": "token storage failed"})
        return
    }

    c.JSON(http.StatusOK, model.TokenResponse{
        AccessToken: accessToken,
        RefreshToken: refreshToken,
        TokenType:   "Bearer",
        ExpiresIn:   int(15 * time.Minute / time.Second),
    })
}

// RefreshTokens rotates a refresh token and issues a new
// access token.
// POST /v1/auth/token/refresh
func (h *TokenHandler) RefreshTokens(c *gin.Context) {
    var req model.RefreshRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error":
            err.Error()})
        return
    }

    ctx := c.Request.Context()

    existing, err := h.tokenRepo.GetRefreshToken(ctx,
        req.RefreshToken)
    if err != nil || existing.ExpiresAt.Before(time.Now()) {
        c.JSON(http.StatusUnauthorized, gin.H{"error":
            "invalid or expired refresh token"})
    }
}

```

```

        return
    }

    // Rotate: revoke the old token first (prevents replay
    // attacks).
    if err := h.tokenRepo.RevokeRefreshToken(ctx,
        req.RefreshToken); err != nil {
        h.logger.Error("failed to revoke old refresh token",
            zap.Error(err))
        c.JSON(http.StatusInternalServerError,
            gin.H{"error": "rotation failed"})
        return
    }

    user, err := h.userRepo.FindByID(ctx, existing.UserID)
    if err != nil {
        c.JSON(http.StatusUnauthorized, gin.H{"error": "user
            not found"})
        return
    }

    claims := jwt.Claims{
        Subject:  user.ID,
        Audience: []string{"helixterm.io"},
        Issuer:   "auth.helixterm.io",
        ExpiresAt: time.Now().Add(15 * time.Minute),
        Extra: map[string]interface{}{
            "org_id": user.OrgID,
            "roles":  user.Roles,
        },
    }

    newAccessToken, err := h.issuer.Sign(ctx, claims)
    if err != nil {
        h.logger.Error("failed to sign new access token",
            zap.Error(err))
        c.JSON(http.StatusInternalServerError,
            gin.H{"error": "token issuance failed"})
        return
    }

    newRefreshToken, err := auth.GenerateOpaqueToken(32)
    if err != nil {
        h.logger.Error("failed to generate new refresh
            token", zap.Error(err))
        c.JSON(http.StatusInternalServerError,
            gin.H{"error": "token issuance failed"})
        return
    }

    if err := h.tokenRepo.StoreRefreshToken(ctx,
        repository.RefreshToken{

```

```

        Token:      newRefreshToken,
        UserID:     user.ID,
        ExpiresAt:  time.Now().Add(30 * 24 * time.Hour),
    }); err != nil {
        c.JSON(http.StatusInternalServerError,
            gin.H{"error": "token storage failed"})
        return
    }

    c.JSON(http.StatusOK, model.TokenResponse{
        AccessToken:  newAccessToken,
        RefreshToken: newRefreshToken,
        TokenType:     "Bearer",
        ExpiresIn:     int(15 * time.Minute / time.Second),
    })
}

```

3.3 Go: API Gateway Token Validation Middleware

```

// File: helixterm.io/services/api-gateway/internal/
//       middleware/auth_middleware.go
package middleware

import (
    "net/http"
    "strings"

    "github.com/gin-gonic/gin"
    "go.uber.org/zap"

    "digital.vasic/auth/jwt"
)

const (
    CtxKeyUserID      = "helix_user_id"
    CtxKeyOrgID       = "helix_org_id"
    CtxKeyRoles       = "helix_roles"
    CtxKeyTokenClaims = "helix_token_claims"
)

// JWTAuth returns a Gin middleware that validates Bearer
// tokens
// using digital.vasic.auth's JWT validator.
func JWTAuth(validator jwt.Validator, logger *zap.Logger)
    gin.HandlerFunc {
    return func(c *gin.Context) {
        authHeader := c.GetHeader("Authorization")
        if authHeader == "" {

```

```

        c.AbortWithStatusJSON(http.StatusUnauthorized,
gin.H{
    "error": "missing Authorization header",
    "code": "MISSING_TOKEN",
})
return
}

parts := strings.SplitN(authHeader, " ", 2)
if len(parts) != 2 || !strings.EqualFold(parts[0],
"Bearer") {
    c.AbortWithStatusJSON(http.StatusUnauthorized,
gin.H{
        "error": "malformed Authorization header;
expected 'Bearer <token>'",
        "code": "MALFORMED_TOKEN",
    })
    return
}

rawToken := parts[1]
claims, err :=
validator.Validate(c.Request.Context(), rawToken)
if err != nil {
    logger.Warn("token validation failed",
        zap.String("ip", c.ClientIP()),
        zap.Error(err),
    )
    c.AbortWithStatusJSON(http.StatusUnauthorized,
gin.H{
        "error": "invalid or expired token",
        "code": "INVALID_TOKEN",
    })
    return
}

// Propagate claims into request context for
// downstream handlers.
c.Set(CtxKeyUserID, claims.Subject)
c.Set(CtxKeyOrgID, claims.Extra["org_id"])
c.Set(CtxKeyRoles, claims.Extra["roles"])
c.Set(CtxKeyTokenClaims, claims)

c.Next()
}
}

// RequireRole returns a Gin middleware that enforces role-
// based access control.
func RequireRole(roles ...string) gin.HandlerFunc {
    return func(c *gin.Context) {
        userRoles, exists := c.Get(CtxKeyRoles)

```

```

        if !exists {
            c.AbortWithStatusJSON(http.StatusForbidden,
                gin.H{"error": "no roles present"})
            return
        }

        rolesSlice, ok := userRoles.([]string)
        if !ok {
            c.AbortWithStatusJSON(http.StatusForbidden,
                gin.H{"error": "invalid roles format"})
            return
        }

        for _, required := range roles {
            for _, has := range rolesSlice {
                if has == required {
                    c.Next()
                    return
                }
            }
        }

        c.AbortWithStatusJSON(http.StatusForbidden, gin.H{
            "error": "insufficient permissions",
            "code": "FORBIDDEN",
        })
    }
}

```

3.4 Go: Token Propagation via gRPC Metadata

All 25 services forward the caller's JWT in gRPC metadata under the `authorization` key. The interceptor is shared via helixterm.io/pkg/shared/grpcauth.

```

// File: helixterm.io/pkg/shared/grpcauth/interceptors.go
package grpcauth

import (
    "context"
    "fmt"
    "strings"

    "google.golang.org/grpc"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/metadata"
    "google.golang.org/grpc/status"

    "digital.vasic/auth/jwt"

```

```

)

const metadataKeyAuthorization = "authorization"

// UnaryClientInterceptor extracts the JWT from the Gin
// context and
// injects it into outgoing gRPC metadata so that downstream
// services
// receive the caller's identity.
func UnaryClientInterceptor(ctx context.Context)
    grpc.UnaryClientInterceptor {
    return func(
        ctx context.Context,
        method string,
        req, reply interface{},
        cc *grpc.ClientConn,
        invoker grpc.UnaryInvoker,
        opts ...grpc.CallOption,
    ) error {
        token, ok := TokenFromContext(ctx)
        if ok {
            ctx = metadata.AppendToOutgoingContext(ctx,
                metadataKeyAuthorization, "Bearer "+token)
        }
        return invoker(ctx, method, req, reply, cc, opts...)
    }
}

// UnaryServerInterceptor validates the JWT present in
// incoming gRPC
// metadata. Services call this so they can enforce auth
// independently
// of the API Gateway.
func UnaryServerInterceptor(validator jwt.Validator)
    grpc.UnaryServerInterceptor {
    return func(
        ctx context.Context,
        req interface{},
        info *grpc.UnaryServerInfo,
        handler grpc.UnaryHandler,
    ) (interface{}, error) {
        md, ok := metadata.FromIncomingContext(ctx)
        if !ok {
            return nil, status.Error(codes.Unauthenticated,
                "missing metadata")
        }

        authValues := md.Get(metadataKeyAuthorization)
        if len(authValues) == 0 {
            return nil, status.Error(codes.Unauthenticated,
                "missing authorization metadata")
        }
    }
}

```

```

        rawToken := strings.TrimPrefix(authValues[0],
            "Bearer ")
        claims, err := validator.Validate(ctx, rawToken)
        if err != nil {
            return nil, status.Errorf(codes.Unauthenticated,
                "invalid token: %v", err)
        }

        ctx = ContextWithClaims(ctx, claims)
        return handler(ctx, req)
    }
}

type contextKey string

const claimsKey contextKey = "jwt_claims"
const tokenKey contextKey = "raw_jwt"

// ContextWithClaims stores JWT claims in the context.
func ContextWithClaims(ctx context.Context, claims
    *jwt.Claims) context.Context {
    return context.WithValue(ctx, claimsKey, claims)
}

// ClaimsFromContext retrieves JWT claims from the context.
func ClaimsFromContext(ctx context.Context) (*jwt.Claims,
    bool) {
    c, ok := ctx.Value(claimsKey).(*jwt.Claims)
    return c, ok
}

// TokenFromContext retrieves the raw JWT string from the
    context.
func TokenFromContext(ctx context.Context) (string, bool) {
    t, ok := ctx.Value(tokenKey).(string)
    return t, ok
}

```

3.5 Go: OIDC Provider Integrations

```

// File: helixterm.io/services/auth/internal/oidc/
    providers.go
package oidc

import (
    "context"
    "fmt"

```

```

        "digital.vasic/auth/oidc"
    )

    // ProviderRegistry holds all configured OIDC providers.
    type ProviderRegistry struct {
        providers map[string]oidc.Provider
    }

    // ProviderConfig configures a single OIDC provider.
    type ProviderConfig struct {
        Name          string
        Issuer         string
        ClientID       string
        ClientSecret   string
        RedirectURL    string
        Scopes         []string
        // Extra attributes for enterprise providers
        TenantID      string // Azure AD tenant ID
        Domain        string // Auth0 domain / Okta domain
    }

    // NewProviderRegistry creates all OIDC providers from
    // configuration.
    func NewProviderRegistry(ctx context.Context, configs
        []ProviderConfig) (*ProviderRegistry, error) {
        r := &ProviderRegistry{providers:
            make(map[string]oidc.Provider)}

        for _, cfg := range configs {
            var p oidc.Provider
            var err error

            switch cfg.Name {
            case "okta":
                p, err = oidc.NewOktaProvider(ctx,
                    oidc.OktaConfig{
                        Domain:        cfg.Domain,
                        ClientID:       cfg.ClientID,
                        ClientSecret:   cfg.ClientSecret,
                        RedirectURL:    cfg.RedirectURL,
                        Scopes:         cfg.Scopes,
                    })
            case "azure-ad":
                p, err = oidc.NewAzureADProvider(ctx,
                    oidc.AzureADConfig{
                        TenantID:       cfg.TenantID,
                        ClientID:       cfg.ClientID,
                        ClientSecret:   cfg.ClientSecret,
                        RedirectURL:    cfg.RedirectURL,
                    })
            case "google":

```

```

        p, err = oidc.NewGoogleProvider(ctx,
oidc.GoogleConfig{
            ClientID:      cfg.ClientID,
            ClientSecret:  cfg.ClientSecret,
            RedirectURL:   cfg.RedirectURL,
            HostedDomain:  cfg.Domain,
        })
    case "auth0":
        p, err = oidc.NewAuth0Provider(ctx,
oidc.Auth0Config{
            Domain:      cfg.Domain,
            ClientID:     cfg.ClientID,
            ClientSecret: cfg.ClientSecret,
            RedirectURL:  cfg.RedirectURL,
        })
    case "keycloak":
        p, err = oidc.NewKeycloakProvider(ctx,
oidc.KeycloakConfig{
            Issuer:      cfg.Issuer,
            ClientID:     cfg.ClientID,
            ClientSecret: cfg.ClientSecret,
            RedirectURL:  cfg.RedirectURL,
        })
    default:
        // Generic OIDC provider for any standards-
        // compliant IdP.
        p, err = oidc.NewGenericProvider(ctx,
oidc.GenericConfig{
            IssuerURL:    cfg.Issuer,
            ClientID:     cfg.ClientID,
            ClientSecret: cfg.ClientSecret,
            RedirectURL:  cfg.RedirectURL,
            Scopes:      cfg.Scopes,
        })
    }

    if err != nil {
        return nil, fmt.Errorf("oidc: initialising
        provider %q: %w", cfg.Name, err)
    }

    r.providers[cfg.Name] = p
}

return r, nil
}

// Get returns a provider by name, or an error if not found.
func (r *ProviderRegistry) Get(name string) (oidc.Provider,
    error) {
    p, ok := r.providers[name]
    if !ok {

```

```

        return nil, fmt.Errorf("oidc: provider %q not
        registered", name)
    }
    return p, nil
}

```

3.6 Go: SCIM 2.0 Provisioning Handler

```

// File: helixterm.io/services/auth/internal/handler/
//       scim_handler.go
package handler

import (
    "net/http"

    "github.com/gin-gonic/gin"
    "go.uber.org/zap"

    "digital.vasic/auth/scim"
    "helixterm.io/services/auth/internal/repository"
)

// SCIMHandler implements SCIM 2.0 endpoints for enterprise
// user provisioning.
// Supports Okta, Azure AD, and Ping Identity as
// provisioners.
type SCIMHandler struct {
    processor scim.Processor
    userRepo  repository.UserRepository
    teamRepo  repository.TeamRepository
    logger    *zap.Logger
}

// NewSCIMHandler creates a new SCIM handler.
func NewSCIMHandler(userRepo repository.UserRepository,
    teamRepo repository.TeamRepository, logger
    *zap.Logger) *SCIMHandler {
    return &SCIMHandler{
        processor: scim.NewProcessor(scim.ProcessorConfig{
            ServiceProviderName: "HelixTerminator",
            BaseURL:             "https://api.helixterm.io/
            v1/scim/v2",
        }),
        userRepo: userRepo,
        teamRepo: teamRepo,
        logger:   logger,
    }
}

```

```

// GetUsers handles SCIM GET /Users with filtering and
// pagination.
func (h *SCIMHandler) GetUsers(c *gin.Context) {
    query := scim.ListQuery{
        Filter:      c.Query("filter"),
        StartIndex:  intQueryOrDefault(c, "startIndex", 1),
        Count:       intQueryOrDefault(c, "count", 100),
    }

    users, total, err :=
        h.userRepo.ListSCIM(c.Request.Context(), query)
    if err != nil {
        h.logger.Error("SCIM: failed to list users",
            zap.Error(err))
        c.JSON(http.StatusInternalServerError,
            scim.ErrorResponse("serverError", err.Error()))
        return
    }

    scimUsers := make([]scim.User, len(users))
    for i, u := range users {
        scimUsers[i] = mapUserToSCIM(u)
    }

    c.JSON(http.StatusOK, scim.ListResponse{
        Schemas:
            []string{"urn:ietf:params:scim:api:messages:2.0:ListResponse"},
        TotalResults: total,
        StartIndex:   query.StartIndex,
        ItemsPerPage: len(scimUsers),
        Resources:    scimUsers,
    })
}

// CreateUser handles SCIM POST /Users – creates a new
// provisioned user.
func (h *SCIMHandler) CreateUser(c *gin.Context) {
    var scimUser scim.User
    if err := c.ShouldBindJSON(&scimUser); err != nil {
        c.JSON(http.StatusBadRequest,
            scim.ErrorResponse("invalidValue", err.Error()))
        return
    }

    ctx := c.Request.Context()
    validated, err := h.processor.ValidateUser(scimUser)
    if err != nil {
        c.JSON(http.StatusBadRequest,
            scim.ErrorResponse("invalidValue", err.Error()))
        return
    }
}

```

```

        created, err := h.userRepo.CreateFromSCIM(ctx, validated)
        if err != nil {
            if isConflict(err) {
                c.JSON(http.StatusConflict,
                    scim.ErrorResponse("uniqueness", "user already
exists"))
                return
            }
            h.logger.Error("SCIM: failed to create user",
                zap.Error(err))
            c.JSON(http.StatusInternalServerError,
                scim.ErrorResponse("serverError", err.Error()))
            return
        }

        c.JSON(http.StatusCreated, mapUserToSCIM(created))
    }

    // PatchUser handles SCIM PATCH /Users/:id – partial update.
    func (h *SCIMHandler) PatchUser(c *gin.Context) {
        userID := c.Param("id")
        var patchOp scim.PatchOperation
        if err := c.ShouldBindJSON(&patchOp); err != nil {
            c.JSON(http.StatusBadRequest,
                scim.ErrorResponse("invalidValue", err.Error()))
            return
        }

        ctx := c.Request.Context()
        updated, err := h.userRepo.PatchFromSCIM(ctx, userID,
            patchOp)
        if err != nil {
            h.logger.Error("SCIM: failed to patch user",
                zap.String("user_id", userID), zap.Error(err))
            c.JSON(http.StatusInternalServerError,
                scim.ErrorResponse("serverError", err.Error()))
            return
        }

        c.JSON(http.StatusOK, mapUserToSCIM(updated))
    }

    func mapUserToSCIM(u repository.User) scim.User {
        return scim.User{
            Schemas:
                []string{"urn:ietf:params:scim:schemas:core:2.0:User"},
            ID:      u.ID,
            UserName: u.Email,
            Name: scim.Name{
                GivenName: u.FirstName,
                FamilyName: u.LastName,
            },
        },
    }

```

```

        Emails: []scim.Email{{Value: u.Email, Primary:
            true}},
        Active: u.IsActive,
        Meta: scim.Meta{
            ResourceType: "User",
            Created:      u.CreatedAt,
            LastModified: u.UpdatedAt,
        },
    },
}

```

4. digital.vasic.cache

Import path: digital.vasic/cache

Go module: digital.vasic/cache v1.x.x

4.1 Purpose

digital.vasic.cache provides a two-layer Redis caching abstraction (L1 in-process + L2 Redis) with encryption-aware TTL handling, workspace-isolated namespacing, and cache-aside patterns. In HelixTerminator it serves:

- Terminal session state (active WebSocket session metadata)
- SSH connection metadata
- Vault item caching (with short TTLs because of at-rest encryption)
- Host list caching per workspace
- Rate limiter state counters
- User preference and config caching

4.2 Go: Cache Manager Initialization

```

// File: helixterm.io/pkg/shared/cachemanager/manager.go
package cachemanager

import (
    "context"
    "fmt"
    "time"

    "digital.vasic/cache"
    "digital.vasic/cache/redis"
    "go.uber.org/zap"
)

// Config holds Redis connection settings.
type Config struct {
    Addrs      []string // Redis Cluster node addresses

```

```

    Password    string
    DB          int
    PoolSize    int
    DialTimeout time.Duration
    ReadTimeout time.Duration
    WriteTimeout time.Duration
    // L1 in-process cache settings
    L1MaxItems   int
    L1DefaultTTL time.Duration
    // Namespace prefix for all keys (e.g.
    // "helixterm:production")
    Namespace string
}

// Manager wraps digital.vasic.cache for HelixTerminator
// services.
type Manager struct {
    cache cache.Cache
    logger *zap.Logger
    ns     string
}

// New initialises the cache manager with Redis Cluster
// support.
func New(ctx context.Context, cfg Config, logger
    *zap.Logger) (*Manager, error) {
    redisClient, err :=
        redis.NewClusterClient(redis.ClusterConfig{
            Addrs:      cfg.Addrs,
            Password:   cfg.Password,
            DialTimeout: cfg.DialTimeout,
            ReadTimeout: cfg.ReadTimeout,
            WriteTimeout: cfg.WriteTimeout,
            PoolSize:   cfg.PoolSize,
        })
    if err != nil {
        return nil, fmt.Errorf("cache: connecting to Redis
            cluster: %w", err)
    }

    // Ping to verify connectivity.
    if err := redisClient.Ping(ctx); err != nil {
        return nil, fmt.Errorf("cache: Redis ping failed:
            %w", err)
    }

    c, err := cache.New(cache.Config{
        Backend: redisClient,
        L1: &cache.L1Config{
            MaxItems:   cfg.L1MaxItems,
            DefaultTTL: cfg.L1DefaultTTL,
        },
    },

```

```

        DefaultTTL: 5 * time.Minute,
    })
    if err != nil {
        return nil, fmt.Errorf("cache: initialising cache
            layer: %w", err)
    }

    logger.Info("cache manager initialised",
        zap.Strings("redis_addrs", cfg.Addrs),
        zap.String("namespace", cfg.Namespace),
    )

    return &Manager{cache: c, logger: logger, ns:
        cfg.Namespace}, nil
}

// key constructs a namespaced cache key.
func (m *Manager) key(parts ...string) string {
    k := m.ns
    for _, p := range parts {
        k += ":" + p
    }
    return k
}

// Get retrieves a value, checking L1 first then L2 (Redis).
func (m *Manager) Get(ctx context.Context, key string, dest
    interface{}) (bool, error) {
    found, err := m.cache.Get(ctx, m.key(key), dest)
    if err != nil {
        m.logger.Error("cache get failed", zap.String("key",
            key), zap.Error(err))
        return false, fmt.Errorf("cache: get %q: %w", key,
            err)
    }
    return found, nil
}

// Set stores a value in both L1 and L2.
func (m *Manager) Set(ctx context.Context, key string, value
    interface{}, ttl time.Duration) error {
    if err := m.cache.Set(ctx, m.key(key), value, ttl); err !=
        nil {
        m.logger.Error("cache set failed", zap.String("key",
            key), zap.Error(err))
        return fmt.Errorf("cache: set %q: %w", key, err)
    }
    return nil
}

// Delete invalidates a key from both L1 and L2.

```

```

func (m *Manager) Delete(ctx context.Context,
    keys ...string) error {
    namespaced := make([]string, len(keys))
    for i, k := range keys {
        namespaced[i] = m.key(k)
    }
    if err := m.cache.Delete(ctx, namespaced...); err != nil {
        return fmt.Errorf("cache: delete: %w", err)
    }
    return nil
}

// DeletePattern removes all keys matching a glob pattern
// within the namespace.
func (m *Manager) DeletePattern(ctx context.Context, pattern
    string) error {
    if err := m.cache.DeletePattern(ctx, m.key(pattern));
    err != nil {
        return fmt.Errorf("cache: delete pattern %q: %w",
            pattern, err)
    }
    return nil
}

```

4.3 Go: Cache-Aside Pattern for Host Lists

```

// File: helixterm.io/services/host-manager/internal/handler/
//       host_handler.go
package handler

import (
    "context"
    "encoding/json"
    "fmt"
    "net/http"
    "time"

    "github.com/gin-gonic/gin"
    "go.uber.org/zap"

    "helixterm.io/pkg/shared/cachemanager"
    "helixterm.io/services/host-manager/internal/model"
    "helixterm.io/services/host-manager/internal/repository"
)

const (
    hostListCacheTTL = 2 * time.Minute
    hostCacheTTL     = 5 * time.Minute

```

```

)

// HostHandler handles host CRUD with cache-aside.
type HostHandler struct {
    cache *cachemanager.Manager
    repo  repository.HostRepository
    logger *zap.Logger
}

// ListHosts returns the host list for a workspace, using
// cache-aside.
// GET /v1/hosts?workspace_id=<id>
func (h *HostHandler) ListHosts(c *gin.Context) {
    workspaceID := c.Query("workspace_id")
    if workspaceID == "" {
        c.JSON(http.StatusBadRequest, gin.H{"error":
            "workspace_id is required"})
        return
    }

    ctx := c.Request.Context()
    cacheKey := fmt.Sprintf("host-manager:hosts:workspace:
        %s", workspaceID)

    // L1/L2 cache lookup.
    var hosts []model.Host
    found, err := h.cache.Get(ctx, cacheKey, &hosts)
    if err != nil {
        h.logger.Warn("cache lookup failed, falling through
            to DB", zap.Error(err))
    }

    if found {
        c.JSON(http.StatusOK, hosts)
        return
    }

    // Cache miss – load from PostgreSQL.
    hosts, err = h.repo.ListByWorkspace(ctx, workspaceID)
    if err != nil {
        h.logger.Error("failed to list hosts",
            zap.String("workspace_id", workspaceID),
            zap.Error(err))
        c.JSON(http.StatusInternalServerError,
            gin.H{"error": "failed to retrieve hosts"})
        return
    }

    // Populate cache (fire-and-forget; errors are logged
    // only).
    if setErr := h.cache.Set(ctx, cacheKey, hosts,
        hostListCacheTTL); setErr != nil {

```

```

        h.logger.Warn("failed to cache host list",
            zap.Error(setErr))
    }

    c.JSON(http.StatusOK, hosts)
}

// InvalidateHostCache is called after any write operation to
// hosts.
func (h *HostHandler) InvalidateHostCache(ctx
    context.Context, workspaceID string) {
    key := fmt.Sprintf("host-manager:hosts:workspace:%s",
        workspaceID)
    if err := h.cache.Delete(ctx, key); err != nil {
        h.logger.Warn("failed to invalidate host cache",
            zap.String("key", key), zap.Error(err))
    }
}

```

4.4 Go: Vault Item Caching with Encryption-Aware TTLs

Vault items are cached in their **decrypted** form for a short TTL (30 seconds) scoped to the requesting session. This avoids repeated decryption round-trips while minimising the window of plaintext exposure in Redis.

```

// File: helixterm.io/services/vault/internal/handler/
// vault_cache.go
package handler

import (
    "context"
    "fmt"
    "time"

    "helixterm.io/pkg/shared/cachemanager"
    "helixterm.io/services/vault/internal/model"
)

// Vault item TTLs – deliberately short to limit plaintext
// exposure.
const (
    vaultItemDecryptedTTL = 30 * time.Second
    vaultItemListTTL      = 10 * time.Second
)

type vaultCache struct {
    cm *cachemanager.Manager
}

```

```

func (vc *vaultCache) getDecryptedItem(ctx context.Context,
    sessionID, itemID string) (*model.VaultItem, bool,
    error) {
    key := fmt.Sprintf("vault:decrypted:session:%s:item:%s",
        sessionID, itemID)
    var item model.VaultItem
    found, err := vc.cm.Get(ctx, key, &item)
    return &item, found, err
}

func (vc *vaultCache) setDecryptedItem(ctx context.Context,
    sessionID string, item *model.VaultItem) error {
    key := fmt.Sprintf("vault:decrypted:session:%s:item:%s",
        sessionID, item.ID)
    return vc.cm.Set(ctx, key, item, vaultItemDecryptedTTL)
}

func (vc *vaultCache) invalidateSession(ctx context.Context,
    sessionID string) error {
    pattern := fmt.Sprintf("vault:decrypted:session:%s:*",
        sessionID)
    return vc.cm.DeletePattern(ctx, pattern)
}

```

4.5 Go: Cache Warming on Service Startup

```

// File: helixterm.io/services/host-manager/internal/startup/
//       cache_warm.go
package startup

import (
    "context"
    "fmt"
    "time"

    "go.uber.org/zap"

    "helixterm.io/pkg/shared/cachemanager"
    "helixterm.io/services/host-manager/internal/repository"
)

// WarmHostCache pre-populates the host list cache for all
// active workspaces.
// Called once during service startup before traffic is
// accepted.
func WarmHostCache(
    ctx context.Context,
    repo repository.HostRepository,
    cache *cachemanager.Manager,

```

```

    logger *zap.Logger,
) error {
    logger.Info("warming host cache")
    start := time.Now()

    workspaces, err := repo.ListActiveWorkspaceIDs(ctx)
    if err != nil {
        return fmt.Errorf("cache_warm: listing workspaces:
        %w", err)
    }

    warmed := 0
    for _, wsID := range workspaces {
        hosts, err := repo.ListByWorkspace(ctx, wsID)
        if err != nil {

            logger.Warn("cache_warm: failed to load hosts for
            workspace",
                zap.String("workspace_id", wsID),
                zap.Error(err),
            )
            continue
        }

        key := fmt.Sprintf("host-manager:hosts:workspace:
        %s", wsID)
        if err := cache.Set(ctx, key, hosts, 2*time.Minute);
        err != nil {
            logger.Warn("cache_warm: failed to set cache for
            workspace",
                zap.String("workspace_id", wsID),
                zap.Error(err),
            )
            continue
        }
        warmed++
    }

    logger.Info("host cache warming complete",
        zap.Int("workspaces_warmed", warmed),
        zap.Int("total_workspaces", len(workspaces)),
        zap.Duration("elapsed", time.Since(start)),
    )
    return nil
}

```

5. digital.vasic.database

Import path: digital.vasic/database

Go module: digital.vasic/database v1.x.x

5.1 Purpose

digital.vasic.database provides a PostgreSQL abstraction layer with support for primary/replica routing, connection pooling, migration runner integration (golang-migrate), soft delete conventions, cursor-based pagination, and structured transaction helpers. All 25 HelixTerminator services use this module for database access.

5.2 Go: Database Initialization Per Service

```
// File: helixterm.io/pkg/shared/dbmanager/manager.go
package dbmanager

import (
    "context"
    "fmt"
    "time"

    "digital.vasic/database"
    "digital.vasic/database/migrate"
    "go.uber.org/zap"
)

// Config holds database configuration for a single service.
type Config struct {
    PrimaryDSN      string
    ReplicaDSNs     []string
    MaxOpenConns    int
    MaxIdleConns    int
    ConnMaxLifetime time.Duration
    ConnMaxIdleTime time.Duration
    MigrationsPath  string // embedded FS path or directory
    ServiceName     string // used for logging and metrics
    labels
}

// Manager wraps digital.vasic.database for a service.
type Manager struct {
    db      database.DB
    logger  *zap.Logger
}

// New initialises a database Manager with read replica
// routing.
```

```

func New(ctx context.Context, cfg Config, logger
    *zap.Logger) (*Manager, error) {
    primary, err := database.Open(database.Config{
        DSN:            cfg.PrimaryDSN,
        MaxOpenConns:   cfg.MaxOpenConns,
        MaxIdleConns:   cfg.MaxIdleConns,
        ConnMaxLifetime: cfg.ConnMaxLifetime,
        ConnMaxIdleTime: cfg.ConnMaxIdleTime,
        ServiceName:    cfg.ServiceName,
    })
    if err != nil {
        return nil, fmt.Errorf("db: opening primary
            connection for %s: %w", cfg.ServiceName, err)
    }

    if err := primary.Ping(ctx); err != nil {
        return nil, fmt.Errorf("db: primary ping failed for
            %s: %w", cfg.ServiceName, err)
    }

    var replicas []database.DB
    for i, dsn := range cfg.ReplicaDSNs {
        r, err := database.Open(database.Config{
            DSN:            dsn,
            MaxOpenConns:   cfg.MaxOpenConns / 2,
            MaxIdleConns:   cfg.MaxIdleConns / 2,
            ConnMaxLifetime: cfg.ConnMaxLifetime,
            ServiceName:    fmt.Sprintf("%s-replica-%d",
                cfg.ServiceName, i),
        })
        if err != nil {
            logger.Warn("db: failed to open replica,
                skipping",
                zap.Int("replica_index", i),
                zap.Error(err),
            )
            continue
        }
        replicas = append(replicas, r)
    }

    db, err := database.NewWithReplicas(primary, replicas,
        database.RoundRobinSelector)
    if err != nil {
        return nil, fmt.Errorf("db: setting up replica
            routing for %s: %w", cfg.ServiceName, err)
    }

    logger.Info("database manager initialised",
        zap.String("service", cfg.ServiceName),
        zap.Int("replicas", len(replicas)),
    )
}

```

```

        return &Manager{db: db, logger: logger}, nil
    }

    // RunMigrations executes all pending migrations.
    func (m *Manager) RunMigrations(ctx context.Context, path
        string) error {
        runner, err := migrate.NewRunner(m.db.Primary(),
            migrate.Config{
                MigrationsPath: path,
                LockTimeout:      30 * time.Second,
            })
        if err != nil {
            return fmt.Errorf("db: creating migration runner:
                %w", err)
        }
        if err := runner.Up(ctx); err != nil {
            return fmt.Errorf("db: running migrations: %w", err)
        }
        m.logger.Info("migrations applied successfully")
        return nil
    }

    // DB returns the underlying database instance for direct
    use.
    func (m *Manager) DB() database.DB { return m.db }

```

5.3 Go: Transaction Helper Patterns

```

// File: helixterm.io/pkg/shared/dbmanager/tx.go
package dbmanager

import (
    "context"
    "database/sql"
    "fmt"

    "digital.vasic/database"
)

// TxFn is a function that executes within a transaction.
type TxFn func(ctx context.Context, tx database.Tx) error

// WithTx executes fn within a serializable database
transaction.
// On success it commits; on any error it rolls back.
// This is the canonical transaction wrapper for all
HelixTerminator services.

```

```

func (m *Manager) WithTx(ctx context.Context, fn TxFn) error
{
    return m.WithTxOpts(ctx, &sql.TxOptions{Isolation:
        sql.LevelSerializable}, fn)
}

// WithTxOpts executes fn with custom transaction options.
func (m *Manager) WithTxOpts(ctx context.Context, opts
    *sql.TxOptions, fn TxFn) error {
    tx, err := m.db.Primary().BeginTx(ctx, opts)
    if err != nil {
        return fmt.Errorf("db: beginning transaction: %w",
            err)
    }

    defer func() {
        if p := recover(); p != nil {
            _ = tx.Rollback()
            panic(p)
        }
    }()

    if err := fn(ctx, tx); err != nil {
        if rbErr := tx.Rollback(); rbErr != nil {
            m.logger.Error("db: rollback failed",
                zap.Error(rbErr))
        }
        return err
    }

    if err := tx.Commit(); err != nil {
        return fmt.Errorf("db: committing transaction: %w",
            err)
    }
    return nil
}

```

5.4 Go: Soft Delete Patterns

All HelixTerminator tables that support soft delete use a `deleted_at` `TIMESTAMPZ` column. The database module provides query helpers that automatically add `WHERE deleted_at IS NULL`.

```

// File: helixterm.io/pkg/shared/dbmanager/softdelete.go
package dbmanager

import (
    "context"
    "fmt"

```

```

        "time"

        "digital.vasic/database"
    )

    // SoftDelete marks a record as deleted without physically
    // removing it.
    // table must be a trusted constant (never user-supplied to
    // avoid SQLi).
    func SoftDelete(ctx context.Context, db database.DB, table,
        column, id string) error {
        query := fmt.Sprintf(

            "UPDATE %s SET deleted_at = $1, updated_at = $1 WHERE
            %s = $2 AND deleted_at IS NULL",
            table, column,
        )
        result, err := db.ExecContext(ctx, query,
            time.Now().UTC(), id)
        if err != nil {
            return fmt.Errorf("db: soft delete on %s id=%s: %w",
                table, id, err)
        }
        rows, _ := result.RowsAffected()
        if rows == 0 {
            return fmt.Errorf("db: soft delete: record not found
            or already deleted (table=%s, id=%s)", table, id)
        }
        return nil
    }

    // IsNotDeleted appends the soft-delete filter clause to a
    // WHERE fragment.
    func IsNotDeleted(alias string) string {
        if alias != "" {
            return alias + ".deleted_at IS NULL"
        }
        return "deleted_at IS NULL"
    }
}

```

5.5 Go: Pagination Helper

HelixTerminator uses cursor-based pagination throughout. The database module provides a generic cursor helper.

```

// File: helixterm.io/pkg/shared/dbmanager/pagination.go
package dbmanager

import (

```

```

        "encoding/base64"
        "encoding/json"
        "fmt"
        "time"
    )

    // PageRequest holds pagination parameters.
    type PageRequest struct {
        Cursor    string // opaque base64-encoded cursor
        PageSize  int    // maximum items to return (default 50,
            max 200)
    }

    // PageResponse wraps a paginated result set.
    type PageResponse[T any] struct {
        Items      []T
        NextCursor string // empty string means no more pages
        Total      int64 // total count across all pages
            (optional, may be -1)
    }

    // cursorPayload is the internal structure encoded in the
        cursor.
    type cursorPayload struct {
        Timestamp time.Time `json:"ts"`
        ID        string   `json:"id"`
    }

    // EncodeCursor creates an opaque cursor from a timestamp and
        ID.
    func EncodeCursor(ts time.Time, id string) string {
        payload, _ := json.Marshal(cursorPayload{Timestamp: ts,
            ID: id})
        return base64.URLEncoding.EncodeToString(payload)
    }

    // DecodeCursor parses an opaque cursor into timestamp and
        ID.
    func DecodeCursor(cursor string) (time.Time, string, error) {
        if cursor == "" {
            return time.Time{}, "", nil
        }
        raw, err := base64.URLEncoding.DecodeString(cursor)
        if err != nil {
            return time.Time{}, "", fmt.Errorf("pagination:
                decoding cursor: %w", err)
        }
        var payload cursorPayload
        if err := json.Unmarshal(raw, &payload); err != nil {
            return time.Time{}, "", fmt.Errorf("pagination:
                unmarshalling cursor: %w", err)
        }
    }

```

```

        return payload.Timestamp, payload.ID, nil
    }

    // NormalizePageSize clamps a requested page size within [1,
    // 200].
    func NormalizePageSize(requested int) int {
        if requested <= 0 {
            return 50
        }
        if requested > 200 {
            return 200
        }
        return requested
    }
}

```

6. digital.vasic.messaging

Import path: digital.vasic/messaging

Go module: digital.vasic/messaging v1.x.x

6.1 Purpose

digital.vasic.messaging abstracts Kafka (audit events, session lifecycle, analytics) and RabbitMQ (SSH command dispatch, SFTP operations) behind a unified interface. It provides message schema validation, retry policies, dead letter queue (DLQ) routing, and observability hooks.

6.2 Kafka Topic Definitions

```

// File: helixterm.io/pkg/shared/topics/topics.go
package topics

// Kafka topic names used across HelixTerminator.
const (
    // Audit events (append-only log)
    TopicAuditEvents = "helixterm.audit.events"

    // SSH session lifecycle
    TopicSessionCreated    = "helixterm.sessions.created"
    TopicSessionClosed     = "helixterm.sessions.closed"
    TopicSessionHeartbeat  = "helixterm.sessions.heartbeat"

    // Analytics
    TopicAnalyticsPageview = "helixterm.analytics.pageview"
    TopicAnalyticsCommand  = "helixterm.analytics.command"

    // Host status changes

```

```

    TopicHostStatusChanged = "helixterm.hosts.status"

    // Vault events
    TopicVaultItemAccessed = "helixterm.vault.accessed"
    TopicVaultKeyRotated   = "helixterm.vault.key_rotated"
)

// RabbitMQ exchange and queue names.
const (
    ExchangeSSHCommands      = "helix.ssh.commands"
    QueueSSHCommandsProxy    = "helix.ssh.commands.proxy"
    QueueSSHCommandsDLQ      = "helix.ssh.commands.dlq"

    ExchangeSFTPCommands     = "helix.sftp.commands"
    QueueSFTPCommandsProxy   = "helix.sftp.commands.proxy"
    QueueSFTPCommandsDLQ     = "helix.sftp.commands.dlq"
)

```

6.3 Go: Kafka Producer Initialization and Publishing

```

// File: helixterm.io/pkg/shared/kafkaproducer/producer.go
package kafkaproducer

import (
    "context"
    "encoding/json"
    "fmt"
    "time"

    "digital.vasic/messaging/kafka"
    "go.uber.org/zap"
)

// Producer is a typed Kafka producer for HelixTerminator
// events.
type Producer struct {
    client kafka.Producer
    logger *zap.Logger
}

// Config holds Kafka producer settings.
type Config struct {
    Brokers          []string
    RequiredAcks      kafka.Acks // kafka.AcksAll (default)
                  | kafka.AcksLeader | kafka.AcksNone
    CompressionCodec string // "snappy" | "lz4" |
                  "zstd"
    BatchSize         int
    BatchTimeout      time.Duration
}

```

```

        MaxRetries      int
        RetryBackoff    time.Duration
    }

    // New creates a new Kafka producer.
    func New(cfg Config, logger *zap.Logger) (*Producer, error) {
        client, err := kafka.NewProducer(kafka.ProducerConfig{
            Brokers:           cfg.Brokers,
            RequiredAcks:       cfg.RequiredAcks,
            CompressionCodec:   cfg.CompressionCodec,
            BatchSize:         cfg.BatchSize,
            BatchTimeout:       cfg.BatchTimeout,
            MaxRetries:         cfg.MaxRetries,
            RetryBackoff:      cfg.RetryBackoff,
        })
        if err != nil {
            return nil, fmt.Errorf("kafka: creating producer: %w", err)
        }
        logger.Info("Kafka producer initialised",
            zap.Strings("brokers", cfg.Brokers))
        return &Producer{client: client, logger: logger}, nil
    }

    // Publish serialises and publishes an event to a Kafka
    // topic.
    // key is used as the Kafka partition key (should be entity
    // ID for ordering).
    func (p *Producer) Publish(ctx context.Context, topic, key
        string, event interface{}) error {
        payload, err := json.Marshal(event)
        if err != nil {
            return
            fmt.Errorf("kafka: marshalling event for topic %q: %w", topic, err)
        }

        msg := kafka.Message{
            Topic: topic,
            Key:   []byte(key),
            Value: payload,
            Headers: []kafka.Header{
                {Key: "content-type", Value: []byte("application/
                    json")},
                {Key: "source-service", Value:
                    []byte("helixterm")},
                {Key: "published-at", Value:
                    []byte(time.Now().UTC().Format(time.RFC3339Nano))},
            },
        }

        if err := p.client.Publish(ctx, msg); err != nil {

```

```

        p.logger.Error("kafka: publish failed",
            zap.String("topic", topic),
            zap.String("key", key),
            zap.Error(err),
        )
        return fmt.Errorf("kafka: publishing to topic %q:
        %w", topic, err)
    }

    p.logger.Debug("kafka: event published",
        zap.String("topic", topic), zap.String("key", key))
    return nil
}

// Close flushes all buffered messages and closes the
// producer.
func (p *Producer) Close() error {
    return p.client.Close()
}

```

6.4 Go: Audit Event Publishing

```

// File: helixterm.io/pkg/shared/audit/publisher.go
package audit

import (
    "context"
    "fmt"
    "time"

    "helixterm.io/pkg/shared/kafkaproducer"
    "helixterm.io/pkg/shared/topics"
)

// EventType classifies audit events.
type EventType string

const (
    EventTypeVaultRead      EventType = "vault.read"
    EventTypeVaultWrite     EventType = "vault.write"
    EventTypeVaultDelete    EventType = "vault.delete"
    EventTypeSSHConnect     EventType = "ssh.connect"
    EventTypeSSHDisconnect EventType = "ssh.disconnect"
    EventTypeAuthLogin      EventType = "auth.login"
    EventTypeAuthLogout     EventType = "auth.logout"
    EventTypeKeyRotation    EventType = "vault.key_rotation"
)

// Event represents an audit log entry.

```

```

type Event struct {
    ID            string          `json:"id"`
    Type          EventType       `json:"type"`
    ActorID       string          `json:"actor_id"`
    OrgID         string          `json:"org_id"`
    ResourceType  string          `json:"resource_type"`
    ResourceID    string          `json:"resource_id"`
    IPAddress     string          `json:"ip_address"`
    UserAgent     string          `json:"user_agent"`
    Metadata      map[string]string
                  `json:"metadata,omitempty"`
    Timestamp     time.Time       `json:"timestamp"`
}

// Publisher wraps the Kafka producer for audit events.
type Publisher struct {
    producer *kafkaproducer.Producer
}

// NewPublisher creates an audit Publisher.
func NewPublisher(producer *kafkaproducer.Producer)
    *Publisher {
    return &Publisher{producer: producer}
}

// Publish emits an audit event. The partition key is the
// actor's org_id
// to ensure all events for an org land in the same partition
// (ordering guarantee).
func (p *Publisher) Publish(ctx context.Context, event
    Event) error {
    if event.Timestamp.IsZero() {
        event.Timestamp = time.Now().UTC()
    }
    if err := p.producer.Publish(ctx,
        topics.TopicAuditEvents, event.OrgID, event); err !=
        nil {
        return fmt.Errorf("audit: publishing event %q: %w",
            event.Type, err)
    }
    return nil
}

```

6.5 Go: RabbitMQ Consumer for SSH Proxy

```

// File: helixterm.io/services/ssh-proxy/internal/messaging/
//       consumer.go
package messaging

```

```

import (
    "context"
    "encoding/json"
    "fmt"

    "digital.vasic/messaging/rabbitmq"
    "go.uber.org/zap"

    "helixterm.io/pkg/shared/topics"
    "helixterm.io/services/ssh-proxy/internal/session"
)

// SSHCommandMessage is the message schema for SSH dispatch
// commands.
type SSHCommandMessage struct {
    SessionID string `json:"session_id"`
    UserID    string `json:"user_id"`
    HostID    string `json:"host_id"`
    Command   string `json:"command"`
    WorkingDir string `json:"working_dir,omitempty"`
}

// SSHCommandConsumer consumes SSH dispatch commands from
// RabbitMQ.
type SSHCommandConsumer struct {
    consumer rabbitmq.Consumer
    manager  *session.Manager
    logger   *zap.Logger
}

// Config holds RabbitMQ consumer configuration.
type Config struct {
    AMQPURL      string
    Concurrency  int
    PrefetchCount int
}

// New creates a new SSHCommandConsumer.
func New(cfg Config, manager *session.Manager, logger
    *zap.Logger) (*SSHCommandConsumer, error) {
    c, err := rabbitmq.NewConsumer(rabbitmq.ConsumerConfig{
        AMQPURL:      cfg.AMQPURL,
        Exchange:     topics.ExchangeSSHCommands,
        Queue:        topics.QueueSSHCommandsProxy,
        DLQExchange:  topics.ExchangeSSHCommands + ".dlx",
        DLQQueue:     topics.QueueSSHCommandsDLQ,
        Concurrency:  cfg.Concurrency,
        PrefetchCount: cfg.PrefetchCount,
        MaxRetries:   3,
        RetryDelay:   rabbitmq.ExponentialBackoff(500, 5000),
    })
}

```

```

    if err != nil {
        return nil, fmt.Errorf("rabbitmq: creating SSH
            command consumer: %w", err)
    }

    return &SSHCommandConsumer{consumer: c, manager:
        manager, logger: logger}, nil
}

// Start begins consuming messages. Blocks until ctx is
// cancelled.
func (c *SSHCommandConsumer) Start(ctx context.Context)
    error {
    return c.consumer.Consume(ctx, c.handleMessage)
}

// handleMessage processes a single SSH command message.
func (c *SSHCommandConsumer) handleMessage(ctx
    context.Context, msg rabbitmq.Message) error {
    var cmd SSHCommandMessage
    if err := json.Unmarshal(msg.Body, &cmd); err != nil {
        // Malformed message – send to DLQ immediately (no
        // retry).
        c.logger.Error("ssh consumer: malformed message",
            zap.Error(err))
        return rabbitmq.ErrPermanent{Err:
            fmt.Errorf("malformed SSH command message: %w", err)}
    }

    if err := validateSSHCommand(cmd); err != nil {
        c.logger.Error("ssh consumer: invalid command",
            zap.String("session_id", cmd.SessionID),
            zap.Error(err))
        return rabbitmq.ErrPermanent{Err: err}
    }

    if err := c.manager.ExecuteCommand(ctx, cmd.SessionID,
        cmd.Command, cmd.WorkingDir); err != nil {
        c.logger.Error("ssh consumer: command execution
            failed",
            zap.String("session_id", cmd.SessionID),
            zap.Error(err),
        )
        // Transient error – allow retry.
        return fmt.Errorf("ssh consumer: executing command:
            %w", err)
    }

    return nil
}

func validateSSHCommand(cmd SSHCommandMessage) error {

```

```

    if cmd.SessionID == "" {
        return fmt.Errorf("missing session_id")
    }
    if cmd.HostID == "" {
        return fmt.Errorf("missing host_id")
    }
    if cmd.Command == "" {
        return fmt.Errorf("missing command")
    }
    return nil
}

```

6.6 Go: Dead Letter Queue Handler

```

// File: helixterm.io/services/ssh-proxy/internal/messaging/
//       dlq_handler.go
package messaging

import (
    "context"
    "encoding/json"
    "time"

    "digital.vasic/messaging/rabbitmq"
    "go.uber.org/zap"

    "helixterm.io/pkg/shared/topics"
)

// DLQEntry records metadata about a failed message.
type DLQEntry struct {
    OriginalQueue string          `json:"original_queue"`
    FailedAt      time.Time          `json:"failed_at"`
    Attempts      int                `json:"attempts"`
    LastError     string             `json:"last_error"`
    Body          json.RawMessage    `json:"body"`
}

// DLQHandler processes messages that exhausted their retry
// budget.
type DLQHandler struct {
    consumer rabbitmq.Consumer
    repo     DLQRepository
    logger   *zap.Logger
}

// DLQRepository persists DLQ entries for manual inspection.
type DLQRepository interface {
    Insert(ctx context.Context, entry DLQEntry) error
}

```

```

}

// StartDLQProcessor begins processing the SSH command DLQ.
// DLQ messages are persisted to PostgreSQL for ops team
inspection.
func (h *DLQHandler) StartDLQProcessor(ctx context.Context)
error {
    return h.consumer.Consume(ctx, func(ctx context.Context,
        msg rabbitmq.Message) error {
        entry := DLQEntry{
            OriginalQueue: topics.QueueSSHCommandsProxy,
            FailedAt:       time.Now().UTC(),
            Attempts:       msg.DeliveryCount,
            Body:          msg.Body,
        }

        if header, ok := msg.Headers["x-last-error"]; ok {
            entry.LastError = string(header)
        }

        if err := h.repo.Insert(ctx, entry); err != nil {
            h.logger.Error("dlq: failed to persist DLQ
                entry", zap.Error(err))
            return err
        }

        h.logger.Warn("dlq: message moved to dead letter
            storage",
            zap.String("queue",
                topics.QueueSSHCommandsProxy),
            zap.Int("attempts", entry.Attempts),
        )
        return nil
    })
}

```

7. digital.vasic.middleware

Import path: digital.vasic/middleware

Go module: digital.vasic/middleware v1.x.x

7.1 Purpose

digital.vasic.middleware provides a suite of production-ready Gin middleware components: request ID injection, structured JSON logging, panic recovery, timeout enforcement, CORS, and compression. Every HelixTerminator service that exposes an HTTP router uses this module.

7.2 Go: Full Middleware Chain Setup

```
// File: helixterm.io/services/api-gateway/internal/server/
server.go
package server

import (
    "net/http"
    "time"

    "github.com/gin-gonic/gin"
    "go.uber.org/zap"

    "digital.vasic/middleware"
    "digital.vasic/middleware/cors"
    "digital.vasic/middleware/recovery"
    "digital.vasic/middleware/requestid"
    "digital.vasic/middleware/timeout"
    "digital.vasic/middleware/compress"
    "digital.vasic/auth/jwt"

    gwmw "helixterm.io/services/api-gateway/internal/
        middleware"
)

// NewRouter builds the full API Gateway Gin router with all
// middleware.
func NewRouter(
    logger *zap.Logger,
    jwtValidator jwt.Validator,
) *gin.Engine {
    gin.SetMode(gin.ReleaseMode)
    r := gin.New() // gin.New() – no default middleware; we
        add our own.

    // 1. Request ID – must be first so all subsequent
    // middleware can log it.
    r.Use(requestid.New(requestid.Config{
        HeaderName: "X-Request-ID",
        Generator:  requestid.UUIDv7Generator,
        SetResponse: true,
    })))

    // 2. Structured JSON logger.
    r.Use(middleware.JSONLogger(middleware.LoggerConfig{
        Logger:      logger,
        SkipPaths:    []string{"/healthz", "/readyz", "/
            metrics"},
        LogLatency:  true,
        LogRequestBody: false, // PII protection
    }))
}
```

```

    )))

    // 3. Panic recovery with structured error response.
    r.Use(recovery.New(recovery.Config{
        Logger:          logger,
        RecoveryHandler: panicHandler,
    }))

    // 4. Request timeout (30s hard ceiling at gateway).
    r.Use(timeout.New(timeout.Config{
        Timeout:          30 * time.Second,
        TimeoutHandler:   timeoutHandler,
    }))

    // 5. CORS – configures allowed origins from environment.
    r.Use(cors.New(cors.Config{
        AllowOrigins:     []string{"https://app.helixterm.io", "https://helixterm.io"},
        AllowMethods:      []string{"GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"},
        AllowHeaders:      []string{"Authorization", "Content-Type", "X-Request-ID", "X-Workspace-ID"},
        ExposeHeaders:     []string{"X-Request-ID"},
        AllowCredentials:  true,
        MaxAge:            12 * time.Hour,
    }))

    // 6. Brotli/Gzip response compression.
    r.Use(compress.New(compress.Config{
        Level:             compress.BestSpeed,
        MinResponseSize:   1024, // compress responses >= 1KB
    }))

    // 7. JWT authentication for all API routes.
    // Routes under /public/ are exempt.
    api := r.Group("/v1")
    api.Use(gmw.JWTAuth(jwtValidator, logger))

    return r
}

func panicHandler(c *gin.Context, err interface{}) {
    c.AbortWithStatusJSON(http.StatusInternalServerError, gin.H{
        "error":      "internal server error",
        "request_id": c.GetHeader("X-Request-ID"),
    })
}

func timeoutHandler(c *gin.Context) {
    c.AbortWithStatusJSON(http.StatusGatewayTimeout, gin.H{
        "error":      "request timeout",
    })
}

```

```

        "request_id": c.GetHeader("X-Request-ID"),
    })
}

```

7.3 Go: Request ID Propagation

```

// File: helixterm.io/pkg/shared/requestid/propagate.go
package requestid

import (
    "context"

    "github.com/gin-gonic/gin"
)

type contextKey string

const requestIDKey contextKey = "request_id"

// FromGinContext extracts the request ID set by the
// middleware.
func FromGinContext(c *gin.Context) string {
    if id, exists := c.Get("request_id"); exists {
        if s, ok := id.(string); ok {
            return s
        }
    }
    return c.GetHeader("X-Request-ID")
}

// WithRequestID injects the request ID into a standard
// context.Context.
// Used when passing context to non-Gin code (repository,
// service layer).
func WithRequestID(ctx context.Context, id string)
context.Context {
    return context.WithValue(ctx, requestIDKey, id)
}

// FromContext retrieves the request ID from a standard
// context.
func FromContext(ctx context.Context) string {
    if id, ok := ctx.Value(requestIDKey).(string); ok {
        return id
    }
}

```

```
    return ""  
}
```

7.4 Go: Structured JSON Logging Middleware

```
// File: helixterm.io/pkg/shared/logging/gin_logger.go  
package logging  
  
import (  
    "time"  
  
    "github.com/gin-gonic/gin"  
    "go.uber.org/zap"  
  
    "helixterm.io/pkg/shared/requestid"  
)  
  
// GinZapLogger returns a Gin middleware that emits  
// structured JSON logs per request.  
// It reads the request ID from the Gin context (set by  
// requestid middleware).  
func GinZapLogger(logger *zap.Logger, skipPaths []string)  
    gin.HandlerFunc {  
    skip := make(map[string]struct{}, len(skipPaths))  
    for _, p := range skipPaths {  
        skip[p] = struct{}{}  
    }  
  
    return func(c *gin.Context) {  
        start := time.Now()  
        path := c.Request.URL.Path  
        query := c.Request.URL.RawQuery  
  
        c.Next()  
  
        if _, ok := skip[path]; ok {  
            return  
        }  
  
        end := time.Now()  
        latency := end.Sub(start)  
  
        fields := []zap.Field{  
            zap.Int("status", c.Writer.Status()),  
            zap.String("method", c.Request.Method),  
            zap.String("path", path),  
            zap.String("query", query),  
            zap.String("ip", c.ClientIP()),  
            zap.String("user_agent", c.Request.UserAgent()),  
        }  
    }  
}
```

```

        zap.Duration("latency", latency),
        zap.Int("response_size", c.Writer.Size()),
        zap.String("request_id",
requestid.FromGinContext(c)),
    }

    if userID, exists := c.Get("helix_user_id"); exists {
        fields = append(fields, zap.Any("user_id",
userID))
    }
    if orgID, exists := c.Get("helix_org_id"); exists {
        fields = append(fields, zap.Any("org_id", orgID))
    }

    if len(c.Errors) > 0 {
        fields = append(fields, zap.String("errors",
c.Errors.ByType(gin.ErrorTypePrivate).String()))
        logger.Error("request completed with errors",
fields...)
    } else {
        logger.Info("request completed", fields...)
    }
}
}

```

7.5 Go: Panic Recovery Middleware

```

// File: helixterm.io/pkg/shared/recovery/recovery.go
package recovery

import (
    "fmt"
    "net/http"
    "runtime/debug"

    "github.com/gin-gonic/gin"
    "go.uber.org/zap"

    "helixterm.io/pkg/shared/requestid"
)

// GinRecovery returns a Gin middleware that recovers from
// panics,
// logs the stack trace, and returns a 500 response.
func GinRecovery(logger *zap.Logger) gin.HandlerFunc {
    return func(c *gin.Context) {
        defer func() {
            if err := recover(); err != nil {
                stack := debug.Stack()

```

```

        logger.Error("recovered from panic",
            zap.String("request_id",
                requestid.FromGinContext(c)),
            zap.String("path", c.Request.URL.Path),
            zap.Any("panic", err),
            zap.ByteString("stack", stack),
        )

        c.AbortWithStatusJSON(http.StatusInternalServerError,
            gin.H{
                "error":      "internal server error",
                "request_id":
                    requestid.FromGinContext(c),
            })
    }
}()
c.Next()
}
}

```

7.6 Go: Timeout Middleware

```

// File: helixterm.io/pkg/shared/timeout/timeout.go
package timeout

import (
    "context"
    "net/http"
    "time"

    "github.com/gin-gonic/gin"

    "helixterm.io/pkg/shared/requestid"
)

// GinTimeout wraps each request in a context with a
// deadline.
// If the handler does not complete before the deadline, a
// 504 is returned.
func GinTimeout(d time.Duration) gin.HandlerFunc {
    return func(c *gin.Context) {
        ctx, cancel :=
            context.WithTimeout(c.Request.Context(), d)
        defer cancel()

        c.Request = c.Request.WithContext(ctx)

        finished := make(chan struct{}, 1)
        go func() {

```

```

        c.Next()
        finished <- struct{}{}
    }()

    select {
    case <-finished:
        // Handler completed normally.
    case <-ctx.Done():
        c.AbortWithStatusJSON(http.StatusGatewayTimeout,
            gin.H{
                "error":      "request timed out",
                "request_id": requestid.FromGinContext(c),
            })
    }
    }
}

```

8. digital.vasic.observability

Import path: digital.vasic/observability

Go module: digital.vasic/observability v1.x.x

8.1 Purpose

digital.vasic.observability provides Prometheus metrics registration, OpenTelemetry distributed tracing with OTLP export, structured health check endpoints, and Kubernetes readiness/liveness probe handlers. All 25 HelixTerminator services initialise this module at startup.

8.2 Go: Observability Initialization Per Service

```

// File: helixterm.io/pkg/shared/obs/obs.go
package obs

import (
    "context"
    "fmt"
    "net/http"
    "time"

    "digital.vasic/observability"
    "digital.vasic/observability/metrics"
    "digital.vasic/observability/tracing"
    "go.uber.org/zap"
)

// Config holds observability configuration for a service.

```

```

type Config struct {
    ServiceName    string
    ServiceVersion string
    Environment    string // "production" | "staging" |
        "development"
    MetricsPort    int      // Prometheus exposition port
        (default 9090)
    OTLPEndpoint   string // OpenTelemetry collector gRPC
        endpoint
    OTLPInsecure   bool
}

// Provider holds the initialised observability components.
type Provider struct {
    MetricsRegistry metrics.Registry
    Tracer           tracing.Tracer
    logger           *zap.Logger
}

// New initialises Prometheus and OpenTelemetry for a
// service.
func New(ctx context.Context, cfg Config, logger
    *zap.Logger) (*Provider, error) {
    // Initialise Prometheus registry with service labels.
    reg, err := metrics.NewRegistry(metrics.RegistryConfig{
        Namespace: "helixterm",
        Subsystem: sanitiseServiceName(cfg.ServiceName),
        ConstLabels: map[string]string{
            "service":    cfg.ServiceName,
            "version":    cfg.ServiceVersion,
            "environment": cfg.Environment,
        },
    })
    if err != nil {
        return nil, fmt.Errorf("obs: creating metrics
            registry for %s: %w", cfg.ServiceName, err)
    }

    // Initialise OpenTelemetry tracer with OTLP gRPC
    // exporter.
    tracer, err := tracing.NewOTLPTracer(ctx,
        tracing.OTLPConfig{
            ServiceName:    cfg.ServiceName,
            ServiceVersion: cfg.ServiceVersion,
            Environment:    cfg.Environment,
            Endpoint:       cfg.OTLPEndpoint,
            Insecure:        cfg.OTLPInsecure,
            BatchTimeout:    5 * time.Second,
            MaxExportBatch: 512,
        })
    if err != nil {

```

```

        return nil,
        fmt.Errorf("obs: creating OTLP tracer for %s: %w",
            cfg.ServiceName, err)
    }

    logger.Info("observability initialised",
        zap.String("service", cfg.ServiceName),
        zap.String("otlp_endpoint", cfg.OTLPEndpoint),
    )

    return &Provider{
        MetricsRegistry: reg,
        Tracer:          tracer,
        logger:          logger,
    }, nil
}

// ServeMetrics starts an HTTP server on metricsPort that
// exposes the
// Prometheus /metrics endpoint. Non-blocking – runs in a
// goroutine.
func (p *Provider) ServeMetrics(ctx context.Context, port
    int) {
    mux := http.NewServeMux()
    mux.Handle("/metrics", p.MetricsRegistry.Handler())

    srv := &http.Server{
        Addr:    fmt.Sprintf(":%d", port),
        Handler: mux,
    }

    go func() {
        p.logger.Info("metrics server started",
            zap.Int("port", port))
        if err := srv.ListenAndServe(); err != nil && err !=
            http.ErrServerClosed {
            p.logger.Error("metrics server error",
                zap.Error(err))
        }
    }()

    go func() {
        <-ctx.Done()
        shutCtx, cancel :=
            context.WithTimeout(context.Background(),
                5*time.Second)
        defer cancel()
        _ = srv.Shutdown(shutCtx)
    }()
}

func sanitiseServiceName(name string) string {

```

```

    // Replace hyphens with underscores for Prometheus label
    // compatibility.
    result := make([]byte, len(name))
    for i := range name {
        if name[i] == '-' {
            result[i] = '_'
        } else {
            result[i] = name[i]
        }
    }
    return string(result)
}

```

8.3 Go: Custom Metric Definitions

```

// File: helixterm.io/services/ssh-proxy/internal/metrics/
// metrics.go
package metrics

import (
    "digital.vasic/observability/metrics"
)

// SSHProxyMetrics holds all Prometheus metrics for the SSH
// Proxy service.
type SSHProxyMetrics struct {
    ActiveSessions      metrics.Gauge
    TotalSessions        metrics.Counter
    SessionDuration      metrics.Histogram
    CommandsExecuted     metrics.Counter
    ConnectionErrors     metrics.Counter
    AuthFailures         metrics.Counter
    BytesTransferred     metrics.Counter
}

// NewSSHProxyMetrics registers all SSH Proxy metrics with
// the registry.
func NewSSHProxyMetrics(reg metrics.Registry)
    (*SSHProxyMetrics, error) {
    active, err := reg.NewGauge(metrics.GaugeOpts{
        Name: "ssh_active_sessions",
        Help: "Number of currently active SSH sessions.",
    })
    if err != nil {
        return nil, err
    }

    total, err := reg.NewCounter(metrics.CounterOpts{
        Name: "ssh_sessions_total",

```

```
        Help: "Total number of SSH sessions since startup.",
        Labels: []string{"status"},
    })
    if err != nil {
        return nil, err
    }

    duration, err := reg.NewHistogram(metrics.HistogramOpts{
        Name:    "ssh_session_duration_seconds",
        Help:    "Duration of SSH sessions in seconds.",
        Buckets: []float64{1, 5, 15, 30, 60, 300, 600, 1800, 3600},
    })
    if err != nil {
        return nil, err
    }

    commands, err := reg.NewCounter(metrics.CounterOpts{
        Name:    "ssh_commands_executed_total",
        Help:    "Total commands executed via SSH sessions.",
        Labels: []string{"host_id"},
    })
    if err != nil {
        return nil, err
    }

    connErrors, err := reg.NewCounter(metrics.CounterOpts{
        Name:    "ssh_connection_errors_total",
        Help:    "Total SSH connection errors.",
        Labels: []string{"error_type"},
    })
    if err != nil {
        return nil, err
    }

    authFail, err := reg.NewCounter(metrics.CounterOpts{
        Name:    "ssh_auth_failures_total",
        Help:    "Total SSH authentication failures.",
        Labels: []string{"reason"},
    })
    if err != nil {
        return nil, err
    }

    bytes, err := reg.NewCounter(metrics.CounterOpts{
        Name:    "ssh_bytes_transferred_total",
        Help:    "Total bytes transferred over SSH sessions.",
        Labels: []string{"direction"},
    })
    if err != nil {
        return nil, err
    }
```

```

    }

    return &SSHProxyMetrics{
        ActiveSessions:    active,
        TotalSessions:     total,
        SessionDuration:   duration,
        CommandsExecuted:  commands,
        ConnectionErrors:  connErrors,
        AuthFailures:      authFail,
        BytesTransferred:  bytes,
    }, nil
}

```

8.4 Go: Trace Span Creation

```

// File: helixterm.io/services/vault/internal/handler/
//       vault_handler.go (tracing excerpt)
package handler

import (
    "context"
    "fmt"

    "digital.vasic/observability/tracing"
    "go.opentelemetry.io/otel/attribute"
    "go.opentelemetry.io/otel/codes"
)

// decryptAndServeItem demonstrates trace span creation for a
// critical operation.
func (h *VaultHandler) decryptAndServeItem(ctx
    context.Context, itemID, userID string)
    (*VaultItemResponse, error) {
    ctx, span := h.tracer.Start(ctx, "vault.decrypt_item",
        tracing.WithAttributes(
            attribute.String("vault.item_id", itemID),
            attribute.String("user.id", userID),
        ),
    )
    defer span.End()

    // Fetch the encrypted blob – nested span for DB
    // operation.
    ctx, dbSpan := h.tracer.Start(ctx,
        "vault.fetch_encrypted_blob")
    blob, err := h.repo.GetEncryptedBlob(ctx, itemID)
    if err != nil {
        dbSpan.RecordError(err)
        dbSpan.SetStatus(codes.Error, "db fetch failed")
    }
}

```

```

        dbSpan.End()
        span.RecordError(err)
        return nil, fmt.Errorf("vault: fetching blob for %q:
        %w", itemID, err)
    }
    dbSpan.End()

    // Decrypt – nested span for crypto operation.
    ctx, cryptoSpan := h.tracer.Start(ctx,
        "vault.aes_decrypt")
    payload, err := h.crypto.DecryptVaultPayload(ctx, blob)
    if err != nil {
        cryptoSpan.RecordError(err)
        cryptoSpan.SetStatus(codes.Error, "decryption
        failed")
        cryptoSpan.End()
        span.RecordError(err)
        return nil, fmt.Errorf("vault: decrypting item %q:
        %w", itemID, err)
    }
    cryptoSpan.SetStatus(codes.Ok, "")
    cryptoSpan.End()

    span.SetAttributes(attribute.String("vault.item_type",
        payload.Type))
    span.SetStatus(codes.Ok, "")

    return mapPayloadToResponse(payload), nil
}

```

8.5 Go: Health Check Endpoint

```

// File: helixterm.io/pkg/shared/health/health.go
package health

import (
    "context"
    "net/http"
    "sync"
    "time"

    "github.com/gin-gonic/gin"
    "go.uber.org/zap"
)

// Check is a function that returns an error if the
// dependency is unhealthy.
type Check func(ctx context.Context) error

```

```

// Handler manages health and readiness probes.
type Handler struct {
    checks map[string]Check
    mu      sync.RWMutex
    logger  *zap.Logger
}

// NewHandler creates a health handler with no checks
// registered.
func NewHandler(logger *zap.Logger) *Handler {
    return &Handler{checks: make(map[string]Check), logger:
        logger}
}

// Register adds a named health check.
func (h *Handler) Register(name string, check Check) {
    h.mu.Lock()
    defer h.mu.Unlock()
    h.checks[name] = check
}

// Liveness handles GET /healthz – always returns 200 if the
// process is running.
func (h *Handler) Liveness(c *gin.Context) {
    c.JSON(http.StatusOK, gin.H{"status": "ok"})
}

// Readiness handles GET /readyz – runs all registered
// checks.
// Returns 200 if all checks pass, 503 if any fail.
func (h *Handler) Readiness(c *gin.Context) {
    ctx, cancel := context.WithTimeout(c.Request.Context(),
        5*time.Second)
    defer cancel()

    h.mu.RLock()
    checks := make(map[string]Check, len(h.checks))
    for k, v := range h.checks {
        checks[k] = v
    }
    h.mu.RUnlock()

    results := make(map[string]string, len(checks))
    allOK := true

    for name, check := range checks {
        if err := check(ctx); err != nil {
            results[name] = "unhealthy: " + err.Error()
            allOK = false
            h.logger.Warn("health check failed",
                zap.String("check", name), zap.Error(err))
        } else {

```

```

        results[name] = "ok"
    }
}

status := http.StatusOK
if !allOK {
    status = http.StatusServiceUnavailable
}

c.JSON(status, gin.H{"checks": results, "status":
    map[bool]string{true: "ready", false: "not_ready"}
    [allOK]})
}

```

8.6 YAML: Prometheus Scrape Config for All Services

```

# prometheus/scrape_config.yaml
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    cluster: helixterm-production
    region: us-east-1

scrape_configs:
  - job_name: helixterm-api-gateway
    kubernetes_sd_configs:
      - role: pod
        namespaces:
          names: [helixterm-production]
    relabel_configs:
      - source_labels: [__meta_kubernetes_pod_label_app]
        regex: api-gateway
        action: keep
      - source_labels: [__meta_kubernetes_pod_ip]
        replacement: "$1:9090"
        target_label: __address__
    metrics_path: /metrics

  - job_name: helixterm-auth
    kubernetes_sd_configs:
      - role: pod
        namespaces:
          names: [helixterm-production]
    relabel_configs:
      - source_labels: [__meta_kubernetes_pod_label_app]
        regex: auth
        action: keep
      - source_labels: [__meta_kubernetes_pod_ip]

```

```

        replacement: "$1:9090"
        target_label: __address__

- job_name: helixterm-vault
  kubernetes_sd_configs:
    - role: pod
      namespaces: [helixterm-production]
  relabel_configs:
    - source_labels: [__meta_kubernetes_pod_label_app]
      regex: vault
      action: keep
    - source_labels: [__meta_kubernetes_pod_ip]
      replacement: "$1:9090"
      target_label: __address__

- job_name: helixterm-ssh-proxy
  kubernetes_sd_configs:
    - role: pod
      namespaces: [helixterm-production]
  relabel_configs:
    - source_labels: [__meta_kubernetes_pod_label_app]
      regex: ssh-proxy
      action: keep
    - source_labels: [__meta_kubernetes_pod_ip]
      replacement: "$1:9090"
      target_label: __address__

# Template for remaining 21 services – each follows the
# same pattern.
# Services: terminal, sftp, host-manager, user, workspace,
# notification,
# audit, analytics, ai, container-bridge, helixtrack-
# bridge, billing,
# scheduler, file-manager, config, identity, team, secret,
# webhook, search, onboarding
- job_name: helixterm-services
  kubernetes_sd_configs:
    - role: pod
      namespaces: [helixterm-production]
  relabel_configs:
    - source_labels:
        [__meta_kubernetes_pod_label_component]
      regex: helixterm-service
      action: keep
    - source_labels: [__meta_kubernetes_pod_ip]
      replacement: "$1:9090"
      target_label: __address__
    - source_labels: [__meta_kubernetes_pod_label_app]
      target_label: service_name

```

9. digital.vasic.ratelimiter

Import path: digital.vasic/ratelimiter

Go module: digital.vasic/ratelimiter v1.x.x

9.1 Purpose

digital.vasic.ratelimiter implements distributed token-bucket and fixed-window rate limiting backed by Redis. In HelixTerminator it protects the API Gateway (per-user, per-IP, per-endpoint limits), the SSH Proxy (per-user concurrent connection limits), and the Auth Service (brute-force login protection).

9.2 Go: Rate Limiter Initialization

```
// File: helixterm.io/pkg/shared/ratelimit/ratelimit.go
package ratelimit

import (
    "context"
    "fmt"
    "time"

    "digital.vasic/ratelimiter"
    "digital.vasic/ratelimiter/redis"
    "go.uber.org/zap"
)

// Config holds rate limiter configuration.
type Config struct {
    RedisAddrs []string
    RedisPass  string
    // Default limits applied when no specific rule matches.
    DefaultPerUserRPM int // requests per minute
    DefaultPerIPRPM   int
    BurstMultiplier   float64 // burst = limit * multiplier
}

// Limiter wraps digital.vasic.ratelimiter.
type Limiter struct {
    rl      ratelimiter.Limiter
    logger  *zap.Logger
}

// New creates a rate limiter backed by Redis Cluster.
func New(ctx context.Context, cfg Config, logger
    *zap.Logger) (*Limiter, error) {
    backend, err := redis.NewBackend(redis.Config{
        Addrs:      cfg.RedisAddrs,
        Password:   cfg.RedisPass,
```

```

    })
    if err != nil {
        return nil, fmt.Errorf("ratelimiter: creating Redis
            backend: %w", err)
    }

    rl, err := ratelimiter.New(ratelimiter.Config{
        Backend: backend,
        KeyPrefix: "helixterm:rl",
        DefaultPolicy: ratelimiter.Policy{
            Algorithm: ratelimiter.TokenBucket,
            Rate:      float64(cfg.DefaultPerUserRPM) /
                60.0,
            Burst:      int(float64(cfg.DefaultPerUserRPM) *
                cfg.BurstMultiplier),
            WindowSize: time.Minute,
        },
    })
    if err != nil {
        return nil, fmt.Errorf("ratelimiter: initialising
            limiter: %w", err)
    }

    return &Limiter{rl: rl, logger: logger}, nil
}

```

9.3 Go: Middleware Integration with Gin

```

// File: helixterm.io/services/api-gateway/internal/
//       middleware/ratelimit_middleware.go
package middleware

import (
    "fmt"
    "net/http"
    "strconv"
    "time"

    "github.com/gin-gonic/gin"
    "go.uber.org/zap"

    "digital.vasic/ratelimiter"
    "helixterm.io/pkg/shared/ratelimit"
)

// PerUserRateLimit enforces per-user rate limiting at the
// API Gateway.
func PerUserRateLimit(limiter *ratelimit.Limiter, rpm int,
    logger *zap.Logger) gin.HandlerFunc {

```

```

return func(c *gin.Context) {
    userID, exists := c.Get(CtxKeyUserID)
    if !exists {
        // Fall through if no user ID – JWT middleware
        hasn't run yet
        // (should not happen in normal flow but handle
        defensively).
        c.Next()
        return
    }

    key := fmt.Sprintf("user:%v:global", userID)
    result, err := limiter.Allow(c.Request.Context(),
    key, ratelimiter.Policy{
        Algorithm: ratelimiter.TokenBucket,
        Rate:      float64(rpm) / 60.0,
        Burst:     rpm * 2,
        WindowSize: time.Minute,
    })
    if err != nil {
        logger.Error("rate limiter error",
        zap.Error(err))
        c.Next() // fail open on limiter errors
        return
    }

    // Set rate limit headers regardless of allow/deny.
    c.Header("X-RateLimit-Limit", strconv.Itoa(rpm))
    c.Header("X-RateLimit-Remaining",
    strconv.Itoa(result.Remaining))
    c.Header("X-RateLimit-Reset",
    strconv.FormatInt(result.ResetAt.Unix(), 10))

    if !result.Allowed {
        c.Header("Retry-After",
        strconv.FormatInt(int64(result.RetryAfter.Seconds()),
        10))

        c.AbortWithStatusJSON(http.StatusTooManyRequests,
        gin.H{
            "error":      "rate limit exceeded",
            "retry_after": result.RetryAfter.Seconds(),
        })
        return
    }

    c.Next()
}

}

// PerEndpointRateLimit enforces stricter limits on specific
endpoints.

```

```

// endpointKey is a short identifier like "auth:login" or
// "vault:decrypt".
func PerEndpointRateLimit(limiter *ratelimit.Limiter,
    endpointKey string, rpm int, logger *zap.Logger)
    gin.HandlerFunc {
    return func(c *gin.Context) {
        userID, _ := c.Get(CtxKeyUserID)
        key := fmt.Sprintf("endpoint:%s:user:%v",
            endpointKey, userID)

        result, err := limiter.Allow(c.Request.Context(),
            key, ratelimiter.Policy{
                Algorithm: ratelimiter.FixedWindow,
                Rate:        float64(rpm) / 60.0,
                Burst:       rpm,
                WindowSize: time.Minute,
            })
        if err != nil {
            logger.Error("endpoint rate limiter error",
                zap.String("endpoint", endpointKey), zap.Error(err))
            c.Next()
            return
        }

        if !result.Allowed {
            c.AbortWithStatusJSON(http.StatusTooManyRequests,
                gin.H{
                    "error":        "endpoint rate limit
exceeded",
                    "endpoint":    endpointKey,
                    "retry_after": result.RetryAfter.Seconds(),
                })
            return
        }
        c.Next()
    }
}

```

9.4 Go: Auth Brute-Force Protection

```

// File: helixterm.io/services/auth/internal/handler/
//       brute_force.go
package handler

import (
    "context"
    "fmt"
    "time"

```

```

    "digital.vasic/ratelimiter"
    "helixterm.io/pkg/shared/ratelimit"
)

// BruteForceGuard enforces progressive logout on failed
// login attempts.
type BruteForceGuard struct {
    limiter *ratelimit.Limiter
}

// CheckAndRecord checks whether a login attempt is allowed
// and records the attempt.
// Returns (allowed bool, logoutUntil time.Time, error).
func (g *BruteForceGuard) CheckAndRecord(ctx
    context.Context, identifier string) (bool,
    time.Time, error) {
    key := fmt.Sprintf("auth:brute:%s", identifier)

    // 5 attempts per minute; after exhaustion, locked for 15
    // minutes.
    result, err := g.limiter.Allow(ctx, key,
        ratelimit.Policy{
            Algorithm: ratelimit.SlidingWindow,
            Rate:      5.0 / 60.0, // 5 attempts per minute
            Burst:     5,
            WindowSize: time.Minute,
            LockoutTime: 15 * time.Minute,
        })
    if err != nil {
        return false, time.Time{}, fmt.Errorf("brute_force:
        checking limiter: %w", err)
    }

    return result.Allowed, result.ResetAt, nil
}

```

9.5 Go: Dynamic Rate Limit Adjustment

```

// File: helixterm.io/services/api-gateway/internal/
//       ratelimit/dynamic.go
package ratelimit

import (
    "context"
    "fmt"

    "digital.vasic/ratelimiter"
    "helixterm.io/pkg/shared/ratelimit"
)

```

```

)

// DynamicController allows runtime adjustment of rate limits
// per org/user.
// Used by the billing service to apply plan-based limits.
type DynamicController struct {
    limiter *ratelimit.Limiter
}

// SetOrgLimit overrides the rate limit for a specific
// organisation.
// plan is "free" | "pro" | "enterprise".
func (d *DynamicController) SetOrgLimit(ctx context.Context,
    orgID, plan string) error {
    limits := map[string]int{
        "free":      60,    // 60 RPM
        "pro":       600,   // 600 RPM
        "enterprise": 6000, // 6000 RPM
    }

    rpm, ok := limits[plan]
    if !ok {
        return fmt.Errorf("dynamic_rl: unknown plan %q",
            plan)
    }

    key := fmt.Sprintf("org:%s:global", orgID)
    return d.limiter.SetPolicy(ctx, key, ratelimiter.Policy{
        Algorithm: ratelimiter.TokenBucket,
        Rate:      float64(rpm) / 60.0,
        Burst:     rpm * 2,
        WindowSize: 60,
    })
}

```

10. digital.vasic.recovery

Import path: digital.vasic/recovery

Go module: digital.vasic/recovery v1.x.x

10.1 Purpose

digital.vasic.recovery provides circuit breaker, bulkhead, and timeout patterns built on top of Sony's Gobreaker and the digital.vasic ecosystem. It protects HelixTerminator's inter-service calls from cascading failures.

10.2 Go: Circuit Breaker Initialization

```
// File: helixterm.io/pkg/shared/circuitbreaker/
// circuitbreaker.go
package circuitbreaker

import (
    "context"
    "fmt"
    "time"

    "digital.vasic/recovery"
    "digital.vasic/recovery/breaker"
    "go.uber.org/zap"
)

// Config holds circuit breaker settings.
type Config struct {
    Name                string
    MaxRequests         uint32           // max requests in half-
                       open state
    Interval            time.Duration // window for counting
                       failures
    Timeout             time.Duration // time in open state
                       before transitioning to half-open
    FailureRatioThreshold float64        // ratio of failures to
                       open the circuit (0.0-1.0)
    MinRequests         uint32           // minimum requests
                       before evaluating failure ratio
}

// Breaker wraps digital.vasic.recovery's circuit breaker.
type Breaker struct {
    cb    breaker.CircuitBreaker
    logger *zap.Logger
}

// New creates a circuit breaker.
func New(cfg Config, logger *zap.Logger) (*Breaker, error) {
    cb, err := breaker.New(breaker.Config{
        Name:                cfg.Name,
        MaxRequests:         cfg.MaxRequests,
        Interval:            cfg.Interval,
        Timeout:             cfg.Timeout,
        ReadyToTrip:         breaker.FailureRatio(cfg.FailureRatioThreshold,
        cfg.MinRequests),
        OnStateChange: func(name string, from, to
        breaker.State) {
            logger.Warn("circuit breaker state change",

```

```

        zap.String("name", name),
        zap.String("from", from.String()),
        zap.String("to", to.String()),
    )
    },
})
if err != nil {
    return nil, fmt.Errorf("circuit_breaker: creating
        breaker %q: %w", cfg.Name, err)
}

return &Breaker{cb: cb, logger: logger}, nil
}

// Execute wraps a function call with circuit breaker
// protection.
func (b *Breaker) Execute(ctx context.Context, fn func(ctx
    context.Context) error) error {
    _, err := b.cb.Execute(func() (interface{}, error) {
        return nil, fn(ctx)
    })
    if err != nil {
        if err == breaker.ErrOpenState {
            return
            fmt.Errorf("circuit_breaker: circuit %q is open
                (service unavailable)", b.cb.Name())
        }
        return err
    }
    return nil
}

```

10.3 Go: SSH Proxy → Auth Service with Circuit Breaker

```

// File: helixterm.io/services/ssh-proxy/internal/auth/
// auth_client.go
package auth

import (
    "context"
    "fmt"
    "time"

    "go.uber.org/zap"
    "google.golang.org/grpc"

    "digital.vasic/recovery/breaker"
    "helixterm.io/pkg/shared/circuitbreaker"

```

```

    authpb "helixterm.io/proto/auth/v1"
)

// Client wraps the Auth Service gRPC client with a circuit
// breaker.
type Client struct {
    grpc    authpb.AuthServiceClient
    breaker *circuitbreaker.Breaker
    logger  *zap.Logger
}

// NewClient creates an Auth gRPC client protected by a
// circuit breaker.
func NewClient(conn *grpc.ClientConn, logger *zap.Logger)
(*Client, error) {
    cb, err := circuitbreaker.New(circuitbreaker.Config{
        Name:                "ssh-proxy->auth",
        MaxRequests:         5,
        Interval:            30 * time.Second,
        Timeout:             10 * time.Second,
        FailureRatioThreshold: 0.5, // open if >50% of
        requests fail
        MinRequests:        10,
    }, logger)
    if err != nil {
        return nil, err
    }

    return &Client{
        grpc:    authpb.NewAuthServiceClient(conn),
        breaker: cb,
        logger:  logger,
    }, nil
}

// ValidateToken calls Auth Service to validate a JWT.
// Falls back to a cached result if the circuit is open.
func (c *Client) ValidateToken(ctx context.Context, token
string) (*authpb.ValidateResponse, error) {
    var resp *authpb.ValidateResponse

    err := c.breaker.Execute(ctx, func(ctx context.Context)
error {
        var grpcErr error
        resp, grpcErr = c.grpc.ValidateToken(ctx,
            &authpb.ValidateTokenRequest{Token: token})
        return grpcErr
    })

    if err != nil {
        c.logger.Error("auth client: ValidateToken failed",
            zap.Error(err))
    }
}

```

```

        return nil, fmt.Errorf("auth_client: validating
        token: %w", err)
    }

    return resp, nil
}

// ValidateTokenWithFallback falls back to offline JWT
// verification
// if the circuit breaker is open (avoids complete
// authentication failure).
func (c *Client) ValidateTokenWithFallback(ctx
    context.Context, token string, fallback func(string)
    (*authpb.ValidateResponse, error))
    (*authpb.ValidateResponse, error) {
    resp, err := c.ValidateToken(ctx, token)
    if err != nil {
        c.logger.Warn("auth circuit open, using offline
        fallback", zap.Error(err))
        return fallback(token)
    }
    return resp, nil
}

```

10.4 Go: Bulkhead Pattern

```

// File: helixterm.io/pkg/shared/bulkhead/bulkhead.go
package bulkhead

import (
    "context"
    "fmt"

    "digital.vasic/recovery/bulkhead"
)

// Bulkhead limits the number of concurrent calls to a
// downstream resource.
// This prevents a slow dependency from consuming all
// goroutines.
type Bulkhead struct {
    bh bulkhead.Bulkhead
}

// New creates a bulkhead with the given max concurrency.
func New(name string, maxConcurrent int) (*Bulkhead, error) {
    bh, err := bulkhead.New(bulkhead.Config{
        Name:          name,
        MaxConcurrent: maxConcurrent,
    })
    if err != nil {
        return nil, err
    }
    return &Bulkhead{bh: bh}, nil
}

```

```

        MaxWaitTime: 0, // 0 = reject immediately when full
    })
    if err != nil {
        return nil, fmt.Errorf("bulkhead: creating %q: %w",
            name, err)
    }
    return &Bulkhead{bh: bh}, nil
}

// Execute runs fn if a slot is available, otherwise returns
// ErrFull.
func (b *Bulkhead) Execute(ctx context.Context, fn func(ctx
    context.Context) error) error {
    if err := b.bh.Acquire(ctx); err != nil {
        return fmt.Errorf("bulkhead: %q is full: %w",
            b.bh.Name(), err)
    }
    defer b.bh.Release()
    return fn(ctx)
}

```

11. digital.vasic.concurrency

Import path: digital.vasic/concurrency

Go module: digital.vasic/concurrency v1.x.x

11.1 Purpose

digital.vasic.concurrency provides worker pool primitives, semaphores, and errgroup wrappers for structured concurrency. In HelixTerminator it is used by the SSH Proxy for goroutine pool-managed sessions, the Vault Service for parallel encryption operations, and the Terminal Service for fan-out to collaboration subscribers.

11.2 Go: Worker Pool for SSH Session Management

```

// File: helixterm.io/services/ssh-proxy/internal/pool/
//       session_pool.go
package pool

import (
    "context"
    "fmt"

    "digital.vasic/concurrency"
    "digital.vasic/concurrency/workerpool"
    "go.uber.org/zap"

```

```

        "helixterm.io/services/ssh-proxy/internal/session"
    )

    // SessionPool manages SSH sessions using a bounded goroutine
    pool.
    type SessionPool struct {
        pool    workerpool.Pool
        logger *zap.Logger
    }

    // NewSessionPool creates a worker pool for SSH session
    management.
    func NewSessionPool(maxWorkers int, queueDepth int, logger
        *zap.Logger) (*SessionPool, error) {
        pool, err := workerpool.New(workerpool.Config{
            Workers:    maxWorkers,
            QueueDepth: queueDepth,
            Name:       "ssh-session-pool",
        })
        if err != nil {
            return nil,
                fmt.Errorf("session_pool: creating worker pool: %w",
                    err)
        }

        logger.Info("SSH session pool initialised",
            zap.Int("max_workers", maxWorkers),
            zap.Int("queue_depth", queueDepth),
        )

        return &SessionPool{pool: pool, logger: logger}, nil
    }

    // SubmitSession enqueues an SSH session handler to be
    executed by the pool.
    func (p *SessionPool) SubmitSession(ctx context.Context, s
        *session.Session) error {
        return p.pool.Submit(ctx, func(ctx context.Context)
            error {
                return s.Handle(ctx)
            })
    }

    // Stop gracefully drains the pool, waiting for all active
    sessions to finish.
    func (p *SessionPool) Stop(ctx context.Context) {
        p.logger.Info("stopping SSH session pool")
        p.pool.Stop(ctx)
    }

```

11.3 Go: Semaphore for Concurrent DB Writes

```
// File: helixterm.io/services/vault/internal/crypto/
//       parallel_encrypt.go
package crypto

import (
    "context"
    "fmt"

    "digital.vasic/concurrency/semaphore"
)

// ParallelEncryptor limits concurrent encryption operations
// to avoid saturating the CPU when processing bulk vault
// imports.
type ParallelEncryptor struct {
    mgr *Manager
    sem *semaphore.Semaphore
}

// NewParallelEncryptor creates a parallel encryptor with a
// concurrency limit.
func NewParallelEncryptor(mgr *Manager, maxConcurrent int)
    *ParallelEncryptor {
    return &ParallelEncryptor{
        mgr: mgr,
        sem: semaphore.New(maxConcurrent),
    }
}

// EncryptBatch encrypts multiple payloads in parallel, up to
// maxConcurrent at once.
func (pe *ParallelEncryptor) EncryptBatch(ctx
    context.Context, payloads []VaultPayload)
    ([]*EncryptedBlob, error) {
    results := make([]*EncryptedBlob, len(payloads))
    errs := make([]error, len(payloads))

    var wg sync.WaitGroup
    for i, payload := range payloads {
        wg.Add(1)
        go func(idx int, p VaultPayload) {
            defer wg.Done()

            if err := pe.sem.Acquire(ctx); err != nil {
                errs[idx] = fmt.Errorf("parallel_encrypt:
                acquiring semaphore: %w", err)
                return
            }
        }(i, payload)
    }
    wg.Wait()

    return results, errs
}
```

```

        defer pe.sem.Release()

        blob, err := pe.mgr.EncryptVaultPayload(ctx, p)
        if err != nil {
            errs[idx] = fmt.Errorf("parallel_encrypt:
encrypting item %d: %w", idx, err)
            return
        }
        results[idx] = blob
    }(i, payload)
}
wg.Wait()

for _, err := range errs {
    if err != nil {
        return nil, err
    }
}
return results, nil
}

```

11.4 Go: errgroup for Parallel Service Calls

```

// File: helixterm.io/services/terminal/internal/handler/
//        collaboration.go
package handler

import (
    "context"
    "fmt"

    "golang.org/x/sync/errgroup"

    "helixterm.io/services/terminal/internal/model"
)

// broadcastToCollaborators fans out a terminal event to all
// collaboration
// subscribers in parallel using errgroup for error
// aggregation.
func (h *TerminalHandler) broadcastToCollaborators(ctx
context.Context, event model.TerminalEvent,
subscriberIDs []string) error {
    g, ctx := errgroup.WithContext(ctx)

    for _, subID := range subscriberIDs {
        subID := subID // capture loop variable
        g.Go(func() error {

```

```

        if err := h.notifier.SendToSubscriber(ctx,
subID, event); err != nil {
            // Log but treat individual subscriber
failures as non-fatal.
            h.logger.Warn("collaboration: failed to
notify subscriber",
                zap.String("subscriber_id", subID),
                zap.Error(err),
            )
            // Return nil so one slow subscriber doesn't
block others.
            return nil
        }
        return nil
    })
}

if err := g.Wait(); err != nil {
    return fmt.Errorf("collaboration: broadcast failed:
%w", err)
}
return nil
}

```

12. digital.vasic.containers

Import path: digital.vasic/containers

Go module: digital.vasic/containers v1.x.x

12.1 Purpose

digital.vasic.containers abstracts container runtime operations (Docker Engine API, Podman socket, Kubernetes exec) behind a unified ContainerRuntime interface. In HelixTerminator it powers:

- Container Bridge Service (helixterm.io/services/container-bridge): full lifecycle management
- SSH Proxy: container-exec sessions (equivalent to docker exec -it)

12.2 Go: Runtime Detection and Initialization

```

// File: helixterm.io/services/container-bridge/internal/
runtime/runtime.go
package runtime

import (
    "context"

```

```

    "fmt"
    "os"

    "digital.vasic/containers"
    "digital.vasic/containers/docker"
    "digital.vasic/containers/kubernetes"
    "digital.vasic/containers/podman"
    "go.uber.org/zap"
)

// AutoDetect probes the host environment to determine the
// available
// container runtime and returns an initialised
// ContainerRuntime.
// Detection order: Kubernetes (in-cluster) → Docker →
// Podman.
func AutoDetect(ctx context.Context, logger *zap.Logger)
    (containers.Runtime, error) {
    // 1. Check for Kubernetes in-cluster environment.
    if _, err := os.Stat("/var/run/secrets/kubernetes.io/
        serviceaccount/token"); err == nil {
        logger.Info("container runtime: detected Kubernetes
            in-cluster")
        rt, err := kubernetes.NewInClusterRuntime(ctx,
            kubernetes.Config{
                Namespace: os.Getenv("HELIX_K8S_NAMESPACE"),
            })
        if err != nil {
            logger.Warn("k8s runtime init failed, trying
                Docker", zap.Error(err))
        } else {
            return rt, nil
        }
    }

    // 2. Try Docker socket.
    dockerSocket := "/var/run/docker.sock"
    if v := os.Getenv("DOCKER_HOST"); v != "" {
        dockerSocket = v
    }
    if _, err := os.Stat(dockerSocket); err == nil {
        logger.Info("container runtime: detected Docker",
            zap.String("socket", dockerSocket))
        rt, err := docker.NewRuntime(ctx, docker.Config{
            SocketPath: dockerSocket,
            APIVersion: "1.47",
        })
        if err != nil {
            logger.Warn("Docker runtime init failed, trying
                Podman", zap.Error(err))
        } else {
            return rt, nil
        }
    }
}

```

```

    }
}

// 3. Try Podman socket.
podmanSocket := "/run/user/1000/podman/podman.sock"
if v := os.Getenv("PODMAN_SOCKET"); v != "" {
    podmanSocket = v
}
if _, err := os.Stat(podmanSocket); err == nil {
    logger.Info("container runtime: detected Podman",
        zap.String("socket", podmanSocket))
    return podman.NewRuntime(ctx,
        podman.Config{SocketPath: podmanSocket})
}

return nil, fmt.Errorf("container_runtime: no container
    runtime detected on this host")
}

```

12.3 Go: Container Bridge Service Handler

```

// File: helixterm.io/services/container-bridge/internal/
//       handler/container_handler.go
package handler

import (
    "context"
    "io"
    "net/http"

    "github.com/gin-gonic/gin"
    "go.uber.org/zap"

    "digital.vasic/containers"
    "helixterm.io/services/container-bridge/internal/model"
)

// ContainerHandler provides container lifecycle endpoints.
type ContainerHandler struct {
    runtime containers.Runtime
    logger  *zap.Logger
}

// ListContainers returns all containers on the host.
// GET /v1/containers?host_id=<id>
func (h *ContainerHandler) ListContainers(c *gin.Context) {
    hostID := c.Query("host_id")
    ctx := c.Request.Context()

```

```

containers, err := h.runtime.ListContainers(ctx,
    containers.ListOptions{
        All: true, // include stopped containers
    })
if err != nil {
    h.logger.Error("list containers failed",
        zap.String("host_id", hostID), zap.Error(err))
    c.JSON(http.StatusInternalServerError,
        gin.H{"error": err.Error()})
    return
}

c.JSON(http.StatusOK, gin.H{"containers":
    mapContainers(containers), "host_id": hostID})
}

// ExecInContainer opens an interactive exec session inside a
// container.
// This is used by the SSH Proxy for container-exec terminal
// sessions.
// POST /v1/containers/:id/exec
func (h *ContainerHandler) ExecInContainer(c *gin.Context) {
    containerID := c.Param("id")
    var req model.ExecRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error":
            err.Error()})
        return
    }

    ctx := c.Request.Context()

    execID, err := h.runtime.ExecCreate(ctx, containerID,
        containers.ExecConfig{
            Cmd:          req.Command,
            Env:           req.Env,
            WorkingDir:    req.WorkingDir,
            AttachStdin:  true,
            AttachStdout: true,
            AttachStderr: true,
            Tty:           req.Tty,
            User:          req.User,
        })
    if err != nil {
        h.logger.Error("exec create failed",
            zap.String("container_id", containerID),
            zap.Error(err))
        c.JSON(http.StatusInternalServerError,
            gin.H{"error": err.Error()})
        return
    }
}

```

```

// Upgrade the HTTP connection to a bidirectional stream.
// The SSH Proxy reads/writes this stream and multiplexes
// it over the SSH channel.
conn, err := h.runtime.ExecAttach(ctx, execID,
    containers.ExecAttachConfig{
        Stream: true,
        Stdin:  true,
        Stdout: true,
        Stderr: true,
    })
if err != nil {
    h.logger.Error("exec attach failed",
        zap.String("exec_id", execID), zap.Error(err))
    c.JSON(http.StatusInternalServerError,
        gin.H{"error": err.Error()})
    return
}
defer conn.Close()

// Hijack the HTTP connection and stream bidirectionally.
hijack(c, conn.Reader, conn.Writer)
}

// GetLogs streams container logs.
// GET /v1/containers/:id/logs
func (h *ContainerHandler) GetLogs(c *gin.Context) {
    containerID := c.Param("id")
    ctx := c.Request.Context()

    follow := c.Query("follow") == "true"
    tail := c.DefaultQuery("tail", "100")

    reader, err := h.runtime.GetLogs(ctx, containerID,
        containers.LogOptions{
            Follow:    follow,
            Tail:      tail,
            Timestamps: true,
            Stdout:    true,
            Stderr:    true,
        })
    if err != nil {
        c.JSON(http.StatusInternalServerError,
            gin.H{"error": err.Error()})
        return
    }
    defer reader.Close()

    c.Stream(func(w io.Writer) bool {
        _, err := io.Copy(w, reader)
        return err == nil
    })
}

```

```

// InspectContainer returns container metadata.
// GET /v1/containers/:id/inspect
func (h *ContainerHandler) InspectContainer(c *gin.Context) {
    containerID := c.Param("id")
    ctx := c.Request.Context()

    info, err := h.runtime.InspectContainer(ctx, containerID)
    if err != nil {
        c.JSON(http.StatusInternalServerError,
            gin.H{"error": err.Error()})
        return
    }

    c.JSON(http.StatusOK, info)
}

// PullImage pulls a container image.
// POST /v1/images/pull
func (h *ContainerHandler) PullImage(c *gin.Context) {
    var req model.PullImageRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error":
            err.Error()})
        return
    }

    ctx := c.Request.Context()

    reader, err := h.runtime.PullImage(ctx, req.Image,
        containers.PullOptions{
            RegistryAuth: req.RegistryAuth,
        })
    if err != nil {
        c.JSON(http.StatusInternalServerError,
            gin.H{"error": err.Error()})
        return
    }
    defer reader.Close()

    // Stream pull progress to the client.
    c.Stream(func(w io.Writer) bool {
        _, err := io.Copy(w, reader)
        return err == nil
    })
}

```

12.4 Flutter Client Integration

On the Flutter side, container operations are surfaced through the `ContainerBridgeClient` :

```
// File: lib/src/containers/container_bridge_client.dart
import 'package:http/http.dart' as http;
import 'dart:convert';

/// Client for the Container Bridge Service.
class ContainerBridgeClient {
  final String baseUrl;
  final String authToken;
  final http.Client _client;

  ContainerBridgeClient({
    required this.baseUrl,
    required this.authToken,
    http.Client? client,
  }) : _client = client ?? http.Client();

  Future<List<Container>> listContainers({required String
    hostID}) async {
    final uri = Uri.parse('$baseUrl/v1/containers').replace(
      queryParameters: {'host_id': hostID},
    );

    final response = await _client.get(uri, headers:
      _headers());
    _checkStatus(response, 200);

    final json = jsonDecode(response.body) as Map<String,
      dynamic>;
    final items = (json['containers'] as
      List).cast<Map<String, dynamic>>();
    return items.map(Container.fromJson).toList();
  }

  Future<ContainerInfo> inspectContainer({
    required String hostID,
    required String containerID,
  }) async {
    final uri = Uri.parse('$baseUrl/v1/containers/
      $containerID/inspect');
    final response = await _client.get(uri, headers:
      _headers());
    _checkStatus(response, 200);
    return ContainerInfo.fromJson(jsonDecode(response.body));
  }
}
```

```

Map<String, String> _headers() => {
    'Authorization': 'Bearer $authToken',
    'Content-Type': 'application/json',
};

void _checkStatus(http.Response res, int expected) {
    if (res.statusCode != expected) {
        throw ContainerBridgeException(
            'HTTP ${res.statusCode}: ${res.body}',
            statusCode: res.statusCode,
        );
    }
}
}

class ContainerBridgeException implements Exception {
    final String message;
    final int statusCode;
    const ContainerBridgeException(this.message, {required
        this.statusCode});
}

```

13. digital.vasic.docs_chain

Import path: digital.vasic/docs_chain (CLI tool: docs-chain)

Go module: digital.vasic/docs_chain v1.x.x

13.1 Purpose

digital.vasic.docs_chain is a Salsa-style DAG (Directed Acyclic Graph) document dependency engine. It tracks inter-document references, validates consistency (e.g., API names referenced in the integration spec must match those defined in the API spec), applies transforms (Pandoc HTML, WeasyPrint PDF, Pandoc DOCX), and provides CI verification commands.

13.2 docs-chain.yaml – Complete HelixTerminator Manifest

```

# docs-chain.yaml
# HelixTerminator specification document dependency graph.
# This file is read by the `docs-chain` CLI tool.

version: "1"
root: docs/

documents:
  - id: 01_architecture
    path: docs/01_architecture.md

```

```
title: "HelixTerminator Architecture Overview"
transforms:
  - pandoc-html
  - weasyprint-pdf

- id: 02_api_spec
  path: docs/02_api_spec.md
  title: "REST API Specification"
  dependencies: [01_architecture]
  transforms:
    - pandoc-html
    - pandoc-docx

- id: 03_data_model
  path: docs/03_data_model.md
  title: "Data Model & Schema Definitions"
  dependencies: [01_architecture]
  transforms:
    - pandoc-html

- id: 04_auth_flows
  path: docs/04_auth_flows.md
  title: "Authentication & Authorization Flows"
  dependencies: [01_architecture, 02_api_spec]
  transforms:
    - pandoc-html
    - weasyprint-pdf

- id: 05_ssh_protocol
  path: docs/05_ssh_protocol.md
  title: "SSH Protocol Implementation"
  dependencies: [01_architecture, 03_data_model]
  transforms:
    - pandoc-html

- id: 06_vault_design
  path: docs/06_vault_design.md
  title: "Vault Service Design"
  dependencies: [01_architecture, 03_data_model,
    04_auth_flows]
  transforms:
    - pandoc-html
    - weasyprint-pdf

- id: 07_container_bridge
  path: docs/07_container_bridge.md
  title: "Container Bridge Service"
  dependencies: [01_architecture, 05_ssh_protocol]
  transforms:
    - pandoc-html
```

```

- id: 08_deployment
  path: docs/08_deployment.md
  title: "Kubernetes Deployment & Operations"
  dependencies: [01_architecture]
  transforms:
    - pandoc-html
    - weasyprint-pdf

- id: 09_client_sdk
  path: docs/09_client_sdk.md
  title: "Flutter Client SDK"
  dependencies: [02_api_spec, 04_auth_flows]
  transforms:
    - pandoc-html
    - pandoc-docx

- id: 10_submodule_integration
  path: docs/10_submodule_integration.md
  title: "Submodule Integration Specification"
  dependencies:
    - 01_architecture
    - 02_api_spec
    - 03_data_model
    - 04_auth_flows
    - 05_ssh_protocol
    - 06_vault_design
    - 07_container_bridge
    - 08_deployment
    - 09_client_sdk
  transforms:
    - pandoc-html
    - weasyprint-pdf
    - pandoc-docx

- id: 11_runbooks
  path: docs/11_runbooks.md
  title: "Operational Runbooks"
  dependencies: [08_deployment, 10_submodule_integration]
  transforms:
    - pandoc-html

consistency_checks:
  # Verify that all service names referenced in
    10_submodule_integration
  # are present in 01_architecture's service registry.
- type: cross_reference
  source: 10_submodule_integration
  target: 01_architecture
  pattern: "helixterm.io/services/([a-z-]+)"
  field: service_registry

```

```

# Verify that all API endpoints mentioned in submodule
  integration
# match those defined in the API spec.
- type: cross_reference
  source: 10_submodule_integration
  target: 02_api_spec
  pattern: "(GET|POST|PUT|PATCH|DELETE) /v1/([a-z/-]+)"
  field: endpoints

transforms:
  pandoc-html:
    command: pandoc
    args: ["--from=markdown", "--to=html5", "--standalone",
           "--css=docs/style.css"]
    output_suffix: ".html"
    output_dir: docs/html/

  weasyprint-pdf:
    command: weasyprint
    args: ["--presentational-hints"]
    input_from: pandoc-html
    output_suffix: ".pdf"
    output_dir: docs/pdf/

  pandoc-docx:
    command: pandoc
    args: ["--from=markdown", "--to=docx", "--reference-
           doc=docs/template.docx"]
    output_suffix: ".docx"
    output_dir: docs/docx/

ci:
  fail_on_cycle: true
  fail_on_broken_reference: true
  fail_on_missing_transform:
    false # transforms are optional in dev
  cache_transforms: true
  cache_dir: .docs-chain-cache/

```

13.3 CLI Commands Used in CI

```

# Verify document graph integrity – checks for cycles,
  missing dependencies,
# and broken cross-references. Exit code 0 = pass.
docs-chain verify --config docs-chain.yaml

# Sync all documents – applies transforms and generates
  output artifacts.
docs-chain sync --config docs-chain.yaml --env production

```

```
# Doctor command – diagnoses configuration issues.
docs-chain doctor --config docs-chain.yaml

# Graph command – outputs a Mermaid or DOT representation of
the DAG.
docs-chain graph --config docs-chain.yaml --format mermaid >
docs/dep-graph.mmd

# Watch mode – re-generates affected outputs when source
documents change.
docs-chain watch --config docs-chain.yaml
```

14. digital.vasic.challenges

Import path: `digital.vasic/challenges`

Go module: `digital.vasic/challenges v1.x.x`

14.1 Architecture and Purpose

`digital.vasic.challenges` is a structured learning and assessment framework. It provides:

- **Challenge definitions:** JSON/YAML schemas for challenges with prerequisites, criteria, and scoring
- **Attempt tracking:** records of user attempts with partial credit
- **AI generation hooks:** callbacks to AI providers for dynamic challenge synthesis
- **Progress events:** Kafka events when users complete challenges

Within HelixTerminator, challenges power the **SSH Onboarding Track** — a gamified learning path that teaches users SSH best practices as they use the terminal.

14.2 Challenge Types in HelixTerminator

ID	Type	Name	Description
ssh-keygen-101	command_execution	Generate your first SSH key	User must run <code>ssh-keygen -t ed25519</code>
ssh-copy-id-101	command_execution	Copy SSH key to host	User must successfully copy public key
	file_creation		

ID	Type	Name	Description
ssh-config-101		Create SSH config entry	User creates ~/.ssh/config entry
tmux-basics	command_execution	Tmux session management	User creates, detaches, reattaches tmux
port-forwarding	command_execution	Local port forwarding	User establishes -L forward
sftp-upload	file_transfer	Upload file via SFTP	User uploads file via SFTP
vault-first-item	ui_action	Store first vault item	User stores a credential

14.3 Go: Challenge Handler

```
// File: helixterm.io/services/onboarding/internal/handler/
//       challenge_handler.go
package handler

import (
    "net/http"

    "github.com/gin-gonic/gin"
    "go.uber.org/zap"

    "digital.vasic/challenges"
    "digital.vasic/challenges/attempt"
    "helixterm.io/services/onboarding/internal/model"
    "helixterm.io/services/onboarding/internal/repository"
    "helixterm.io/pkg/shared/kafkaproducer"
    "helixterm.io/pkg/shared/topics"
)

// ChallengeHandler manages challenge operations.
type ChallengeHandler struct {
    registry    challenges.Registry
    tracker     attempt.Tracker
    repo        repository.ChallengeRepository
    producer    *kafkaproducer.Producer
    aiClient    AIClient
    logger      *zap.Logger
}
```

```

// ListChallenges returns all challenges available to the
// authenticated user.
// GET /v1/challenges
func (h *ChallengeHandler) ListChallenges(c *gin.Context) {
    userID := c.GetString("helix_user_id")
    ctx := c.Request.Context()

    all, err := h.registry.List(ctx)
    if err != nil {
        h.logger.Error("challenges: list failed",
            zap.Error(err))
        c.JSON(http.StatusInternalServerError,
            gin.H{"error": err.Error()})
        return
    }

    // Load the user's completion status for each challenge.
    statuses, err := h.repo.GetUserStatuses(ctx, userID)
    if err != nil {
        h.logger.Error("challenges: loading user statuses",
            zap.Error(err))
    }

    var resp []model.ChallengeListItem
    for _, ch := range all {
        status := "available"
        if s, ok := statuses[ch.ID]; ok {
            status = s
        }
        // Check prerequisites.
        if !h.prerequisitesMet(statuses, ch.Prerequisites) {
            status = "locked"
        }
        resp = append(resp, model.ChallengeListItem{
            ID:          ch.ID,
            Title:        ch.Title,
            Description:  ch.Description,
            Type:         string(ch.Type),
            Points:      ch.Points,
            Status:      status,
            Prerequisites: ch.Prerequisites,
        })
    }

    c.JSON(http.StatusOK, gin.H{"challenges": resp})
}

// SubmitAttempt records a challenge attempt and evaluates
// it.
// POST /v1/challenges/:id/attempt
func (h *ChallengeHandler) SubmitAttempt(c *gin.Context) {
    challengeID := c.Param("id")

```

```

userID := c.GetString("helix_user_id")
orgID := c.GetString("helix_org_id")
ctx := c.Request.Context()

var req model.AttemptRequest
if err := c.ShouldBindJSON(&req); err != nil {
    c.JSON(http.StatusBadRequest, gin.H{"error":
        err.Error()})
    return
}

ch, err := h.registry.Get(ctx, challengeID)
if err != nil {
    c.JSON(http.StatusNotFound, gin.H{"error":
        "challenge not found"})
    return
}

result, err := h.tracker.Evaluate(ctx,
    attempt.EvaluationRequest{
        Challenge: ch,
        UserID:    userID,
        Evidence:  req.Evidence,
    })
if err != nil {
    h.logger.Error("challenges: evaluation failed",
        zap.String("challenge_id", challengeID),
        zap.Error(err))
    c.JSON(http.StatusInternalServerError,
        gin.H{"error": err.Error()})
    return
}

if err := h.repo.RecordAttempt(ctx,
    repository.AttemptRecord{
        ChallengeID: challengeID,
        UserID:      userID,
        Passed:      result.Passed,
        Score:       result.Score,
        Feedback:    result.Feedback,
    }); err != nil {
    h.logger.Error("challenges: recording attempt",
        zap.Error(err))
}

if result.Passed {
    // Publish completion event to Kafka.
    event := map[string]interface{}{
        "user_id":      userID,
        "org_id":       orgID,
        "challenge_id": challengeID,
        "score":        result.Score,
    }
}

```

```

        "points":      ch.Points,
    }
    if err := h.producer.Publish(ctx,
        topics.TopicAnalyticsCommand, userID, event); err !=
        nil {

        h.logger.Error("challenges: publishing completion
            event", zap.Error(err))
    }
}

c.JSON(http.StatusOK, model.AttemptResponse{
    Passed:    result.Passed,
    Score:     result.Score,
    Feedback:  result.Feedback,
    Points:    ch.Points,
})
}

// GenerateAIChallenge requests the AI Service to generate a
// dynamic challenge.
// POST /v1/challenges/generate
func (h *ChallengeHandler) GenerateAIChallenge(c
    *gin.Context) {
    userID := c.GetString("helix_user_id")
    ctx := c.Request.Context()

    var req model.GenerateRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error":
            err.Error()})
        return
    }

    ch, err := h.aiClient.GenerateChallenge(ctx, userID,
        req.Topic, req.Difficulty)
    if err != nil {
        h.logger.Error("challenges: AI generation failed",
            zap.Error(err))
        c.JSON(http.StatusInternalServerError,
            gin.H{"error": err.Error()})
        return
    }

    // Register the generated challenge in the registry for
    // this session.
    if err := h.registry.Register(ctx, *ch); err != nil {
        h.logger.Error("challenges: registering AI-generated
            challenge", zap.Error(err))
        c.JSON(http.StatusInternalServerError,
            gin.H{"error": err.Error()})
        return
    }
}

```

```

    }

    c.JSON(http.StatusCreated, ch)
}

func (h *ChallengeHandler) prerequisitesMet(statuses
    map[string]string, prereqs []string) bool {
    for _, prereq := range prereqs {
        if statuses[prereq] != "completed" {
            return false
        }
    }
    return true
}

```

15. helixqa

Source: HelixDevelopment

Config file: `.helixqa.yaml`

15.1 Purpose

`helixqa` is an AI-driven QA orchestration platform from HelixDevelopment. It:

- Generates test cases from OpenAPI specs
- Detects and quarantines flaky tests
- Runs mutation testing
- Aggregates coverage and pushes reports to the HelixQA dashboard
- Integrates with GitHub Actions via first-class workflow steps

15.2 Complete `.helixqa.yaml` for HelixTerminator

```

# .helixqa.yaml
# HelixTerminator – HelixQA Configuration
# See: https://helixdevelopment.io/helixqa/docs/config

version: "2"
project: helixterm
organization: helixterm-io

# API spec for AI-driven test generation.
openapi:
  path: docs/openapi.yaml
  version: "3.1.0"
  base_url: https://api.helixterm.io

# Test framework settings.

```

```
testing:
  framework: go-test
  test_binary_pattern: "./..."
  timeout: 10m
  parallel: true
  max_parallel: 8
  build_tags: []
  race_detector: true

# AI test generation configuration.
generation:
  enabled: true
  model: helixqa-codegen-v3
  target_coverage: 85
  generate_for:
    - services/api-gateway
    - services/auth
    - services/vault
    - services/ssh-proxy
    - services/terminal
    - services/host-manager
    - services/user
    - services/container-bridge
  exclude_patterns:
    - "**/*_generated.go"
    - "**/mock_*.go"
    - "**/testdata/**"
  output_dir: ".helixqa/generated_tests/"
  overwrite_existing: false # never overwrite human-written
                             tests

# Flaky test detection and quarantine.
flaky:
  enabled: true
  detection_strategy: statistical
  min_runs: 10
  flaky_threshold: 0.1 # >10% failure rate = flaky
  quarantine:
    enabled: true
    quarantine_dir: ".helixqa/quarantine/"
    notify_slack: true
    slack_channel: "#helixterm-qa"
    auto_rerun: true
    rerun_count: 3

# Mutation testing.
mutation:
  enabled: true
  tool: go-mutesting
  timeout: 30m
  target_packages:
```

```

- helixterm.io/services/vault/...
- helixterm.io/services/auth/...
- helixterm.io/pkg/shared/...
mutation_score_threshold:
    65 # fail CI if mutation score < 65%
exclude_patterns:
- "**/cmd/**"
- "**/migration/**"

# Coverage configuration.
coverage:
    minimum: 80 # fail CI if coverage < 80%
    per_package_minimum: 70 # individual package threshold
    report_format:
        - html
        - lcov
        - json
    upload_to_dashboard: true
    dashboard_url: https://qa.helixdevelopment.io
    dashboard_token_env: HELIXQA_DASHBOARD_TOKEN

# Service-specific overrides.
service_overrides:
    vault:
        minimum_coverage: 90
        mutation_score_threshold: 75
    auth:
        minimum_coverage: 90
        mutation_score_threshold: 70

# CI integration.
ci:
    provider: github-actions
    fail_fast: false # run all checks even if some fail
    artifact_retention: 30 # days
    comment_on_pr: true
    pr_comment_template: ".helixqa/templates/pr_comment.md"
    status_check_name: "helixqa / quality-gate"

# Notifications.
notifications:
    slack:
        enabled: true
        webhook_env: HELIXQA_SLACK_WEBHOOK
        events:
            - coverage_regression
            - mutation_score_regression
            - new_flaky_test
            - quarantine_cleared

```

```
email:  
  enabled: false
```

15.3 GitHub Actions Job for HelixQA

```
# .github/workflows/helixqa.yml  
name: HelixQA Quality Gate  
  
on:  
  push:  
    branches: [main, develop]  
  pull_request:  
    branches: [main, develop]  
  
permissions:  
  contents: read  
  checks: write  
  pull-requests: write  
  
jobs:  
  helixqa:  
    name: HelixQA / Quality Gate  
    runs-on: ubuntu-latest  
    timeout-minutes: 45  
  
    services:  
      postgres:  
        image: postgres:16-alpine  
        env:  
          POSTGRES_PASSWORD: test  
          POSTGRES_DB: helixterm_test  
        options: >-  
          --health-cmd pg_isready  
          --health-interval 10s  
          --health-timeout 5s  
          --health-retries 5  
  
      redis:  
        image: redis:7-alpine  
        options: >-  
          --health-cmd "redis-cli ping"  
          --health-interval 10s  
          --health-timeout 5s  
          --health-retries 5  
  
    steps:  
      - name: Checkout  
        uses: actions/checkout@v4
```

```

with:
  fetch-depth: 0

- name: Set up Go
  uses: actions/setup-go@v5
  with:
    go-version: "1.25"
    cache: true

- name: Install HelixQA CLI
  run: |
    curl -fsSL https://get.helixdevelopment.io/helixqa/
    install.sh | sh
    helixqa version

- name: Generate tests from OpenAPI spec
  run: helixqa generate --config .helixqa.yaml
  env:
    HELIXQA_API_KEY: ${ secrets.HELIXQA_API_KEY }

- name: Run tests with coverage
  run: helixqa test --config .helixqa.yaml --coverage
  env:
    TEST_DB_DSN: postgres://
    postgres:test@localhost:5432/helixterm_test?
    sslmode=disable
    TEST_REDIS_ADDR: localhost:6379
    HELIXQA_DASHBOARD_TOKEN: $
    ${ secrets.HELIXQA_DASHBOARD_TOKEN }

- name: Run mutation testing
  if: github.event_name == 'push' && github.ref ==
    'refs/heads/main'
  run: helixqa mutate --config .helixqa.yaml
  env:
    HELIXQA_API_KEY: ${ secrets.HELIXQA_API_KEY }

- name: Check for flaky tests
  run: helixqa flaky-check --config .helixqa.yaml
  env:
    HELIXQA_API_KEY: ${ secrets.HELIXQA_API_KEY }

- name: Upload coverage report
  run: helixqa coverage-upload --config .helixqa.yaml
  env:
    HELIXQA_DASHBOARD_TOKEN: $
    ${ secrets.HELIXQA_DASHBOARD_TOKEN }

- name: Quality gate evaluation
  run: helixqa gate --config .helixqa.yaml --fail-on-
    regression
  env:

```

```

    HELIXQA_API_KEY: ${ secrets.HELIXQA_API_KEY }}

- name: Upload test artifacts
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: helixqa-reports-${ github.run_id }}
    path: |
      .helixqa/reports/
      coverage.html
    retention-days: 30

- name: Comment on PR
  if: github.event_name == 'pull_request'
  run: helixqa pr-comment --config .helixqa.yaml
  env:
    GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }}
    HELIXQA_API_KEY: ${ secrets.HELIXQA_API_KEY }}

```

16. helixtrack.ru/core

Import path: helixtrack.ru/core

Go module: helixtrack.ru/core v1.x.x

16.1 Purpose

helixtrack.ru/core is the Helix-Track project management SDK. HelixTerminator integrates Helix-Track via the dedicated **HelixTrack Bridge Service** (helixterm.io/services/helixtrack-bridge), which:

- Associates SSH terminal sessions with Helix-Track tasks
- Updates task status based on detected terminal command patterns
- Provides the Flutter client with a task picker embedded in the terminal UI

16.2 Go: HelixTrack Bridge Service Handler

```

// File: helixterm.io/services/helixtrack-bridge/internal/
//       handler/bridge_handler.go
package handler

import (
    "net/http"

    "github.com/gin-gonic/gin"
    "go.uber.org/zap"

    "helixtrack.ru/core"

```

```

    "helixtrack.ru/core/tasks"
    "helixterm.io/services/helixtrack-bridge/internal/model"
    "helixterm.io/services/helixtrack-bridge/internal/
        repository"
)

// BridgeHandler handles HelixTrack integration operations.
type BridgeHandler struct {
    ht      core.Client
    repo    repository.BridgeRepository
    logger  *zap.Logger
}

// NewBridgeHandler creates a HelixTrack bridge handler.
func NewBridgeHandler(apiURL, apiKey string, repo
    repository.BridgeRepository, logger *zap.Logger)
    (*BridgeHandler, error) {
    client, err := core.NewClient(core.Config{
        APIURL: apiURL,
        APIKey: apiKey,
    })
    if err != nil {
        return nil, fmt.Errorf("helixtrack_bridge: creating
            client: %w", err)
    }
    return &BridgeHandler{ht: client, repo: repo, logger:
        logger}, nil
}

// AssociateSession associates an SSH terminal session with a
    HelixTrack task.
// POST /v1/helixtrack/sessions/:session_id/associate
func (h *BridgeHandler) AssociateSession(c *gin.Context) {
    sessionID := c.Param("session_id")
    var req model.AssociateRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error":
            err.Error()})
        return
    }

    ctx := c.Request.Context()

    // Verify the task exists in HelixTrack.
    task, err := h.ht.Tasks().Get(ctx, req.TaskID)
    if err != nil {
        h.logger.Error("helixtrack_bridge: task not found",
            zap.String("task_id", req.TaskID), zap.Error(err))
        c.JSON(http.StatusNotFound, gin.H{"error":
            "HelixTrack task not found"})
        return
    }
}

```

```

// Persist the association.
if err := h.repo.AssociateSession(ctx,
    repository.SessionAssociation{
        SessionID: sessionID,
        TaskID:    req.TaskID,
        UserID:    c.GetString("helix_user_id"),
        OrgID:     c.GetString("helix_org_id"),
    }); err != nil {
    h.logger.Error("helixtrack_bridge: persisting
        association", zap.Error(err))
    c.JSON(http.StatusInternalServerError,
        gin.H{"error": err.Error()})
    return
}

// Log work started in HelixTrack.
if err := h.ht.Tasks().LogActivity(ctx, req.TaskID,
    tasks.Activity{
        Type:    tasks.ActivityTypeWorkStarted,
        Message: fmt.Sprintf("SSH session %s started",
            sessionID),
        UserID:  c.GetString("helix_user_id"),
    }); err != nil {
    h.logger.Warn("helixtrack_bridge: logging activity
        failed", zap.Error(err))
}

c.JSON(http.StatusOK, gin.H{
    "session_id": sessionID,
    "task_id":    req.TaskID,
    "task_title": task.Title,
})
}

// GetUserTasks returns the authenticated user's open
// HelixTrack tasks.
// Used by the Flutter UI to populate the task picker.
// GET /v1/helixtrack/tasks
func (h *BridgeHandler) GetUserTasks(c *gin.Context) {
    userID := c.GetString("helix_user_id")
    orgID := c.GetString("helix_org_id")
    ctx := c.Request.Context()

    taskList, err := h.ht.Tasks().List(ctx, tasks.ListFilter{
        AssigneeID: userID,
        OrgID:      orgID,
        Status:     []tasks.Status{tasks.StatusOpen,
            tasks.StatusInProgress},
        PageSize:   50,
    })
    if err != nil {

```

```

        h.logger.Error("helixtrack_bridge: listing tasks",
            zap.Error(err))
        c.JSON(http.StatusInternalServerError,
            gin.H{"error": err.Error()})
        return
    }

    c.JSON(http.StatusOK, gin.H{"tasks": taskList.Items,
        "total": taskList.Total})
}

// UpdateTaskFromCommand analyses a terminal command and
// optionally updates
// the associated HelixTrack task status.
// POST /v1/helixtrack/sessions/:session_id/command
func (h *BridgeHandler) UpdateTaskFromCommand(c
    *gin.Context) {
    sessionID := c.Param("session_id")
    var req model.CommandEventRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error":
            err.Error()})
        return
    }

    ctx := c.Request.Context()

    assoc, err := h.repo.GetAssociation(ctx, sessionID)
    if err != nil {
        // No association – nothing to update.
        c.JSON(http.StatusOK, gin.H{"status":
            "no_association"})
        return
    }

    // Detect task-relevant patterns in the command.
    action := detectTaskAction(req.Command, req.ExitCode)
    if action == nil {
        c.JSON(http.StatusOK, gin.H{"status": "no_action"})
        return
    }

    if err := h.ht.Tasks().Transition(ctx, assoc.TaskID,
        *action); err != nil {
        h.logger.Warn("helixtrack_bridge: task transition
            failed", zap.Error(err))
    }

    c.JSON(http.StatusOK, gin.H{"status": "updated",
        "action": action.Type})
}

```

```
// detectTaskAction maps command patterns to HelixTrack task
// transitions.
func detectTaskAction(command string, exitCode int)
    *tasks.Transition {
    if exitCode != 0 {
        return nil
    }

    patterns := []struct {
        keyword string
        action  tasks.TransitionType
    }{
        {"git push", tasks.TransitionTypeComplete},
        {"make deploy", tasks.TransitionTypeComplete},
        {"kubectrl apply", tasks.TransitionTypeInProgress},
        {"make test", tasks.TransitionTypeInProgress},
    }

    for _, p := range patterns {
        if strings.Contains(command, p.keyword) {
            return &tasks.Transition{Type: p.action}
        }
    }
    return nil
}
```

16.3 Dart/Flutter: Task Picker in Terminal UI

```
// File: lib/src/helixtrack/task_picker.dart

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../api/helixtrack_bridge_client.dart';
import '../models/helix_task.dart';

/// TaskPickerWidget is embedded in the terminal toolbar.
/// It allows the user to associate the current SSH session
/// with a HelixTrack task.
class TaskPickerWidget extends StatefulWidget {
    final String sessionID;

    const TaskPickerWidget({super.key, required
        this.sessionID});

    @override
    State<TaskPickerWidget> createState() =>
        _TaskPickerWidgetState();
}
```

```

class _TaskPickerWidgetState extends State<TaskPickerWidget>
{
  HelixTask? _selectedTask;
  List<HelixTask> _tasks = [];
  bool _loading = false;
  String? _error;
  @override
  void initState() {
    super.initState();
    _loadTasks();
  }

  Future<void> _loadTasks() async {
    setState(() { _loading = true; _error = null; });
    try {
      final client = context.read<HelixTrackBridgeClient>();
      _tasks = await client.listActiveTasks();
    } catch (e) {
      _error = e.toString();
    } finally {
      setState(() { _loading = false; });
    }
  }

  Future<void> _associateTask(HelixTask task) async {
    final client = context.read<HelixTrackBridgeClient>();
    await client.associateSession(
      sessionID: widget.sessionID,
      taskID: task.id,
    );
    setState(() { _selectedTask = task; });
  }

  @override
  Widget build(BuildContext context) {
    if (_loading) return const SizedBox(width: 24, height: 24, child: CircularProgressIndicator(strokeWidth: 2));
    if (_error != null) return Tooltip(message: _error!, child: const Icon(Icons.error_outline, color: Colors.red));

    return PopupMenuButton<HelixTask>(
      tooltip: _selectedTask == null ? 'Associate HelixTrack task' : _selectedTask!.title,
      onSelected: _associateTask,
      itemBuilder: (_) => _tasks.map((t) =>
        PopupMenuItem<HelixTask>(
          value: t,
          child: Row(
            children: [

```

```

        _statusIcon(t.status),
        const SizedBox(width: 8),
        Expanded(child: Text(t.title, overflow:
TextOverflow.ellipsis)),
      ],
    ),
  )),.toList(),
  child: Chip(
    avatar: const Icon(Icons.task_alt, size: 16),
    label: Text(
      _selectedTask?.title ?? 'Link Task',
      style: const TextStyle(fontSize: 12),
    ),
  ),
);
}

Widget _statusIcon(String status) {
  switch (status) {
    case 'in_progress': return const
      Icon(Icons.play_circle_outline, color: Colors.blue,
        size: 16);
    case 'completed': return const
      Icon(Icons.check_circle_outline, color:
        Colors.green, size: 16);
    default: return const
      Icon(Icons.radio_button_unchecked, color:
        Colors.grey, size: 16);
  }
}
}

```

```

// File: lib/src/api/helixtrack_bridge_client.dart

import 'dart:convert';
import 'package:http/http.dart' as http;
import '../models/helix_task.dart';

class HelixTrackBridgeClient {
  final String baseUrl;
  final String accessToken;

  HelixTrackBridgeClient({required this.baseUrl, required
    this.accessToken});

  Future<List<HelixTask>> listActiveTasks() async {
    final uri = Uri.parse('$baseUrl/helixtrack/tasks?
      status=active');
    final resp = await http.get(uri, headers: _headers());
  }
}

```

```

    if (resp.statusCode != 200) throw
      Exception('listActiveTasks: ${resp.statusCode} $
        {resp.body}');
    final data = jsonDecode(resp.body) as Map<String,
      dynamic>;
    return (data['tasks'] as List).map((j) =>
      HelixTask.fromJson(j as Map<String,
        dynamic>)).toList();
  }

Future<void> associateSession({required String sessionID,
  required String taskID}) async {
  final uri = Uri.parse('$baseUrl/helixtrack/sessions/
    $sessionID/task');
  final resp = await http.put(
    uri,
    headers: _headers(),
    body: jsonEncode({'task_id': taskID}),
  );
  if (resp.statusCode != 200) throw
    Exception('associateSession: ${resp.statusCode} $
      {resp.body}');
}

Map<String, String> _headers() => {
  'Authorization': 'Bearer $accessToken',
  'Content-Type': 'application/json',
};
}

```

```

// File: lib/src/models/helix_task.dart

class HelixTask {
  final String id;
  final String title;
  final String status;
  final String? projectID;

  const HelixTask({
    required this.id,
    required this.title,
    required this.status,
    this.projectID,
  });

  factory HelixTask.fromJson(Map<String, dynamic> j) =>
    HelixTask(
      id: j['id'] as String,
      title: j['title'] as String,
      status: j['status'] as String,

```

```
    projectID: j['project_id'] as String?,  
  );  
}
```

17. HelixConstitution — AGENTS.MD, CLAUDE.MD, Constitution.md

Module: `github.com/HelixDevelopment/helix-constitution` **Purpose:** Governance layer for all Helix-family codebases. Defines coding standards, AI-agent behaviour rules, CI compliance checks, dependency hygiene rules, package naming conventions, test coverage thresholds, and inter-service contract rules. HelixTerminator must satisfy every rule in the constitution for CI to pass.

17.1 `helix-deps.yaml` — HelixTerminator Dependency Manifest

This file lives at the repository root. The `helix-constitution` CI action reads it and verifies that all referenced submodule versions exist, licenses are approved, and dependency graph is acyclic.

```
# helix-deps.yaml  
# HelixTerminator — Helix Dependency Manifest  
# Governed by §11.4.31 of HelixConstitution v2.  
  
schema_version: "2.1"  
project:  
  name: HelixTerminator  
  module: helixterm.io  
  team: core-platform  
  go_version: "1.25"  
  constitution_version: "2.0"  
  license: AGPL-3.0  
  
submodules:  
  - id: digital.vasic.security  
    source: github.com/vasic-digital/security  
    version: v1.4.2  
    import_path: digital.vasic.security  
    license: MIT  
    required_by:  
      - helixterm.io/services/vault  
      - helixterm.io/services/ssh-key  
      - helixterm.io/services/auth  
    governs:  
      - encryption_at_rest  
      - key_rotation  
  
  - id: digital.vasic.auth
```

```
source: github.com/vasic-digital/auth
version: v2.3.1
import_path: digital.vasic.auth
license: MIT
required_by:
  - helixterm.io/services/auth
  - helixterm.io/services/gateway
governs:
  - jwt_issuance
  - token_rotation
  - oidc_federation
  - scim_provisioning

- id: digital.vasic.cache
source: github.com/vasic-digital/cache
version: v1.2.7
import_path: digital.vasic.cache
license: MIT
required_by:
  - helixterm.io/services/gateway
  - helixterm.io/services/vault
  - helixterm.io/services/session
  - helixterm.io/services/host-manager
governs:
  - redis_l1_l2
  - ttl_policy

- id: digital.vasic.database
source: github.com/vasic-digital/database
version: v1.8.0
import_path: digital.vasic.database
license: MIT
required_by: ["*"] # all 25 services
governs:
  - migration_runner
  - read_replica_routing
  - soft_delete
  - pagination

- id: digital.vasic.messaging
source: github.com/vasic-digital/messaging
version: v1.5.3
import_path: digital.vasic.messaging
license: MIT
required_by:
  - helixterm.io/services/audit
  - helixterm.io/services/analytics
  - helixterm.io/services/ssh-proxy
  - helixterm.io/services/sftp-proxy
governs:
  - kafka_producers
```

```
- rabbitmq_consumers
- dlq_handling

- id: digital.vasic.middleware
  source: github.com/vasic-digital/middleware
  version: v1.1.4
  import_path: digital.vasic.middleware
  license: MIT
  required_by: ["*"]
  governs:
    - request_id
    - logging_middleware
    - panic_recovery
    - timeout_middleware

- id: digital.vasic.observability
  source: github.com/vasic-digital/observability
  version: v1.3.0
  import_path: digital.vasic.observability
  license: MIT
  required_by: ["*"]
  governs:
    - prometheus_exposition
    - otel_traces
    - health_endpoints

- id: digital.vasic.ratelimiter
  source: github.com/vasic-digital/ratelimiter
  version: v1.0.9
  import_path: digital.vasic.ratelimiter
  license: MIT
  required_by:
    - helixterm.io/services/gateway
    - helixterm.io/services/auth
    - helixterm.io/services/ssh-proxy
  governs:
    - per_user_limits
    - per_ip_limits
    - brute_force_protection

- id: digital.vasic.recovery
  source: github.com/vasic-digital/recovery
  version: v1.2.1
  import_path: digital.vasic.recovery
  license: MIT
  required_by: ["*"]
  governs:
    - circuit_breakers
    - fallback_handlers
    - bulkhead_pattern
```

```
- id: digital.vasic.concurrency
  source: github.com/vasic-digital/concurrency
  version: v1.1.0
  import_path: digital.vasic.concurrency
  license: MIT
  required_by:
    - helixterm.io/services/ssh-proxy
    - helixterm.io/services/vault
    - helixterm.io/services/terminal
  governs:
    - worker_pools
    - semaphore_pattern
    - errgroup_usage

- id: digital.vasic.containers
  source: github.com/vasic-digital/containers
  version: v1.0.5
  import_path: digital.vasic.containers
  license: MIT
  required_by:
    - helixterm.io/services/container-bridge
    - helixterm.io/services/ssh-proxy
  governs:
    - runtime_abstraction
    - container_exec
    - image_management

- id: digital.vasic.docs_chain
  source: github.com/vasic-digital/docs-chain
  version: v1.0.3
  import_path: digital.vasic.docs_chain
  license: MIT
  required_by: ["ci"]
  governs:
    - spec_doc_consistency
    - transform_pipeline

- id: digital.vasic.challenges
  source: github.com/vasic-digital/challenges
  version: v0.9.2
  import_path: digital.vasic.challenges
  license: MIT
  required_by:
    - helixterm.io/services/challenge
    - helixterm.io/services/ai
    - helixterm.io/services/user
  governs:
    - challenge_lifecycle
    - challenge_scoring

- id: helixqa
```

```
source: github.com/HelixDevelopment/helixqa
version: v1.7.0
import_path: helixqa
license: Proprietary-Helix
required_by: ["ci"]
governs:
  - test_generation
  - flaky_detection
  - mutation_testing
  - coverage_reporting

- id: helixtrack_core
  source: helixtrack.ru/core
  version: v2.1.4
  import_path: helixtrack.ru/core
  license: Proprietary-Helix
  required_by:
    - helixterm.io/services/helixtrack-bridge
  governs:
    - task_lifecycle
    - session_association

- id: helix_constitution
  source: github.com/HelixDevelopment/helix-constitution
  version: v2.0.0
  import_path: helix-constitution
  license: Proprietary-Helix
  required_by: ["ci"]
  governs:
    - coding_standards
    - agent_rules
    - coverage_thresholds
    - naming_conventions

policies:
  test_coverage:
    minimum_total: 80
    minimum_per_package: 70
    critical_packages:
      - helixterm.io/services/vault: 90
      - helixterm.io/services/auth: 90
      - helixterm.io/services/ssh-proxy: 85
  naming:
    packages: snake_case
    exported_types: PascalCase
    unexported_vars: camelCase
    constants: SCREAMING_SNAKE_CASE_or_PascalCase
  imports:
    no_dot_imports: true
    no_blank_imports_except:
      - database/sql drivers
```

```

    - embed
  no_init_functions_except:
    - main packages
    - test helpers
errors:
  must_wrap_with_context: true
  no_panic_in_library_code: true
  sentinel_errors_in: errors.go
docs:
  every_exported_symbol_must_have_godoc: true
  package_doc_required: true
linters:
  golangci_lint_version: "1.60"
  required_linters:
    - govet
    - errcheck
    - staticcheck
    - gosec
    - godot
    - misspell
    - revive
    - gocyclo
    - dupl
security:
  no_hardcoded_secrets: true
  no_unencrypted_sensitive_fields: true
  all_credentials_via_vault: true

```

17.2 AGENTS.MD for HelixTerminator

```

# AGENTS.MD – HelixTerminator
# Governed by HelixConstitution §3 (AI Agent Rules), v2.0
# Last updated: 2026-06-28

## Identity

You are an AI coding agent operating on the HelixTerminator
codebase.
HelixTerminator module path: `helixterm.io`
Go 1.25, Gin Gonic, Kafka, RabbitMQ, PostgreSQL, Redis,
Kubernetes.
25 microservices under `helixterm.io/services/<name>`.
Flutter/Dart client: package `io.helixterm.client`.

## Mandatory Rules

### §A1 – Code Style
- All Go code MUST pass `golangci-lint run ./...` with zero
  warnings.

```

- All exported functions, types, and constants MUST have godoc comments.
- All packages MUST have a package-level godoc comment.
- Error messages MUST be lowercase with no trailing punctuation.
- Errors MUST be wrapped with ``fmt.Errorf("context: %w", err)``.

§A2 – No Hallucinated Imports

- NEVER import packages that do not exist in ``go.work`` or ``go.mod``.
- When adding a new dependency, update ``go.work`` AND ``helix-deps.yaml``.
- Do NOT import internal packages from sibling services directly; use gRPC contracts.

§A3 – Security

- Never generate hardcoded secrets, tokens, passwords, or private keys.
- All encryption operations MUST use ``digital.vasic.security``.
- All authentication MUST go through ``digital.vasic.auth``.
- SQL queries MUST use parameterised statements – no string concatenation.
- User input MUST be validated before use in DB queries, shell commands, or file paths.

§A4 – Testing

- Every new exported function MUST have at least one unit test.
- Table-driven tests are preferred over sequential test functions.
- Mock external dependencies with interfaces, not monkey-patching.
- Tests MUST NOT rely on live network, filesystem, or database state unless tagged ``//go:build integration``.
- Minimum coverage: 80% overall; 90% for ``vault`` and ``auth`` services.

§A5 – gRPC Contracts

- Service-to-service calls MUST use the Proto definitions in ``helixterm.io/proto``.
- Never call another service's HTTP handler directly from Go code.
- gRPC metadata MUST propagate the ``x-request-id``, ``x-trace-id``, and JWT token.

§A6 – Database

- Every DB schema change MUST include an up AND down migration in ``migrations/``.
- Migrations MUST be idempotent (use ``IF NOT EXISTS``, ``IF EXISTS`` guards).

- All database access MUST go through ``digital.vasic.database`` connection helpers.
- Raw ``database/sql`` usage is forbidden; use the abstraction layer.

§A7 – Messaging

- Kafka and RabbitMQ usage MUST go through ``digital.vasic.messaging``.
- Every new Kafka topic MUST be registered in ``messaging/topics.go``.
- Messages MUST include ``event_id``, ``timestamp``, ``source_service``, and ``schema_version``.

§A8 – Observability

- Every new service endpoint MUST record a Prometheus counter and histogram.
- Every cross-service call MUST create an OpenTelemetry span.
- All log statements MUST use structured fields (zap or slog), no ``fmt.Printf``.

§A9 – Kubernetes

- New services MUST include a Helm chart under ``deploy/charts/<service>``.
- All Kubernetes manifests MUST define resource requests and limits.
- Health checks (``/healthz/live``, ``/healthz/ready``) MUST be implemented.
- Secrets MUST be sourced from Kubernetes Secrets or Vault, never ConfigMaps.

§A10 – Git & PRs

- Branch names: ``feat/<ticket-id>-<slug>``, ``fix/<ticket-id>-<slug>``, ``chore/<slug>``.
- Commit messages: Conventional Commits format (``feat:``, ``fix:``, ``chore:``, ``docs:``, ``test:``).
- PRs MUST reference a HelixTrack task ID in the description.
- No force-pushes to ``main`` or ``develop``.

§A11 – Forbidden Actions

- Do NOT delete migration files.
- Do NOT modify ``go.sum`` manually.
- Do NOT bypass CI checks with ``[skip ci]`` without team-lead approval.
- Do NOT add ``//nolint`` directives without an explanatory comment.
- Do NOT use ``context.Background()`` in service handlers – propagate the request context.

§A12 – Submodule Updates

- Submodule version bumps MUST be reflected in both ``go.work`` and ``helix-deps.yaml``.

- Run ``make submodules-verify`` after any submodule version change.
- Breaking changes in submodule APIs MUST be tracked in ``CHANGELOG.md``.

Read These Files

- ``Constitution.md`` – full governance rules
- ``helix-deps.yaml`` – dependency manifest
- ``docs/10_submodule_integration.md`` – this spec
- ``docs/01_architecture.md`` – system architecture
- ``docs/02_data_model.md`` – database schema

17.3 CLAUDE.MD for HelixTerminator

```
# CLAUDE.MD – HelixTerminator
# Claude-specific rules for operating on the HelixTerminator
# codebase.
# Governed by HelixConstitution §3.2 (Claude Rules), v2.0
# Last updated: 2026-06-28
```

Persona

You are a senior Go/Flutter engineer contributing to HelixTerminator.
Be precise, concise, and opinionated. Prefer correctness over brevity.

Toolchain Assumptions

- Go toolchain: ``go1.25``
- Linter: ``golangci-lint v1.60``
- Proto compiler: ``protoc v3.21`` with ``protoc-gen-go v1.34``
- Flutter SDK: ``3.22``
- Dart: ``3.4``
- Docker: ``27.x``
- Kubernetes: ``1.31``

Code Generation Rules

Go

1. Use ``context.Context`` as the first parameter of every function that does I/O.
2. Return concrete types from constructors, interfaces from factory functions.
3. Define package-level ``var ErrXxx = errors.New("...")`` sentinels in ``errors.go``.
4. Use ``//noLint:gosec // reason`` if suppressing a linter – always include reason.
5. Prefer ``errors.Is`` / ``errors.As`` over string matching.

6. Use ``slog`` (stdlib) for logging in new code unless the service already uses ``zap``.
7. Mutex fields named ``mu`` for unexported, ``Mu`` for embedded exported structs.
8. HTTP handlers: always set ``c.Header("Content-Type", "application/json")`` explicitly.
9. gRPC interceptors must log at DEBUG for success, WARN for user errors, ERROR for server faults.
10. Config structs tagged with ``mapstructure``, ``yaml``, and ``validate`` tags.

Dart/Flutter

1. Prefer ``riverpod`` over ``provider`` for new state management.
2. Use ``freezed`` for immutable data classes.
3. All API calls go through ``HelixApiClient`` – never call ``http.get()`` directly.
4. Localise all user-facing strings via ``flutter_gen``-produced ``AppLocalizations``.
5. Widget tests MUST use ``pumpAndSettle()`` for async operations.
6. No ``BuildContext`` usage after ``await`` unless wrapped in ``if (!mounted) return``.

Diff Rules

- When editing existing Go files: preserve the existing ``import`` grouping order:
 1. stdlib
 2. third-party
 3. ``helixterm.io/*`` internal
 4. ``digital.vasic.*`` submodules
- Do not reformat lines you are not changing.
- Do not add trailing commas to single-argument function calls.

Response Style

- Show full file content only when creating new files.
- For edits, show only the changed function/block with surrounding context (± 5 lines).
- Always include the file path as a comment on the first line of every code block.
- When generating Proto files, always include ``option go_package = "helixterm.io/proto/<pkg>;pb"``.

Off-Limits

- Do NOT suggest ``github.com/pkg/errors`` – use stdlib ``fmt.Errorf("%w")``.
- Do NOT suggest ``github.com/jmoiron/sqlx`` – use ``digital.vasic.database``.

- Do NOT suggest global `http.DefaultClient` – use the injected client.
- Do NOT suggest `os.Exit()` outside of `main()`.
- Do NOT add `init()` functions in library packages.

When Stuck

If a function or type is not visible in the current context, check:

1. `helixterm.io/proto/<service>/` – gRPC definitions
2. `helixterm.io/pkg/<name>/` – shared internal packages
3. `digital.vasic.<module>` – submodule public API
4. `docs/10_submodule_integration.md` – integration reference

17.4 CI Constitution Compliance Check

```
# .github/workflows/constitution-compliance.yml
name: HelixConstitution Compliance

on:
  push:
    branches: [main, develop, "feat/*", "fix/*"]
  pull_request:
    branches: [main, develop]

jobs:
  constitution-check:
    name: Constitution Compliance
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          submodules: recursive
          fetch-depth: 0

      - name: Set up Go 1.25
        uses: actions/setup-go@v5
        with:
          go-version: "1.25"
          cache: true

      - name: Restore helix-constitution binary
        uses: actions/cache@v4
        with:
          path: ~/.helix/bin
          key: helix-constitution-v2.0.0

      - name: Install helix-constitution CLI
        run: |
```

```

    if [ ! -f ~/.helix/bin/helix-constitution ]; then
        mkdir -p ~/.helix/bin
        curl -sSfL https://releases.helixdevelopment.io/
constitution/v2.0.0/helix-constitution-linux-amd64 \
        -o ~/.helix/bin/helix-constitution
        chmod +x ~/.helix/bin/helix-constitution
    fi
    echo "$HOME/.helix/bin" >> $GITHUB_PATH

- name: Validate helix-deps.yaml schema
  run: helix-constitution deps validate --file helix-
deps.yaml

- name: Check dependency graph (no cycles)
  run: helix-constitution deps graph --check-cycles --
file helix-deps.yaml

- name: Verify submodule versions exist
  run: helix-constitution deps verify --file helix-
deps.yaml

- name: Check license compliance
  run: helix-constitution license check --file helix-
deps.yaml --allowed MIT,Apache-2.0,BSD-3-
Clause,Proprietary-Helix,AGPL-3.0

- name: Run golangci-lint
  uses: golangci/golangci-lint-action@v6
  with:
    version: v1.60
    args: --timeout 10m ./...

- name: Check godoc coverage
  run: |
    go install golang.org/x/tools/cmd/godoc@latest
    helix-constitution godoc check ./... --min-coverage
100 --exported-only

- name: Verify import grouping
  run: helix-constitution imports check ./...

- name: Check forbidden patterns
  run: |
    helix-constitution patterns check \
      --no-context-background-in-handlers \
      --no-hardcoded-secrets \
      --no-fmt-printf-in-services \
      --no-direct-sql-string-concat \
      ./...

- name: Check package naming conventions
  run: helix-constitution naming check ./...

```

```

- name: Verify AGENTS.MD and CLAUDE.MD present and up-
  to-date
  run: |
    helix-constitution agents check \
      --agents-file AGENTS.MD \
      --claude-file CLAUDE.MD \
      --constitution-version 2.0

- name: Check migration files not deleted
  run: |
    git diff --name-only origin/${
    {{ github.base_ref }}...HEAD \
      | grep "^migrations/" \
      | xargs -I{{ helix-constitution migrations check-
        deleted }}

- name: Report to HelixTrack
  if: always()
  env:
    HELIXTRACK_TOKEN: $
    {{ secrets.HELIXTRACK_CI_TOKEN }}
  run: |
    helix-constitution report \
      --to helixtrack \
      --token "$HELIXTRACK_TOKEN" \
      --project HelixTerminator \
      --job constitution-check \
      --status ${{ job.status }}

```

17.5 Constitution Rules Applicable to HelixTerminator

The following rules from HelixConstitution v2.0 are binding on all HelixTerminator code:

§	Rule	Enforcement
§2.1	All packages must have godoc	helix-constitution godoc check
§2.2	All exported symbols must have godoc	golangci-lint godot
§2.3	No <code>init()</code> in library packages	helix-constitution patterns
§3.1	AGENTS.MD required at root	CI step
§3.2	CLAUDE.MD required at root	CI step

§	Rule	Enforcement
§4.1	helix-deps.yaml required	CI step
§4.2	All submodules tracked in helix-deps.yaml	CI step
§5.1	SQL must use parameterised queries	helix-constitution patterns
§5.2	No secrets in source code	helix-constitution patterns
§6.1	Migrations must have up and down scripts	helix-constitution migrations
§6.2	Migrations must be idempotent	Code review + helix-constitution
§7.1	Minimum 80% test coverage overall	helixqa coverage gate
§7.2	Minimum 90% coverage on vault and auth	helixqa per-package gate
§8.1	All errors must be wrapped with context	errcheck + helix-constitution
§9.1	All services must expose Prometheus metrics	CI health check
§9.2	All services must expose OTEL traces	CI health check
§10.1	Commit messages must follow Conventional Commits	branch protection rules
§11.4.31	helix-deps.yaml schema v2.1 required	helix-constitution deps validate

Appendix A: go.work — Workspace File

The Go workspace file enables local development across all submodules and services simultaneously, providing module resolution without requiring tagged releases for every change cycle.

```
// go.work
// HelixTerminator Go Workspace
// Go 1.25
// Run: go work sync

go 1.25

use (
    // Root module
    .

    // Microservices
    ./services/gateway
    ./services/auth
    ./services/vault
    ./services/ssh-proxy
    ./services/sftp-proxy
    ./services/terminal
    ./services/host-manager
    ./services/user
    ./services/team
    ./services/workspace-svc
    ./services/audit
    ./services/analytics
    ./services/notification
    ./services/billing
    ./services/ai
    ./services/challenge
    ./services/container-bridge
    ./services/helixtrack-bridge
    ./services/scheduler
    ./services/rbac
    ./services/secret-manager
    ./services/session
    ./services/snippet
    ./services/webhook
    ./services/config-svc

    // Proto definitions (generated code)
    ./proto

    // Shared internal packages
    ./pkg/testutil
    ./pkg/config
    ./pkg/errors

    // Submodules (local checkouts via git submodule)
    ./submodules/vasic-digital/security
    ./submodules/vasic-digital/auth
    ./submodules/vasic-digital/cache
```

```
./submodules/vasic-digital/database
./submodules/vasic-digital/messaging
./submodules/vasic-digital/middleware
./submodules/vasic-digital/observability
./submodules/vasic-digital/ratelimiter
./submodules/vasic-digital/recovery
./submodules/vasic-digital/concurrency
./submodules/vasic-digital/containers
./submodules/vasic-digital/docs-chain
./submodules/vasic-digital/challenges
./submodules/helixdevelopment/helixqa
./submodules/helixtrack/core
./submodules/helixdevelopment/helix-constitution
)

// Replace directives map module paths to local submodule
// paths.
// These are active only in the workspace and do not affect
// go.sum in production.
replace (
    digital.vasic.security      => ./submodules/vasic-digital/
        security
    digital.vasic.auth          => ./submodules/vasic-digital/
        auth
    digital.vasic.cache         => ./submodules/vasic-digital/
        cache
    digital.vasic.database      => ./submodules/vasic-digital/
        database
    digital.vasic.messaging     => ./submodules/vasic-digital/
        messaging
    digital.vasic.middleware    => ./submodules/vasic-digital/
        middleware
    digital.vasic.observability => ./submodules/
        vasic-digital/observability
    digital.vasic.ratelimiter   => ./submodules/vasic-digital/
        ratelimiter
    digital.vasic.recovery      => ./submodules/vasic-digital/
        recovery
    digital.vasic.concurrency   => ./submodules/vasic-digital/
        concurrency
    digital.vasic.containers    => ./submodules/vasic-digital/
        containers
    digital.vasic.docs_chain    => ./submodules/vasic-digital/
        docs-chain
    digital.vasic.challenges    => ./submodules/vasic-digital/
        challenges
    helixqa                    => ./submodules/
        helixdevelopment/helixqa
    helixtrack.ru/core         => ./submodules/helixtrack/core
)
```

```
    helix-constitution      => ./submodules/  
        helixdevelopment/helix-constitution  
)
```

Appendix B: Makefile — Submodule Targets

```
# Makefile — Submodule management targets  
# Part of the HelixTerminator root Makefile.  
  
SHELL := /bin/bash  
.PHONY: submodules-init submodules-update submodules-verify  
        submodules-status submodules-diff  
  
SUBMODULE_PATHS := \  
    submodules/vasic-digital/security \  
    submodules/vasic-digital/auth \  
    submodules/vasic-digital/cache \  
    submodules/vasic-digital/database \  
    submodules/vasic-digital/messaging \  
    submodules/vasic-digital/middleware \  
    submodules/vasic-digital/observability \  
    submodules/vasic-digital/ratelimiter \  
    submodules/vasic-digital/recovery \  
    submodules/vasic-digital/concurrency \  
    submodules/vasic-digital/containers \  
    submodules/vasic-digital/docs-chain \  
    submodules/vasic-digital/challenges \  
    submodules/helixdevelopment/helixqa \  
    submodules/helixtrack/core \  
    submodules/helixdevelopment/helix-constitution  
  
## submodules-init: Clone and initialise all submodules.  
## Run this once after cloning the repository.  
submodules-init:  
    @echo "==> Initialising git submodules..."  
    git submodule update --init --recursive --jobs 8  
    @echo "==> Syncing Go workspace..."  
    go work sync  
    @echo "==> Verifying submodule integrity..."  
    $(MAKE) submodules-verify  
    @echo "==> Done. All submodules initialised."  
  
## submodules-update: Pull latest changes for all submodules.  
## Use with care: this updates to HEAD of each tracked  
    branch.  
submodules-update:
```

```

@echo "==> Updating all submodules to tracked branch
HEAD..."
git submodule update --remote --merge --jobs 8
@echo "==> Syncing Go workspace after update..."
go work sync
@echo "==> Tidying all module go.mod files..."
for svc in services/*/; do \
    if [ -f "$$svc/go.mod" ]; then \
        echo "    tidy: $$svc"; \
        (cd "$$svc" && go mod tidy); \
    fi; \
done
@echo "==> Running verification..."
$(MAKE) submodules-verify
@echo "==> Submodule update complete."

## submodules-verify: Verify submodule state matches helix-
deps.yaml.
## Runs helix-constitution deps verify and go work sync.
submodules-verify:
@echo "==> Verifying helix-deps.yaml against checked-out
submodules..."
@if command -v helix-constitution &>/dev/null; then \
    helix-constitution deps verify --file helix-
    deps.yaml; \
else \
    echo "    WARNING: helix-constitution not installed,
    skipping manifest check."; \
    echo "    Install from: https://
    releases.helixdevelopment.io/constitution/"; \
fi
@echo "==> Verifying go.work replace directives..."
for path in $(SUBMODULE_PATHS); do \
    if [ ! -f "$$path/go.mod" ]; then \
        echo "    ERROR: Missing go.mod in $$path. Run:
        make submodules-init"; \
        exit 1; \
    fi; \
done
@echo "==> Checking for dirty submodule state..."
@DIRTY=$(git submodule foreach --quiet 'git diff --exit-
code HEAD > /dev/null 2>&1 || echo $$name'); \
if [ -n "$$DIRTY" ]; then \
    echo "    WARNING: Dirty submodules detected: $
    $DIRTY"; \
fi
@echo "==> Verifying Go workspace builds..."
go build ./...
@echo "==> All checks passed."

## submodules-status: Show current commit of each submodule.
submodules-status:

```

```

@echo "==> Submodule status:"
@git submodule foreach 'echo "  $$name @ $(git log -1 --
    format="%h %s (%ai)\")"'

## submodules-diff: Show changes in submodules since last
    commit.
submodules-diff:
    @echo "==> Submodule diff vs HEAD:"
    git diff --submodule=diff HEAD

## submodules-pin: Pin all submodules to their current commit
    (for release tagging).
submodules-pin:
    @echo "==> Pinning submodules at current commit in helix-
        deps.yaml..."
    @for path in $(SUBMODULE_PATHS); do \
        HASH=$(git -C "$$path" rev-parse HEAD); \
        echo "  $$path => $$HASH"; \
    done
    @echo
        "==> Update helix-deps.yaml manually with the above
        hashes for a release build."

## submodules-clean: Remove all submodule directories
    (requires re-init).
submodules-clean:
    @echo "==> WARNING: This will remove all submodule
        working trees."
    @read -p "Are you sure? [y/N] " CONFIRM; \
    if [ "$$CONFIRM" = "y" ]; then \
        for path in $(SUBMODULE_PATHS); do \
            git submodule deinit -f "$$path"; \
            rm -rf ".git/modules/$$path"; \
            rm -rf "$$path"; \
        done; \
        echo "Done. Run: make submodules-init"; \
    else \
        echo "Aborted."; \
    fi

```

Appendix C: GitHub Actions — Submodule Compliance Workflow

```

# .github/workflows/submodule-compliance.yml
name: Submodule Compliance

on:

```

```

push:
  branches: [main, develop, "feat/*", "fix/*", "chore/*"]
pull_request:
  branches: [main, develop]
schedule:
  # Daily check at 03:00 UTC to catch upstream submodule
  drift
  - cron: "0 3 * * *"
workflow_dispatch:
  inputs:
    force_update:
      description: "Force submodule update to latest"
      required: false
      default: "false"
      type: boolean

env:
  GO_VERSION: "1.25"
  HELIX_CONSTITUTION_VERSION: "2.0.0"

jobs:
  # — Job 1: Validate submodule versions and manifest
  validate-manifest:
    name: Validate helix-deps.yaml
    runs-on: ubuntu-latest
    steps:
      - name: Checkout (with submodules)
        uses: actions/checkout@v4
        with:
          submodules: recursive
          fetch-depth: 0
          token: ${ secrets.HELIX_SUBMODULE_TOKEN }

      - name: Install helix-constitution CLI
        run: |
          curl -sSfL \
            "https://releases.helixdevelopment.io/
            constitution/v${HELIX_CONSTITUTION_VERSION}/helix-
            constitution-linux-amd64" \
            -o /usr/local/bin/helix-constitution
          chmod +x /usr/local/bin/helix-constitution
          helix-constitution version

      - name: Validate manifest schema
        run: helix-constitution deps validate --file helix-
        deps.yaml --strict

      - name: Check for dependency cycles
        run: helix-constitution deps graph --check-cycles --
        file helix-deps.yaml

```

```

- name: Verify all submodule versions exist upstream
  run: helix-constitution deps verify --file helix-
      deps.yaml --check-upstream

- name: License compliance check
  run: |
    helix-constitution license check \
      --file helix-deps.yaml \
      --allowed "MIT,Apache-2.0,BSD-3-
      Clause,Proprietary-Helix,AGPL-3.0"

# — Job 2: Build verification with submodules


---


build-verify:
  name: Build with Submodules
  runs-on: ubuntu-latest
  needs: validate-manifest
  steps:
    - name: Checkout (with submodules)
      uses: actions/checkout@v4
      with:
        submodules: recursive
        token: ${ secrets.HELIX_SUBMODULE_TOKEN }

    - name: Set up Go ${ env.GO_VERSION }
      uses: actions/setup-go@v5
      with:
        go-version: ${ env.GO_VERSION }
        cache: true

    - name: Sync Go workspace
      run: go work sync

    - name: Verify go.work replace directives are
      consistent
      run: |
        # Each submodule path must exist and have a go.mod
        MISSING=0
        while IFS= read -r line; do
          path=$(echo "$line" | grep -oP '(?<==> ).*')
          if [ -n "$path" ] && [ ! -f "$path/go.mod" ];
        then
          echo "ERROR: Missing go.mod at $path"
          MISSING=$((MISSING+1))
        fi
        done < <(grep "=>" go.work)
        if [ $MISSING -gt 0 ]; then exit 1; fi

    - name: Build all services
      run: |
        go build ./...
        for svc in services/*/; do

```

```

        if [ -f "$svc/go.mod" ]; then
            echo "Building $svc..."
            (cd "$svc" && go build ./...)
        fi
    done

- name: Run unit tests
  run: |
    go test ./... -count=1 -race -timeout 300s \
      -coverprofile=coverage.out \
      -covermode=atomic

- name: Check coverage thresholds
  run: |
    TOTAL=$(go tool cover -func=coverage.out | tail -1
    | awk '{print $NF}' | tr -d '%')
    echo "Total coverage: ${TOTAL}%"
    if (( $(echo "$TOTAL < 80" | bc -l) )); then
        echo "ERROR: Coverage ${TOTAL}% is below 80%
        threshold"
        exit 1
    fi

```

— Job 3: Submodule drift detection

```

drift-detection:
  name: Submodule Drift Detection
  runs-on: ubuntu-latest
  if: github.event_name == 'schedule' ||
    github.event.inputs.force_update == 'true'
  steps:
    - name: Checkout
      uses: actions/checkout@v4
      with:
        submodules: recursive
        token: ${ secrets.HELIX_SUBMODULE_TOKEN }}

    - name: Check for upstream changes in submodules
      id: drift
      run: |
        DRIFT_FOUND=0
        DRIFT_REPORT=""
        git submodule foreach --quiet '
          CURRENT=$(git rev-parse HEAD)
          git fetch origin --quiet 2>/dev/null
          LATEST=$(git rev-parse FETCH_HEAD 2>/dev/null ||
        echo "")
          if [ -n "$LATEST" ] && [ "$CURRENT" !=
        "$LATEST" ]; then
            echo "DRIFT: $name ($CURRENT -> $LATEST)"
            DRIFT_FOUND=1
          fi

```

```

    echo "drift_found=$DRIFT_FOUND" >> $GITHUB_OUTPUT

- name: Create drift issue
  if: steps.drift.outputs.drift_found == '1'
  uses: actions/github-script@v7
  with:
    script: |
      github.rest.issues.create({
        owner: context.repo.owner,
        repo: context.repo.repo,
        title: 'Submodule drift detected - ' + new
Date().toISOString().slice(0,10),
        body: 'One or more submodules have upstream
changes not yet merged into HelixTerminator.\n\nRun
`make submodules-update` to update.\n\nGenerated by
submodule-compliance workflow.',
        labels: ['submodules', 'maintenance',
'automated'],
      });

# — Job 4: API compatibility check


---


api-compat:
  name: Submodule API Compatibility
  runs-on: ubuntu-latest
  needs: build-verify
  steps:
    - name: Checkout
      uses: actions/checkout@v4
      with:
        submodules: recursive
        token: ${ secrets.HELIX_SUBMODULE_TOKEN }}

    - name: Set up Go
      uses: actions/setup-go@v5
      with:
        go-version: ${ env.GO_VERSION }}
        cache: true

    - name: Install apidiff
      run: go install golang.org/x/exp/cmd/apidiff@latest

    - name: Check for breaking API changes in submodules
      run: |
        # For each submodule, compare current API against
        pinned version in helix-deps.yaml
        helix-constitution deps api-compat \
        --file helix-deps.yaml \
        --tool apidiff \
        --fail-on-breaking

```

— Job 5: Security scan

security-scan:

```
name: Security Scan
runs-on: ubuntu-latest
needs: build-verify
steps:
  - name: Checkout
    uses: actions/checkout@v4
    with:
      submodules: recursive
      token: ${ secrets.HELIX_SUBMODULE_TOKEN }

  - name: Set up Go
    uses: actions/setup-go@v5
    with:
      go-version: ${ env.GO_VERSION }

  - name: Run govulncheck on all modules
    run: |
      go install golang.org/x/vuln/cmd/govulncheck@latest
      govulncheck ./...
      for svc in services/*/; do
        if [ -f "$svc/go.mod" ]; then
          echo "Scanning $svc..."
          (cd "$svc" && govulncheck ./...) || true
        fi
      done

  - name: Run gosec
    uses: securego/gosec@master
    with:
      args: "-exclude-generated -fmt sarif -out gosec.sarif ./..."

  - name: Upload SARIF to GitHub Security
    uses: github/codeql-action/upload-sarif@v3
    with:
      sarif_file: gosec.sarif
```

— Job 6: Generate compliance report

compliance-report:

```
name: Compliance Report
runs-on: ubuntu-latest
needs: [validate-manifest, build-verify, api-compat, security-scan]
if: always()
steps:
  - name: Checkout
    uses: actions/checkout@v4
```

```

with:
  submodules: recursive
  token: ${ secrets.HELIX_SUBMODULE_TOKEN }}

- name: Install helix-constitution CLI
  run: |
    curl -sSfL \
      "https://releases.helixdevelopment.io/
      constitution/v${HELIX_CONSTITUTION_VERSION}/helix-
      constitution-linux-amd64" \
      -o /usr/local/bin/helix-constitution
    chmod +x /usr/local/bin/helix-constitution

- name: Generate HTML compliance report
  run: |
    helix-constitution report generate \
      --project HelixTerminator \
      --deps helix-deps.yaml \
      --format html \
      --output compliance-report.html \
      --job-status validate-manifest=$
      {{ needs.validate-manifest.result }} \
      --job-status build-verify=${{ needs.build-
      verify.result }} \
      --job-status api-compat=${{ needs.api-
      compat.result }} \
      --job-status security-scan=${{ needs.security-
      scan.result }}

- name: Upload compliance report artifact
  uses: actions/upload-artifact@v4
  with:
    name: compliance-report-${{ github.run_number }}
    path: compliance-report.html
    retention-days: 90

- name: Post to HelixTrack
  env:
    HELIXTRACK_TOKEN: $
    {{ secrets.HELIXTRACK_CI_TOKEN }}
  run: |
    helix-constitution report push \
      --to helixtrack \
      --token "$HELIXTRACK_TOKEN" \
      --project HelixTerminator \
      --run-id "${{ github.run_id }}" \
      --file compliance-report.html

```

Appendix D: Service–Submodule Dependency Graph (Mermaid)

The following Mermaid diagram shows which of the 25 HelixTerminator microservices depend on which submodules. All services depend on `digital.vasic.database`, `digital.vasic.middleware`, and `digital.vasic.observability` (shown as dashed cluster edges for clarity).

```
graph TD
    classDef submod
    fill:#1e3a5f,stroke:#4a9eff,color:#e0f0ff,font-size:12px
    classDef service
    fill:#1a3a1a,stroke:#4aff6a,color:#e0ffe0,font-size:11px
    classDef universal
    fill:#3a1a3a,stroke:#ff4aff,color:#ffe0ff,font-size:11px,stroke-dasharray: 5 5

    %% Universal submodules (all services)
    DB[digital.vasic.database]:::universal
    MW[digital.vasic.middleware]:::universal
    OBS[digital.vasic.observability]:::universal

    %% Submodules
    SEC[digital.vasic.security]:::submod
    AUTH[digital.vasic.auth]:::submod
    CACHE[digital.vasic.cache]:::submod
    MSG[digital.vasic.messaging]:::submod
    RL[digital.vasic.ratelimiter]:::submod
    REC[digital.vasic.recovery]:::submod
    CONC[digital.vasic.concurrency]:::submod
    CONT[digital.vasic.containers]:::submod
    DOCS[digital.vasic.docs_chain]:::submod
    CHAL[digital.vasic.challenges]:::submod
    HQA[helixqa]:::submod
    HT[helixtrack.ru/core]:::submod
    CONST[helix-constitution]:::submod

    %% Services
    GW[gateway]:::service
    AUTHSVC[auth]:::service
    VAULT[vault]:::service
    SSH[ssh-proxy]:::service
    SFTP[sftp-proxy]:::service
    TERM[terminal]:::service
    HOSTMGR[host-manager]:::service
    USER[user]:::service
    TEAM[team]:::service
    WS[workspace-svc]:::service
    AUDIT[audit]:::service
```

```
ANALYTICS[analytics]:::service
NOTIF[notification]:::service
BILLING[billing]:::service
AI[ai]:::service
CHALSVC[challenge]:::service
CB[container-bridge]:::service
HTB[helixtrack-bridge]:::service
SCHED[scheduler]:::service
RBAC[rbac]:::service
SECMGR[secret-manager]:::service
SESSION[session]:::service
SNIPPET[snippet]:::service
WEBHOOK[webhook]:::service
CONFSVC[config-svc]:::service
```

%% Security dependencies

```
SEC --> VAULT
SEC --> AUTHSVC
SEC --> SSH
```

%% Auth dependencies

```
AUTH --> AUTHSVC
AUTH --> GW
```

%% Cache dependencies

```
CACHE --> GW
CACHE --> VAULT
CACHE --> SESSION
CACHE --> HOSTMGR
```

%% Messaging dependencies

```
MSG --> AUDIT
MSG --> ANALYTICS
MSG --> SSH
MSG --> SFTP
```

%% Rate limiter dependencies

```
RL --> GW
RL --> AUTHSVC
RL --> SSH
```

%% Recovery (circuit breaker) dependencies

```
REC --> SSH
REC --> VAULT
REC --> GW
REC --> AUTHSVC
```

%% Concurrency dependencies

```
CONC --> SSH
CONC --> VAULT
CONC --> TERM
```

```
%% Containers dependencies
CONT --> CB
CONT --> SSH

%% Challenges dependencies
CHAL --> CHALSVC
CHAL --> AI
CHAL --> USER

%% HelixTrack dependencies
HT --> HTB

%% Universal submodule connections (sampled – all 25
services use these)
DB --> GW
DB --> AUTHSVC
DB --> VAULT
DB --> SSH
DB --> USER
DB --> TEAM
DB --> WS
DB --> HOSTMGR
DB --> AUDIT
DB --> ANALYTICS
DB --> NOTIF
DB --> BILLING
DB --> AI
DB --> CHALSVC
DB --> CB
DB --> HTB
DB --> SCHED
DB --> RBAC
DB --> SECMGR
DB --> SESSION
DB --> SNIPPET
DB --> WEBHOOK
DB --> CONFSVC
DB --> TERM
DB --> SFTP

MW --> GW
MW --> AUTHSVC
MW --> VAULT
MW --> SSH
MW --> USER
MW --> TEAM

OBS --> GW
OBS --> AUTHSVC
OBS --> VAULT
OBS --> SSH
```

```
OBS -.-> USER
OBS -.-> TEAM
```

Appendix E: docs-chain.yaml for HelixTerminator

```
# docs-chain.yaml
# HelixTerminator specification document dependency graph.
# Managed by digital.vasic.docs_chain v1.0.3.
# Run: docs-chain sync - to regenerate derived outputs
# Run: docs-chain verify - to check consistency
# Run: docs-chain graph - to visualise the dependency DAG

schema_version: "1.1"
project: HelixTerminator
output_dir: docs/generated

documents:
  - id: doc_01_architecture
    path: docs/01_architecture.md
    title: "System Architecture"
    tags: [architecture, overview]
    transforms:
      - id: html_arch
        type: pandoc-html
        output: docs/generated/01_architecture.html
      - id: pdf_arch
        type: weasyprint-pdf
        output: docs/generated/01_architecture.pdf

  - id: doc_02_data_model
    path: docs/02_data_model.md
    title: "Data Model"
    tags: [database, schema]
    depends_on: [doc_01_architecture]
    transforms:
      - id: html_dm
        type: pandoc-html
        output: docs/generated/02_data_model.html
      - id: pdf_dm
        type: weasyprint-pdf
        output: docs/generated/02_data_model.pdf

  - id: doc_03_api_contracts
    path: docs/03_api_contracts.md
    title: "API Contracts"
    tags: [api, openapi, grpc]
```

```
depends_on: [doc_01_architecture, doc_02_data_model]
transforms:
  - id: html_api
    type: pandoc-html
    output: docs/generated/03_api_contracts.html
  - id: docx_api
    type: pandoc-docx
    output: docs/generated/03_api_contracts.docx

- id: doc_04_auth_flows
  path: docs/04_auth_flows.md
  title: "Authentication & Authorisation Flows"
  tags: [auth, security, oauth2]
  depends_on: [doc_01_architecture, doc_03_api_contracts]
  transforms:
    - id: html_auth
      type: pandoc-html
      output: docs/generated/04_auth_flows.html

- id: doc_05_ssh_internals
  path: docs/05_ssh_internals.md
  title: "SSH Proxy Internals"
  tags: [ssh, proxy, sessions]
  depends_on: [doc_01_architecture, doc_02_data_model,
    doc_04_auth_flows]
  transforms:
    - id: html_ssh
      type: pandoc-html
      output: docs/generated/05_ssh_internals.html
    - id: pdf_ssh
      type: weasyprint-pdf
      output: docs/generated/05_ssh_internals.pdf

- id: doc_06_vault_design
  path: docs/06_vault_design.md
  title: "Vault Service Design"
  tags: [vault, encryption, security]
  depends_on: [doc_01_architecture, doc_02_data_model,
    doc_04_auth_flows]
  transforms:
    - id: html_vault
      type: pandoc-html
      output: docs/generated/06_vault_design.html

- id: doc_07_realtime_collab
  path: docs/07_realtime_collab.md
  title: "Real-Time Collaboration"
  tags: [websocket, collaboration, terminal]
  depends_on: [doc_01_architecture, doc_05_ssh_internals]
  transforms:
    - id: html_rt
```

```
    type: pandoc-html
    output: docs/generated/07_realtime_collab.html

- id: doc_08_ai_service
  path: docs/08_ai_service.md
  title: "AI Service Integration"
  tags: [ai, llm, challenges]
  depends_on: [doc_01_architecture, doc_03_api_contracts]
  transforms:
    - id: html_ai
      type: pandoc-html
      output: docs/generated/08_ai_service.html

- id: doc_09_deployment
  path: docs/09_deployment.md
  title: "Deployment & Kubernetes"
  tags: [kubernetes, helm, ci, deployment]
  depends_on: [doc_01_architecture]
  transforms:
    - id: html_deploy
      type: pandoc-html
      output: docs/generated/09_deployment.html
    - id: pdf_deploy
      type: weasyprint-pdf
      output: docs/generated/09_deployment.pdf

- id: doc_10_submodule_integration
  path: docs/10_submodule_integration.md
  title: "Submodule Integration"
  tags: [submodules, integration, vasic-digital, helixqa,
    helixtrack]
  depends_on:
    - doc_01_architecture
    - doc_02_data_model
    - doc_03_api_contracts
    - doc_04_auth_flows
    - doc_05_ssh_internals
    - doc_06_vault_design
    - doc_07_realtime_collab
    - doc_08_ai_service
    - doc_09_deployment
  transforms:
    - id: html_sub
      type: pandoc-html
      output: docs/generated/10_submodule_integration.html
      options:
        toc: true
        toc_depth: 3
        syntax_highlight: pygments
    - id: pdf_sub
      type: weasyprint-pdf
```

```

    output: docs/generated/10_submodule_integration.pdf
    options:
      css: docs/styles/helix-pdf.css
  - id: docx_sub
    type: pandoc-docx
    output: docs/generated/10_submodule_integration.docx

- id: doc_11_runbooks
  path: docs/11_runbooks.md
  title: "Operations Runbooks"
  tags: [operations, runbooks, sre]
  depends_on:
    - doc_01_architecture
    - doc_09_deployment
  transforms:
    - id: html_run
      type: pandoc-html
      output: docs/generated/11_runbooks.html
    - id: pdf_run
      type: weasyprint-pdf
      output: docs/generated/11_runbooks.pdf

consistency_rules:
  - rule: no_broken_internal_links

      description: All [[doc_id]] cross-references must
        resolve to a registered document.
  - rule: depends_on_must_exist
    description: Every document in depends_on must be
      registered.
  - rule: no_circular_dependencies
    description: The document dependency graph must be a DAG
      (no cycles).
  - rule: output_paths_unique
    description: No two transforms may write to the same
      output path.
  - rule: all_source_files_exist
    description: The path for each document must exist on
      disk.

ci:
  fail_on_broken_links: true
  fail_on_circular_deps: true
  fail_on_missing_source: true
  verify_command: "docs-chain verify --strict"
  sync_command: "docs-chain sync --parallel"
  graph_command: "docs-chain graph --format svg --output
    docs/generated/dep-graph.svg"

```

Appendix F: Integration Testing Matrix

The following table defines the mandatory integration test suites that exercise cross-submodule behaviour. All suites run in CI on every PR to `main`.

Test Suite	Submodules Exercised	Test File
Vault encrypt/decrypt round-trip	security , database , cache	services/vault/integration_test.go
Key rotation end-to-end	security , messaging , observability	services/vault/key_rotation_integration_test.go
JWT issuance and validation	auth , cache , middleware	services/auth/jwt_integration_test.go
OIDC federation (Okta, Azure AD)	auth , middleware , observability	services/auth/oidc_integration_test.go
SSH session lifecycle	auth , messaging , concurrency , ratelimiter	services/ssh-proxy/session_integration_test.go
Rate limiter under load	ratelimiter , cache , observability	services/gateway/ratelimit_integration_test.go
Circuit breaker open/half-open/close	recovery , observability	pkg/recovery/circuit_breaker_integration_test.go
Kafka audit event round-trip	messaging , database , observability	services/audit/kafka_integration_test.go
Container exec via Container Bridge	containers , auth , middleware	services/container-bridge/exec_integration_test.go
HelixTrack task association	helixtrack.ru/core , auth , database	services/helixtrack-bridge/task_integration_test.go
docs_chain DAG consistency	docs_chain	ci/docs_chain_integration_test.go

Test Suite	Submodules Exercised	Test File
Challenge lifecycle (create→attempt→score)	challenges , database , messaging	services/challenge/lifecycle_integration_test.go
SCIM 2.0 provisioning	auth , database , messaging	services/auth/scim_integration_test.go
Worker pool under goroutine pressure	concurrency , observability	pkg/concurrency/workerpool_integration_test.go
Cache warm-on-startup	cache , database	pkg/cache/warm_integration_test.go

Appendix G: Error Code Registry

Each submodule defines a canonical set of error codes used across HelixTerminator. Services must map submodule errors to appropriate HTTP/gRPC status codes.

```
// File: pkg/errors/submodule_errors.go
package errors

import (
    "errors"

    vasicauth    "digital.vasic.auth/errors"
    vasicdb      "digital.vasic.database/errors"
    vasiccache   "digital.vasic.cache/errors"
    vasicsec     "digital.vasic.security/errors"
    vasicmsg     "digital.vasic.messaging/errors"
    vasicrl      "digital.vasic.ratelimiter/errors"
    vasicrec     "digital.vasic.recovery/errors"
    vasiccont    "digital.vasic.containers/errors"
    vasicconc    "digital.vasic.concurrency/errors"
    chal         "digital.vasic.challenges/errors"
    htcore       "helixtrack.ru/core/errors"

    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/status"
)

// SubmoduleErrorToGRPC maps known submodule errors to gRPC
// status codes.
```

```

// This function is used by all service gRPC interceptors.
func SubmoduleErrorToGRPC(err error) error {
    if err == nil {
        return nil
    }

    switch {
    // Auth errors
    case errors.Is(err, vasicauth.ErrTokenExpired):
        return status.Error(codes.Unauthenticated, "token
            expired")
    case errors.Is(err, vasicauth.ErrTokenInvalid):
        return status.Error(codes.Unauthenticated, "token
            invalid")
    case errors.Is(err, vasicauth.ErrInsufficientScope):
        return status.Error(codes.PermissionDenied,
            "insufficient scope")
    case errors.Is(err, vasicauth.ErrProviderUnavailable):
        return status.Error(codes.Unavailable, "identity
            provider unavailable")

    // Database errors
    case errors.Is(err, vasicdb.ErrNotFound):
        return status.Error(codes.NotFound, "resource not
            found")
    case errors.Is(err, vasicdb.ErrDuplicateKey):
        return status.Error(codes.AlreadyExists, "resource
            already exists")
    case errors.Is(err, vasicdb.ErrConnectionFailed):
        return status.Error(codes.Unavailable, "database
            unavailable")
    case errors.Is(err, vasicdb.ErrTransactionConflict):
        return status.Error(codes.Aborted, "transaction
            conflict, retry")

    // Cache errors
    case errors.Is(err, vasiccache.ErrCacheMiss):
        return nil // cache miss is not an error in gRPC
            context; caller handles
    case errors.Is(err, vasiccache.ErrConnectionFailed):
        return status.Error(codes.Unavailable, "cache
            unavailable")

    // Security errors
    case errors.Is(err, vasicsec.ErrKeyNotFound):
        return status.Error(codes.NotFound, "encryption key
            not found")
    case errors.Is(err, vasicsec.ErrDecryptionFailed):
        return status.Error(codes.Internal, "decryption
            failed")
    case errors.Is(err, vasicsec.ErrKeyRotationInProgress):

```

```
        return status.Error(codes.Unavailable, "key rotation
        in progress, retry later")

// Messaging errors
case errors.Is(err, vasicmsg.ErrPublishFailed):
    return status.Error(codes.Internal, "message publish
    failed")
case errors.Is(err, vasicmsg.ErrSchemaValidation):
    return status.Error(codes.InvalidArgument, "message
    schema validation failed")
case errors.Is(err, vasicmsg.ErrBrokerUnavailable):
    return status.Error(codes.Unavailable, "message
    broker unavailable")

// Rate limiter errors
case errors.Is(err, vasicrl.ErrRateLimitExceeded):
    return status.Error(codes.ResourceExhausted, "rate
    limit exceeded")

// Recovery errors
case errors.Is(err, vasicrec.ErrCircuitOpen):
    return status.Error(codes.Unavailable, "circuit
    breaker open")
case errors.Is(err, vasicrec.ErrBulkheadFull):
    return status.Error(codes.ResourceExhausted,
    "bulkhead capacity exceeded")

// Container errors
case errors.Is(err, vasiccont.ErrContainerNotFound):
    return status.Error(codes.NotFound, "container not
    found")
case errors.Is(err, vasiccont.ErrRuntimeUnavailable):
    return status.Error(codes.Unavailable, "container
    runtime unavailable")
case errors.Is(err, vasiccont.ErrExecFailed):
    return status.Error(codes.Internal, "container exec
    failed")

// Concurrency errors
case errors.Is(err, vasicconc.ErrPoolExhausted):
    return status.Error(codes.ResourceExhausted, "worker
    pool exhausted")
case errors.Is(err, vasicconc.ErrSemaphoreTimeout):
    return status.Error(codes.DeadlineExceeded,
    "semaphore acquisition timeout")

// Challenges errors
case errors.Is(err, chal.ErrChallengeNotFound):
    return status.Error(codes.NotFound, "challenge not
    found")
case errors.Is(err, chal.ErrAlreadyCompleted):
```

```

        return status.Error(codes.AlreadyExists, "challenge
        already completed")
    case errors.Is(err, chal.ErrGenerationFailed):
        return status.Error(codes.Internal, "challenge
        generation failed")

    // HelixTrack errors
    case errors.Is(err, htcore.ErrTaskNotFound):
        return status.Error(codes.NotFound, "helixtrack task
        not found")
    case errors.Is(err, htcore.ErrProjectNotFound):
        return status.Error(codes.NotFound, "helixtrack
        project not found")
    case errors.Is(err, htcore.ErrAPIUnavailable):
        return status.Error(codes.Unavailable, "helixtrack
        api unavailable")

    default:
        return status.Error(codes.Internal, "internal error")
    }
}

// SubmoduleErrorToHTTP maps known submodule errors to HTTP
// status codes.
// Used by Gin handlers that cannot propagate gRPC status.
func SubmoduleErrorToHTTP(err error) int {
    if err == nil {
        return 200
    }
    grpcErr := SubmoduleErrorToGRPC(err)
    st, _ := status.FromError(grpcErr)
    switch st.Code() {
    case codes.NotFound:           return 404
    case codes.AlreadyExists:      return 409
    case codes.InvalidArgument:    return 400
    case codes.Unauthenticated:    return 401
    case codes.PermissionDenied:   return 403
    case codes.ResourceExhausted:  return 429
    case codes.Unavailable:        return 503
    case codes.DeadlineExceeded:   return 504
    case codes.Aborted:            return 409
    default:                       return 500
    }
}

```

Appendix H: Submodule Version Compatibility Matrix

The following matrix specifies the minimum and maximum tested versions of each submodule against HelixTerminator Go 1.25.

Submodule	Min Version	Max Tested	Breaking Change Version	Notes
<code>digital.vasic.security</code>	v1.3.0	v1.4.2	v2.0.0 (planned)	v1.4.0 added AES-256-GCM-SIV support
<code>digital.vasic.auth</code>	v2.0.0	v2.3.1	v3.0.0 (planned)	v2.2.0 added SCIM 2.0
<code>digital.vasic.cache</code>	v1.1.0	v1.2.7	—	v1.2.0 added L2 cluster mode
<code>digital.vasic.database</code>	v1.6.0	v1.8.0	—	v1.7.0 added read-replica routing
<code>digital.vasic.messaging</code>	v1.4.0	v1.5.3	—	v1.5.0 added Kafka schema registry
<code>digital.vasic.middleware</code>	v1.0.0	v1.1.4	—	Stable API
<code>digital.vasic.observability</code>	v1.2.0	v1.3.0	—	v1.3.0 upgraded to OTEL SDK 1.28
<code>digital.vasic.ratelimiter</code>	v1.0.0	v1.0.9	—	v1.0.5 added dynamic adjustment

Submodule	Min Version	Max Tested	Breaking Change Version	Notes
<code>digital.vasic.recovery</code>	v1.1.0	v1.2.1	—	v1.2.0 added bulkhead pattern
<code>digital.vasic.concurrency</code>	v1.0.0	v1.1.0	—	v1.1.0 added errgroup helpers
<code>digital.vasic.containers</code>	v1.0.0	v1.0.5	—	v1.0.3 added Podman support
<code>digital.vasic.docs_chain</code>	v1.0.0	v1.0.3	—	CI-only dependency
<code>digital.vasic.challenges</code>	v0.8.0	v0.9.2	v1.0.0 (pending)	Pre-stable; API may change
<code>helixqa</code>	v1.5.0	v1.7.0	—	v1.6.0 added mutation testing
<code>helixtrack.ru/core</code>	v2.0.0	v2.1.4	—	v2.1.0 added real-time webhooks
<code>helix-constitution</code>	v2.0.0	v2.0.0	—	Major version locked per policy

Appendix I: Common Integration Pitfalls

I.1 `digital.vasic.security` — Key Rotation Race

Problem: If the Vault Service starts key rotation while another goroutine is mid-encryption, `ErrKeyRotationInProgress` is returned. Callers that do not check for this sentinel and retry will corrupt vault entries.

Solution:

```
// Always wrap encryption in a retry loop that checks for
// rotation.
for attempt := 0; attempt < 3; attempt++ {
    ciphertext, err = sec.Encrypt(ctx, keyID, plaintext)
    if errors.Is(err, vasicsec.ErrKeyRotationInProgress) {
        time.Sleep(time.Duration(attempt+1) * 100 *
            time.Millisecond)
        continue
    }
    break
}
```

I.2 `digital.vasic.auth` — Token Clock Skew

Problem: JWT validation fails on services whose system clocks drift more than 30 seconds relative to the Auth Service.

Solution: All Kubernetes pods must sync time via the node's NTP daemon. The `digital.vasic.auth` JWT validator accepts a configurable `ClockSkewTolerance` (default 30s). Set to 60s only in testing environments:

```
validator :=
    vasicauth.NewJWTValidator(vasicauth.JWTValidatorConfig{
        JWKEndpoint:      cfg.Auth.JWKEndpoint,
        ClockSkewTolerance: 30 * time.Second, // never increase
        // in production
    })
```

I.3 `digital.vasic.cache` — L2 Stampede on Cold Start

Problem: On service restart, the L1 (in-process) cache is empty. If many requests arrive simultaneously, all miss L2 (Redis) and hit the database, causing a thundering herd.

Solution: Use the `SingleFlight` wrapper provided by `digital.vasic.cache`:

```
cacheManager := vasiccache.NewManager(vasiccache.Config{
    L1MaxSize: 1000,
    L2Client:  redisClient,
    SingleFlight: true, // deduplicate concurrent misses
})
```

1.4 digital.vasic.database – Unheld Transaction Connections

Problem: If a goroutine acquires a `*sql.Tx` but the context is cancelled before `Commit` or `Rollback`, the connection leaks back to the pool in an unknown state.

Solution: Always defer rollback before any early return:

```
tx, err := db.Begin(ctx)
if err != nil { return err }
defer func() {
    if p := recover(); p != nil {
        _ = tx.Rollback()
        panic(p)
    }
    if err != nil {
        _ = tx.Rollback()
    }
}()
// ... work ...
return tx.Commit()
```

The `digital.vasic.database WithTransaction` helper does this automatically – always prefer it over manual transaction management.

1.5 digital.vasic.messaging – Kafka Producer RequiredAcks

Problem: Using `RequiredAcks: AcksNone` on the Kafka producer gives highest throughput but risks message loss on broker restart.

Solution: HelixTerminator mandates `RequiredAcks: AcksAll` with `MinInSyncReplicas: 2` for all audit and compliance topics. Analytics topics may use `AcksLeader`.

1.6 digital.vasic.recovery – Circuit Breaker State Not Shared

Problem: Each pod maintains its own circuit breaker state. A failing downstream service causes the circuit to open independently per pod, leading to inconsistent behaviour during rolling restarts.

Solution: This is expected and correct for most cases. For services where global open state is required, use the Redis-backed distributed circuit breaker:

```
cb := vasicrec.NewCircuitBreaker(vasicrec.Config{
    Name:        "vault-db",
    Backend:     vasicrec.BackendRedis,
    RedisClient: redisClient,
    Threshold:   5,
    Timeout:     30 * time.Second,
    HalfOpenProbes: 2,
})
```

I.7 digital.vasic.concurrency – Worker Pool Blocking on Shutdown

Problem: If the worker pool's `Stop()` is not called before application shutdown, in-flight tasks may be abandoned and goroutines leaked.

Solution: Always register the pool with the application lifecycle manager:

```
pool := vasicconc.NewWorkerPool(vasicconc.Config{MaxWorkers:
    50})
lifecycle.RegisterShutdown(func(ctx context.Context) error {
    return pool.Stop(ctx)
})
```

I.8 helixqa – Flaky Tests Blocking CI

Problem: Some integration tests (especially those involving SSH session timing) are inherently flaky. Without quarantine, they block PRs.

Solution: Add `@helixqa:quarantine` annotation to the test function and open a `flaky-test` issue. The quarantine annotation excludes the test from blocking CI while keeping it in the test suite for diagnostic runs:

```
//helixqa:quarantine flaky-test-id=HT-1234 reason="SSH
    session timing sensitive"
func TestSSHSessionHandshakeTimeout(t *testing.T) {
    // ...
}
```

*End of HelixTerminator Submodule Integration Specification Document ID:
doc_10_submodule_integration Version: 1.0.0 Last Updated: 2026-06-28 Governed
by: HelixConstitution v2.0 §11.4.31*